

Canon

X-07

PROGRAMMATION POUR DEBUTANTS

Canon

Canon

X-07

**PROGRAMMATION
POUR
DEBUTANTS**

ORDINATEUR INDIVIDUEL



ORDINATEUR INDIVIDUEL

X-07

*PROGRAMMATION
POUR DEBUTANTS*



Ce manuel est un guide de programmation destiné à ceux désirant utiliser librement le X-07. Concernant la manipulation du clavier et la manière d'entrer un programme, prière de se reporter au guide de l'utilisateur.

La programmation consiste à traduire sous forme assimilable pour l'ordinateur une tâche qu'on désire exécuter.

Par exemple, nous expliquerons ici comment faire comprendre à l'ordinateur "Je vais lire les reçus, donnez-moi le total" et "reproduire ce passage musical".

CANON décline toute responsabilité quant à l'utilisation des exemples de programmes donnés dans ce manuel et ne saurait être engagé pour tout préjudice financier, manque à gagner et autres réclamations de tiers.

Table des matières

1. Création d'un programme	
1.1 Qu'est-ce qu'un programme	3
1.2 Organigramme	5
1.3 Qu'est-ce qu'un bon programme	9
1.4 Comment sauvegarder un programme	12
2. Bases de la programmation	
2.1 Les instructions fondamentales	17
2.2 Lecture de données à partir du clavier	20
2.3 Changement de route de contrôle	25
2.4 Prise de décision	28
2.5 Traitement de boucles	31
2.6 Lecture de données dans un programme	33
2.7 Tableaux de variables	37
A. Variables indicées	
B. Tableaux	
2.8 Traitement des chaînes de caractères	46
A. Liaison de caractères	
B. Lecture de chaînes de caractères	
C. Fonctions traitant des chaînes de caractères	
2.9 Définition de fonctions	54
2.10 Sous-programmes	56
3. Programmes pratiques	
3.1 Liste de chiffre d'affaire	63
3.2 Remboursements égaux de capital et d'intérêt	70
3.3 Marque d'un concours de golf	74
3.4 Gestion d'un journal	81
A. Comment créer un fichier de journal	
B. Calcul de statistiques à partir de journaux	
3.5 Annuaire de téléphone	91
A. Comment créer un annuaire téléphonique	
B. Comment lire un annuaire téléphonique	
C. Comment retirer des données d'un annuaire téléphonique	
3.6 Gestion d'inventaire	100

4. Programme tirant le meilleur parti des caractéristiques du X-07	
4.1 Affichage de caractères	111
A. Horloge-réveil numérique	
B. Vidage de mémoire	
C. Image miroir	
D. Fixation d'un caractère	
4.2 Reproduction d'un passage musical	128
A. Quand trois poules vont aux champs	
B. Clavier	
4.3 Affichage graphique	136
A. Axes de coordonnées et points	
B. Déplacement de points	
C. Tracé de cercles	
4.4 Imprimante	143
A. Impression de caractères	
B. Impression graphique de caractères	
C. Tracé d'un motif	
4.5 Coupleur optique	152
A. Transmission de valeurs numériques	
B. Transmission de caractères	
C. Transmission de fichiers	
D. Dialogue	
5. Programme de jeu	
5.1 Résumé du jeu	165
5.2 Explication du jeu	171
A. Programme principal	
B. Sous-programmes	
C. Liste du jeu	
5.3 Exécution	195
6. Localisation des défauts	
6.1 Boucle sans fin	203
6.2 Problèmes d'affichage	205
6.3 Valeurs numériques	208
6.4 Autres erreurs difficiles à déceler	210

Conseils

Un programme sans but défini	4
Faisce que je fais	11
FOR = I?	45
Programmes pratiques	90
Identité d'un ordinateur	120
Fichier	159
Comment indiquer clairement les ambiguïtés	194



INDEX

1 Création d'un programme



2 Bases de la programmation



3 Programmes pratiques



4 Programme tirant le meilleur parti des caractéristiques du X-07



5 Programme de jeu



6 Localisation des défauts



Chapitre 1

Création d'un programme



Ce chapitre concerne ceux qui n'ont aucune expérience dans la rédaction de programmes en BASIC. Il décrit les concepts de base d'un programme et les points à se rappeler lors de la rédaction d'un programme en BASIC.

Si vous préférez directement passer à la théorie de programmation, vous pouvez sauter ce chapitre et passer directement au chapitre 2. Dans ce cas, veuillez revenir plus tard à ce chapitre et le lire attentivement.



1.1 Qu'est-ce qu'un programme

Comme nous l'avons vu dans le guide de l'utilisateur, un programme est une série d'instructions comportant des numéros de ligne. Il faut cependant que ces instructions soient correctement structurées pour qu'elles puissent accomplir une tâche spécifique. Le processus qui consiste à rédiger un programme est appelé "programmation".

Voici un exemple de programme.

```
10 I=INT(RND(0)*100)
20 INPUT "NOMBRE";NUM
30 IF I=NUM THEN 100
40 IF I>NUM THEN PRINT "TROP PETIT"
50 IF I<NUM THEN PRINT "TROP GRAND"
60 GOTO 20
100 PRINT "TU AS GAGNE"
110 END
```

Ce programme est un jeu qui consiste à deviner un nombre entier de 0 à 9 produit par l'ordinateur. Ne vous inquiétez pas si vous ne comprenez pas la signification de chaque instruction. Vous la comprendrez progressivement au fur et à mesure que vous avancerez dans ce manuel. Il vous suffit pour l'instant de sentir ce que c'est qu'un programme.

Composez et exécutez le programme ci-dessus. Nous espérons que l'expérience de la programmation vous fera plaisir.

Nota: Si vous ne savez pas comment introduire ce programme ou si une erreur se produit, reportez-vous à 1.4 "PROGRAMMATION EN BASIC" du guide de l'utilisateur.

Un programme sans buts définis

Lors de l'apprentissage de la programmation en BASIC, la chose la plus difficile n'est pas la syntaxe concrète du langage BASIC, mais plutôt de définir de façon claire les buts du programme. Par exemple, bien que vous connaissiez la grammaire et le vocabulaire de votre propre langue, si quelqu'un vous demandait à brûle pourpoint "Dites quelque chose dans votre langue", vous vous sentiriez probablement emprunté. S'il vous venait quelque chose à l'esprit, vous diriez probablement une banalité du genre "Il fait beau aujourd'hui". Par contre, si l'on vous demandait quelque chose de défini, par exemple "Comment dites-vous "Bonjour" dans votre langue?", il vous serait plus facile de parler. Ceci s'applique également à une langue étrangère, telle le BASIC. Il est très difficile d'apprendre et de maîtriser le langage BASIC si l'on n'a pas défini ce qu'on désire exprimer dans ce langage. La manière la plus rapide d'apprendre une langue est de tenter de remplacer quelques mots par d'autres dans une phrase simple. Ceci s'applique également au BASIC. La voie la plus rapide est de changer des parties de programmes que quelqu'un d'autre a créés.

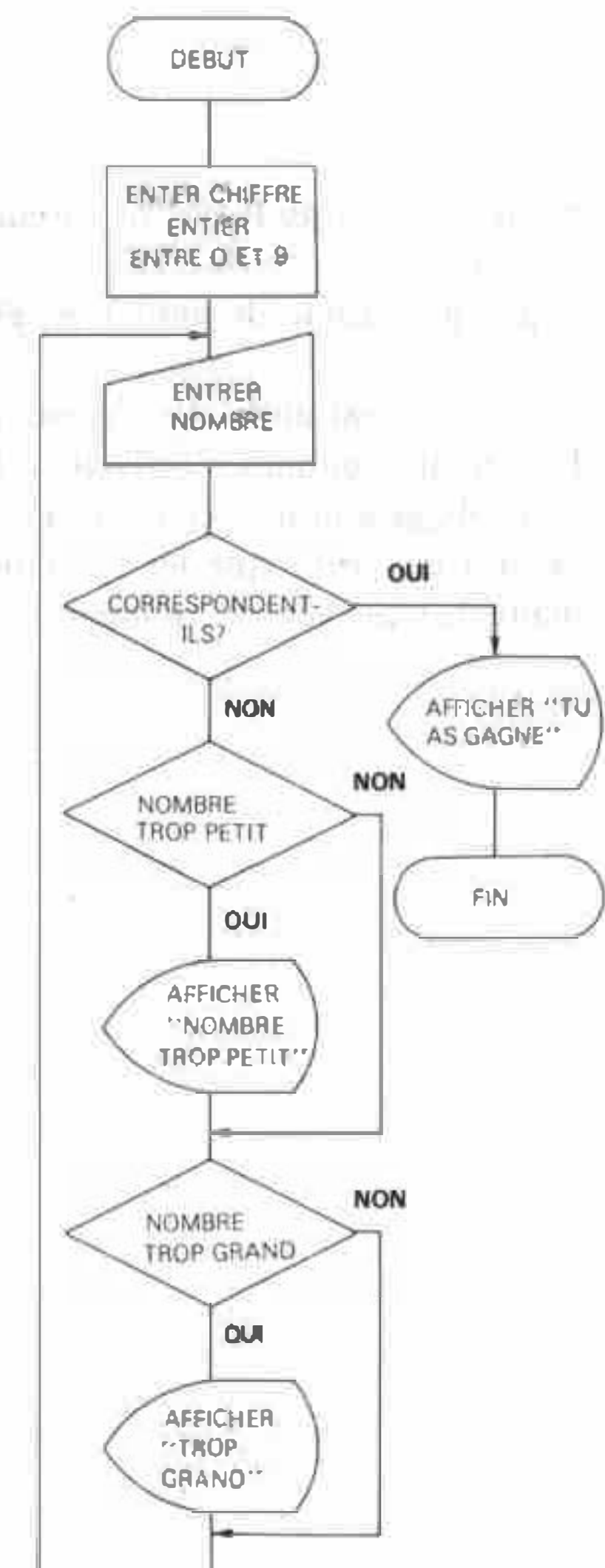


1.2 Organigramme

Il est difficile de comprendre la séquence générale et l'objet d'un programme rédigé en langage BASIC d'après la liste de son contenu. Plus le programme est long, plus il est difficile de le comprendre. Par exemple, il est difficile de saisir les grandes lignes d'un programme comportant 500 ou 1000 pas en regardant simplement sa liste.

Il est plus facile de comprendre un programme si l'on trace le schéma de sa séquence d'événements; un tel schéma est appelé "organigramme".

Le schéma ci-contre est l'organigramme du programme de la section précédente. En rédigeant un tel programme, vous dégagerez plus facilement son but, et la manière par laquelle il se propose de l'atteindre.

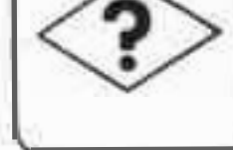


On rédige un organigramme pour deux raisons:

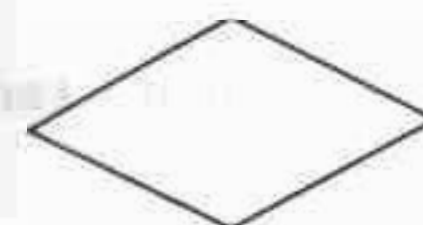
1. Pour éclaircir le suivi du contrôle avant de rédiger le programme; et
2. Pour permettre à une autre personne de comprendre votre programme, ou pour en conserver un résumé.

Nous vous conseillons de prendre l'habitude de tracer un organigramme avant de rédiger tout programme. Vous vous imaginez qu'il vous est possible de garder en tête la logique d'un programme; mais s'il est très long, vous finirez par oublier la logique de votre programme. C'est alors que l'organigramme se révèle précieux.

Les organigrammes s'écrivent à l'aide de quelques symboles; rien ne vous oblige à utiliser ceux-ci, mais ce sont les symboles standard. Cette section ne décrit que les symboles principaux, et dans la suite de ce manuel, vous verrez comment ils sont utilisés pratiquement.



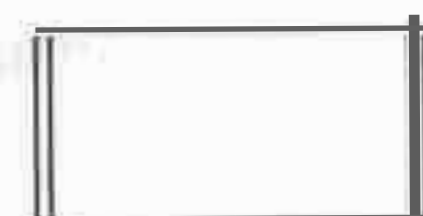
Procédé: Indique diverses fonctions de traitement



Décision: Signifie un branchement en fonction d'une condition imposée



Préparation: Indique divers réglages initiaux



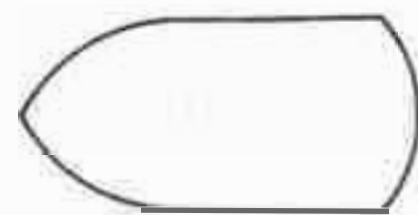
Procédé défini: Indique un sous-programme, etc.



Entrée/sortie: Indique une fonction d'entrée ou sortie



Entrée manuelle: Indique l'entrée manuelle de données sur le clavier, etc.



Affichage: Indique un affichage sur écran



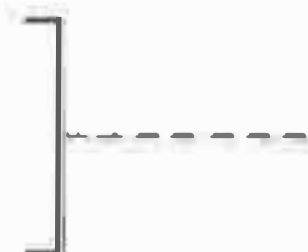
Impression: Indique la sortie vers imprimante, etc.



Élément de liaison: Indique une sortie vers ou une entrée d'un autre endroit de l'organigramme.



Terminal: Indique: début, fin, arrêt, retard, suspension, etc.



Remarque: Utilisé pour insérer des explications.



1.3 Qu'est-ce qu'un bon programme

Pour atteindre un même but, un programme peut être rédigé de nombreuses manières. par exemple, le programme précédent peut être modifié, comme indiqué ci-dessous, et tout de même réaliser le même résultat.

```
10 I=RND(0)*10
20 INPUT "NOMBRE";NUM
30 IF I=NUM THEN PRINT "TU AS GAGNE":END
40 IF I>NUM THEN PRINT "TROP PETIT"
50 IF I<NUM THEN PRINT "TROP GRAND"
60 GOTO 20
```

Mais vous vous trompez si vous pensez qu'un programme est aussi bon qu'un autre si les mêmes résultats sont obtenus. Il existe une grande différence de vitesse d'exécution et de longueur du programme. Oui, il existe des bons programmes et de mauvais programmes.

Si vous allez créer un programme, autant en créer un bon. Mais qu'est-ce qu'un bon programme? Voici une liste des caractéristiques d'un bon programme.

1. Il réalise avec précision les résultats désirés.
2. Il est facile à comprendre.
3. Il est facile à corriger.
4. Sa vitesse d'exécution est élevée.
5. Son emploi est facile.
6. Il est compact (pas trop dispersé).

Afin de satisfaire à ces exigences, gardez ces points à l'esprit lorsque vous rédigez un programme.

1. L'exigence minimale d'un programme est de réaliser les résultats désirés.
2. Il est important de rédiger un programme dans un style à facile à comprendre. La simplicité favorise la facilité de correction (exigence 3). Pour faciliter sa compréhension, insérez des remarques, harmonisez le suivi du contrôle, etc.

3. Pour que la correction du programme soit aisée, évitez d'utiliser trop de lignes multi-instructions. Structurez votre programme de sorte que lors d'une correction cela n'affecte pas d'autres parties du programme.
4. Pour réduire le temps d'exécution, écrivez des parties du programme dont le temps d'exécution est long en langage machine. Etudiez l'algorithme de traitement de boucles pour éviter le gaspillage; fixez les variables utilisées souvent dès le début; finalement, utilisez des variables entières et des multi-instructions.
5. Lorsque vous créez le programme, imaginez la manière dont il sera exécuté. Demandez à quelqu'un d'exécuter votre programme et demandez conseil.
6. Pour harmoniser vos programmes, utilisez des sous-programmes. Cependant, évitez de rendre le programme trop difficile à comprendre ou trop complexe dans le seul but de le raccourcir.

Pour résumer les points ci-dessus:

1. **Fixez clairement ce que vous voulez introduire et obtenir.**
2. **Regardez le déroulement du programme dans son ensemble, et non par parties.**
3. **Prenez votre temps pour rédiger un programme.**



Fais ce que je fais

Comparativement à l'anglais, au français ou à l'allemand, le langage BASIC possède beaucoup moins de règles de grammaire et un vocabulaire beaucoup moins étendu. En l'étudiant avec bonne volonté, vous pourrez le maîtriser en trois jours. Toutefois, en programmation réelle, il vous apparaîtra la chose suivante: "Seuls les mots composés sur le clavier comptent, les intentions n'ont aucune signification pour l'ordinateur." Les insinuations n'existent pas; tout doit être spécifié correctement de A jusqu'à Z. Un mot sans insinuation semble très artificiel aux humains. Cependant, un programme se compose d'instructions destinées à un ordinateur. Si les instructions sont ambiguës, les résultats peuvent être inattendus. Par exemple:

Instruction: "Fais ce que je fais"

Résultat: "Fais ce que je fais"

Instruction: "Non, pas encore"

Résultat: "Non pas encore"

Instruction: "Cesse de répéter ce que je dis"

Résultat: "Cesse de répéter ce que je dis"



...C'est une routine de comédie. Il n'y a pas d'instruction telle que "Fais ce que je fais" en langage BASIC. S'il en existait une, un programme rédigé dans la manière du langage BASIC pour humains ressemblerait à ceci:

Instruction: "Quand j'aurais dit départ, écoute-moi. Si la phrase "Cesse de répéter ce que je dis" n'est pas incluse, répète ce que je dis. Si elle est incluse, cesse de répéter ce que je dis et écoute ma prochaine instruction. Départ."

1.4 Comment sauvegarder un programme

Un programme peut être sauvegardé en mémoire vive (RAM) ou sur une cassette.

Si le programme est court, ou si vous désirez le conserver temporairement, utilisez la mémoire vive (RAM). D'autre part, si vous désirez le conserver à long terme, utilisez une bande cassette.

La zone de la mémoire (RAM) utilisée pour sauvegarder des programmes est appelée "mémoire RAM", et l'ordre FSET est utilisé pour réserver une zone en mémoire. En conséquence, pour sauvegarder des programmes, il faut d'abord exécuter l'instruction FSET.

Exemple:

Composez **FSET 4000** (réservation de 4000 octets)

Ensuite **SAVE "désignation de fichier"**

A présent, un programme portant la désignation de fichier spécifiée peut être sauvegardé en mémoire RAM. Pour rappeler le programme ainsi sauvegardé, composez

LOAD "désignation de fichier"

De même, à l'aide d'un ordre DIR, le contenu de la mémoire RAM peut être affiché. Par exemple:

```
RAM
PTEL  P  DATA 1  D
4000/3256
≥
```

L'affichage montre que la mémoire RAM contient un programme appelé "PTEL" et un fichier de données appelé "DATA 1", et qu'il reste 3256 octets libres dans la mémoire RAM. Pour plus de détails, se reporter à "1.5 Mémoire du X-07" du guide de l'utilisateur.

Pour sauvegarder sur ou rappeler un programme d'une bande cassette, on utilise les ordres suivants:

CSAVE "désignation de fichier"

CLOAD "désignation de fichier"

Chapitre 2

Bases de la programmation



Ce chapitre donne quelques exemples de programmes pour vous aider à assimiler les bases de la programmation et montrer comment utiliser les instructions fondamentales.



2.1 Les instructions fondamentales

Le programme suivant calcule et affiche les ventes totales pour une quantité et un prix donnés.

```
10 REM VENTES TOTALES
20 LET PRIX=50
30 LET NUM=8
40 LET SOM=PRIX*NUM
50 PPINT SOM
60 END
```

Explication de ce programme:

10 REM VENTES TOTALES

L'instruction REM sert à insérer une remarque pour identifier ou éclaircir le programme. Cette instruction n'est pas exécutée. Après le mot "REM", on peut écrire n'importe quoi. Utilisez les instructions REM pour vous remémorer un programme ou une partie de celui-ci. L'apostrophe (') est également utilisable à la place du mot d'instruction "REM".

```
10 ' VENTES TOTALES
20 LET PRIX=50
30 LET NUM=8
40 LET SOM=PRIX*NUM
```

Ce sont des "instructions d'affectation". En d'autres termes, à la ligne 20, la valeur 50 est affectée à la variable PRIX; à la ligne 30, la

valeur 8 est affectée à la variable NUM, et à la ligne 40, le produit du contenu des variables PRIX et NUM et attribué à la variable SOM. Dans une instruction d'affectation, le mot d'instruction "LET" peut être omis. De cette manière, les instructions ci-dessus deviennent:

```
20 PRIX = 50
30 NUM = 8
40 SOM = PRIX * NUM
```

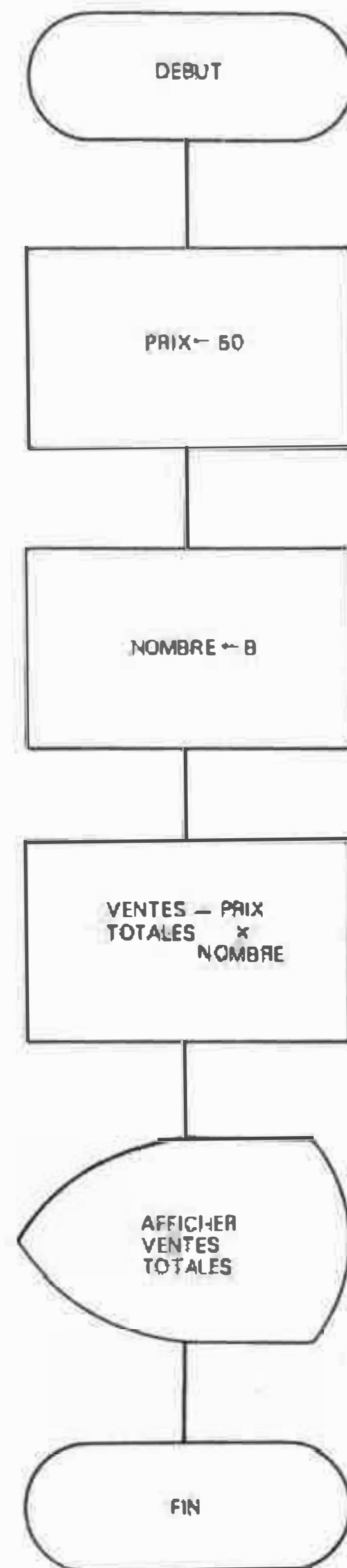
Il est plus facile d'exprimer le programme sous la forme ci-dessus, de sorte que le mot d'instruction "LET" est souvent omis.

50 PRINT SOM

L'instruction PRINT affiche le contenu de la variable SOM sur l'affichage. Il est possible d'utiliser un point d'interrogation (?) à la place du mot d'instruction "PRINT". De cette manière, la ligne 50 devient:

```
50 ?SOM
60 END
```

L'instruction END achève l'exécution du programme. Cependant, l'instruction END peut être omise à la fin d'un programme. Dans l'exemple ci-dessus, le programme



se termine même sans la ligne 60.

Exécutons maintenant ce programme, et si tout est en ordre, on obtiendra l'affichage suivant.

```
RUN
400
=
```

Nouvelles instructions

REM, '	Instruction de remarque
LET	Instruction d'affectation
PRINT, ?	Affiche les données sur l'écran
END	Achève l'exécution d'un programme



2.2 Lecture de données à partir du clavier

Dans le programme précédent, le prix et la quantité ont été fixés à l'avance à 50 et 8 respectivement. Modifions maintenant ce programme pour que des valeurs quelconques puissent être calculées.

```
10 REM VENTES TOTALES
20 INPUT "PRIX";PRIX
30 INPUT "NOMBRE";NUM
40 LET SOM=PRIX*NUM
50 PRINT "TOTAL";SOM
60 END
```

La différence entre ce programme et le précédent réside dans les lignes 20 et 30 qui ont été changées comme suit:

```
20 INPUT PRIX
30 INPUT NUM
```

L'instruction INPUT lit les données introduites au moyen du clavier et les affecte aux variables.



Maintenant, exécutons ce programme.

RUN

? _

Lorsque l'ordre RUN est entré, un point d'interrogation (?) est affiché à la première position, et le curseur apparaît à la troisième position en clignotant. Ceci indique que le X-07 demande l'introduction d'une donnée sur le clavier. Composez 60

RUN

? 60

? _

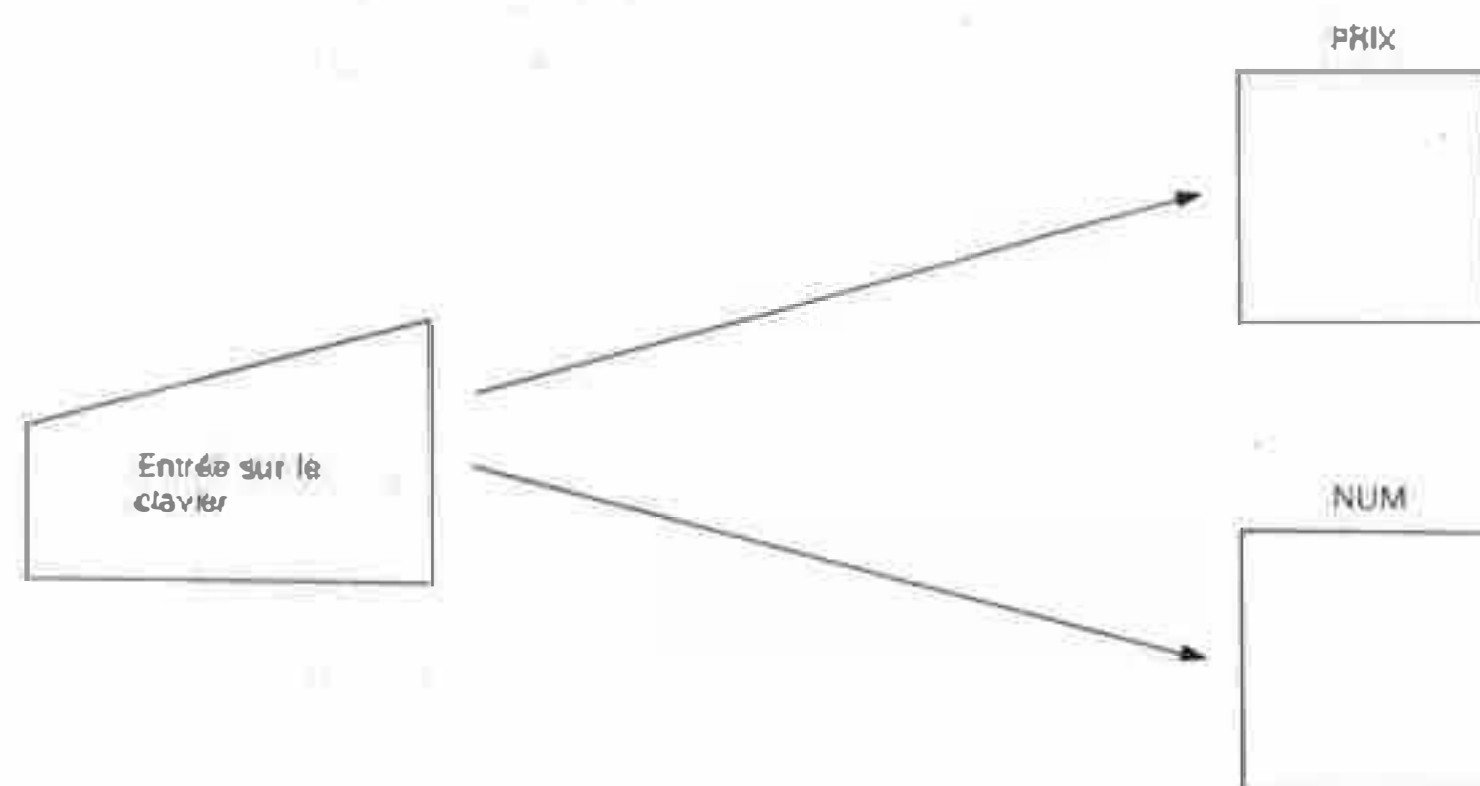
"?" et le curseur apparaissent à nouveau. Le premier "?" a été affiché par l'exécution de l'instruction INPUT de la ligne 20, et le second "?" par l'exécution de la ligne 30. Composez 4 cette fois.

```

? 60
? 4
240
=

```

Dans cet exemple, 60 a été affecté à la variable PRIX et 4 à la variable NUM. Le produit du contenu de ces variables a été affecté à la variable SOM.



Dans ce programme, l'affichage ne présente que des points d'interrogation, et personne d'autre pourrait savoir de quoi il s'agit. Par respect d'autrui, modifions ce programme ainsi.

```

10 REM TOTAL
20 INPUT "PRIX";PRIX
30 INPUT "NOMBRE";NUM
40 LET SOM=PRIX*NUM
50 PRINT "TOTAL";SOM
60 END

```

Lorsque ce programme est exécuté, on obtient l'affichage suivant:

```

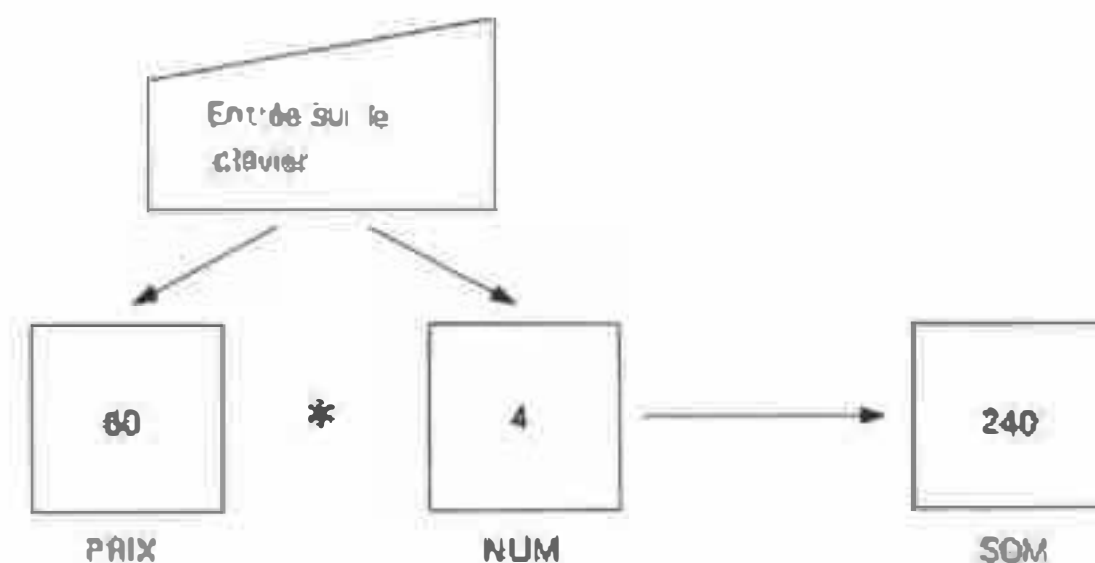
PRIX? 40
NOMBRE? 5
TOTAL 200
=

```

Il est maintenant apparent qu'il faut entrer un prix et une quantité pour que le total puisse être obtenu. En écrivant l'instruction INPUT de la manière suivante,

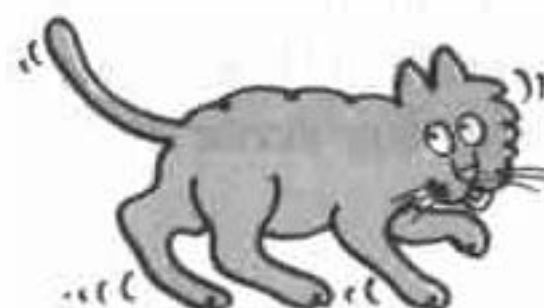
INPUT "chaîne d'appel"; variable

Les caractères d'appel (message indiquant ce qu'il faut entrer) et "?" sont affichés au début d'une ligne pour permettre à quiconque de comprendre quelle valeur doit être entrée. L'instruction PRINT n'est pas une instruction d'entrée, mais peut être écrite de cette manière.



Nouvelle instruction

INPUT Entre la donnée composée sur le clavier



2.3 Changement de la route de contrôle

Le programme ci-dessus s'achève après calcul des ventes totales. Modifions-le afin qu'il effectue plusieurs fois ce calcul.

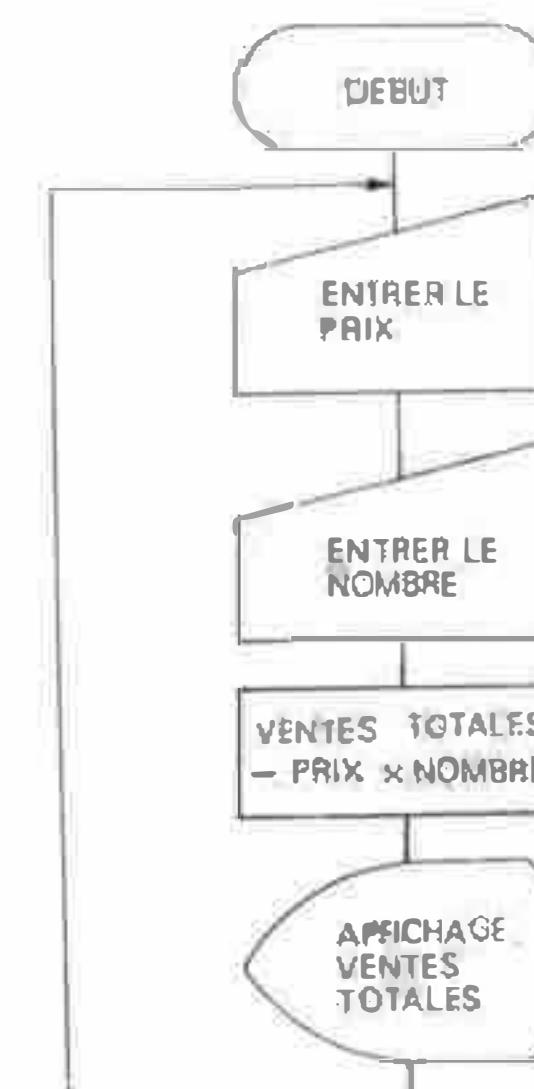
```

10 REM VENTES TOTALES
20 INPUT "PRIX";PRIX
30 INPUT "NOMBRE";NUM
40 LET SOM=PRIX*NUM
50 PRINT "TOTAL";SOM
55 GOTO 20
60 END
  
```

Dans ce programme, l'instruction suivante a été ajoutée:

55 GOTO 20

En général, un programme est exécuté dans l'ordre à partir du numéro de ligne le plus bas. Cette instruction GOTO change la route de contrôle, de sorte qu'après l'exécution de la ligne 55, la prochaine n'est pas la ligne 60, mais la ligne 20.



Exécutons maintenant ce programme.

PRIX? 25
NOMBRE? 5
TOTAL 125
PRIX? 23
NOMBRE? 7
TOTAL 161
PRIX? 70
NOMBRE? 4
TOTAL 280
PRIX? 70

Comme vous le voyez, ce programme est très pratique; il permet de calculer sans fin.

Cependant, il existe un petit problème: la ligne 60 n'est jamais exécutée, de sorte que le programme ne se termine jamais. Il est par conséquent nécessaire de l'amener à terme par pression de la touche ON/BREAK. Faites cela.

Break in 30

Le message ci-dessus est alors affiché, et le déroulement du programme est terminé. Ce message signifie que l'exécution a été interrompue à la ligne 30 (le numéro de ligne est variable). Si vous désirez poursuivre l'exécution de ce programme, frappez:

CONT 



Nouvelles instructions

GOTO n
CONT

Transfère le contrôle à la ligne n
Reprend l'exécution du programme au point où il a été interrompu.



2.4 Prise de décision

La pression de la touche ON/BREAK est une façon inadéquate de terminer l'exécution d'un programme. Il est plus naturel de l'achever au moyen de l'instruction END. Corrigeons ce programme de sorte qu'il arrête le calcul après 3 fois.

```
10 REM VENTES TOTALES
15 I=0
20 INPUT "PRIX";PRIX
30 INPUT "NOMBRE";NUM
40 LET SOM=PRIX*NUM
50 PRINT "TOTAL";SOM
52 I=I+1
55 IF I<3 THEN GOTO 20
60 END
```



!
 Ici, la variable I est utilisée comme compteur. Premièrement, à la ligne 15, le compteur est fixé à 0. Cependant, comme au stade initial 0 est attribué à toutes les variables, cette ligne peut être omise. A la ligne 52, à chaque fois que le total est calculé, I est ajouté au compteur. En d'autres termes, I est ajouté au contenu de la variable I, et la somme est affectée à la variable I. Chaque fois que la ligne 52 est exécutée, la valeur de la variable change comme suit:

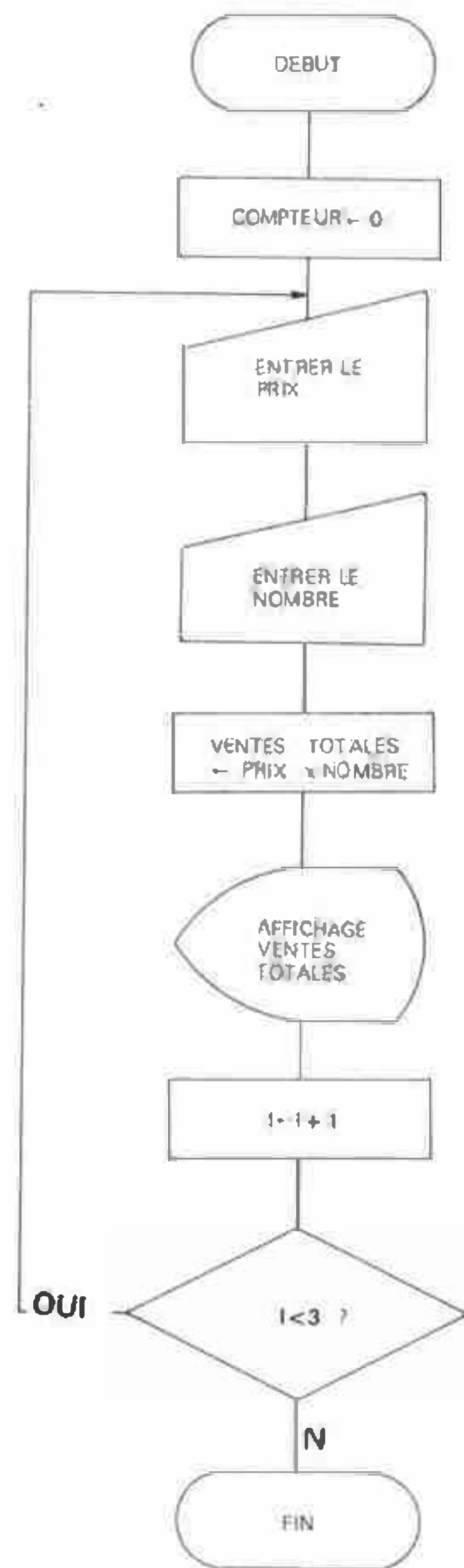
0→1 Première fois
1→2 Seconde fois
2→3 Troisième fois

La ligne 55 signifie "Si la valeur de I est inférieure à 3, retourner à la ligne 20. Sinon, exécuter la ligne suivante".

L'instruction IF prend une décision en fonction de la condition imposée, et modifie la route du contrôle selon que cette condition est remplie ou non. Exécutons maintenant ce programme.

Première fois (PRIX? 50
 NOMBRE? 4
 TOTAL 200
Seconde fois (PRIX? 60
 NOMBRE? 3
 TOTAL 180
Troisième fois (PRIX? 75
 NOMBRE? 6
 TOTAL 450
 ≥

Comme vous le voyez, l'exécution du programme a été terminée après trois fois.



Nouvelles instructions



IF ~ THEN

Prend une décision

2.5 Traitement de boucles

Une suite d'instructions répétées à plusieurs reprises comme dans le programme précédent est appelée une "boucle". Il existe un moyen bien plus simple de créer des boucles, au moyen d'instructions FOR - NEXT. Reprenons le programme précédent en utilisant des instructions FOR - NEXT. Changeons les lignes 15 et 55, et éliminons la ligne 52.

```

10 REM VENTES TOTALES
15 FOR I=1 TO 3
20 INPUT "PRIX";PRIX
30 INPUT "NOMBRE";NUM
40 LET SOM=PRIX*NUM
50 PRINT "TOTAL";SOM
55 NEXT I
60 END
  
```

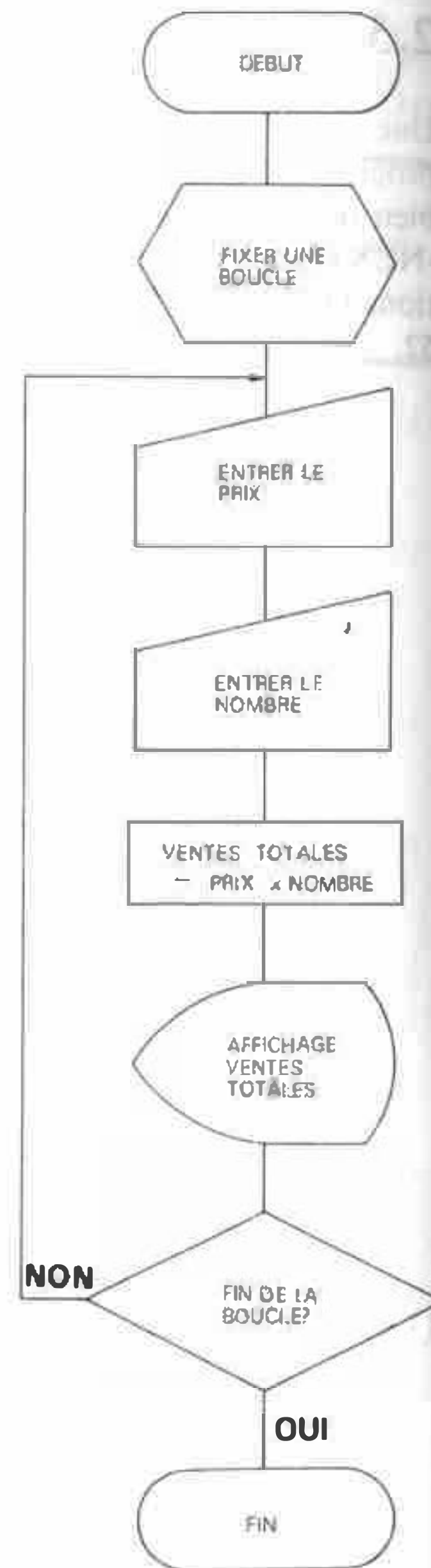


Les instructions entre FOR et NEXT sont exécutées à 3 reprises. De façon plus correcte, la valeur initiale de I est fixée à 1, et chaque fois que les instructions entre FOR et NEXT sont exécutées, elle est incrémentée de 1. Lorsque la valeur de la variable I est supérieure à 3, l'exécution du programme sort de la boucle et passe à la ligne 60.

Exécutez ce programme. Comme auparavant, après que le total ait été calculé 3 fois, le programme est terminé.

Nouvelles instructions

FOR ~ NEXT Exécute une boucle



2.6 Lecture de données dans un programme

Le programme suivant lit les notes d'élèves et calcule le total.

```

10 REM NOTES
20 DATA 58, 60, 30, 60
30 READ X1, X2, X3, X4
40 S=X1+X2+X3+X4
50 PRINT S
60 END
  
```

Dans ce programme, les notes sont lues de l'intérieur du programme, et non du clavier. En d'autres termes, les valeurs numériques spécifiées par l'instruction DATA sont lues et attribuées aux variables spécifiées par l'instruction READ. Dans ce programme, les valeurs 58, 60, 90 et 60 sont lues et attribuées aux variables X1, X2, X3 et X4 respectivement.



DATA 58, 60, 90, 60

```
READ X1, X2, X3, X4
```

Contrairement à l'instruction INPUT qui lit les données à partir du clavier, l'instruction READ lit les données à partir de l'instruction DATA. Exécutons maintenant ce programme.

RUN

268

Si tout est correct, on obtient l'affichage ci-dessus. Ce programme peut être réécrit de la manière suivante:

```
10 REM NOTES
20 DATA 58,60,90,60
25 FOR I=1 TO 4
30 READ X
40 S=S+X
45 NEXT I
50 PRINT S
60 END
```

Dans ce programme, la variable X lit dans l'ordre les valeurs numériques dans l'instruction DATA.

20 DATA 58, 60, 90, 60

30 READ X

A la ligne 40, les valeurs numériques ainsi lues sont ajoutées et la somme est affectée à la variable S. Les valeurs de S et X changent ainsi:

	S	X	S
$l=1$		$0 + 58 \rightarrow$	58
$l=2$		$58 + 60 \rightarrow$	118
$l=3$		$118 + 90 \rightarrow$	208
$l=4$		$208 + 60 \rightarrow$	268

Avec cette méthode, une seule variable est nécessaire pour traiter autant de valeurs numériques qu'on désire, ce qui la rend très pratique.

Voici quelques exemples d'instructions READ et DATA correspondantes.

DATA 1, 2, 3, 4

READ X, Y

Lorsque les valeurs numériques sont lues dans une même variable, seule la dernière donnée lue est valable (3 et 4)

DATA	1,	2,	3,	4,	5
------	----	----	----	----	---

```
READ      X,  Y,  Z
```

DATA 1. 2

READ X, Y, Z-

Une erreur est produite, car il n'y a pas de valeur numérique correspondante.

Une instruction DATA peut être placée n'importe où dans le programme. Il peut également y avoir plusieurs instructions DATA. Dans un tel cas, les données sont lues dans l'ordre à partir de l'instruction DATA dont le numéro de ligne est le plus bas.



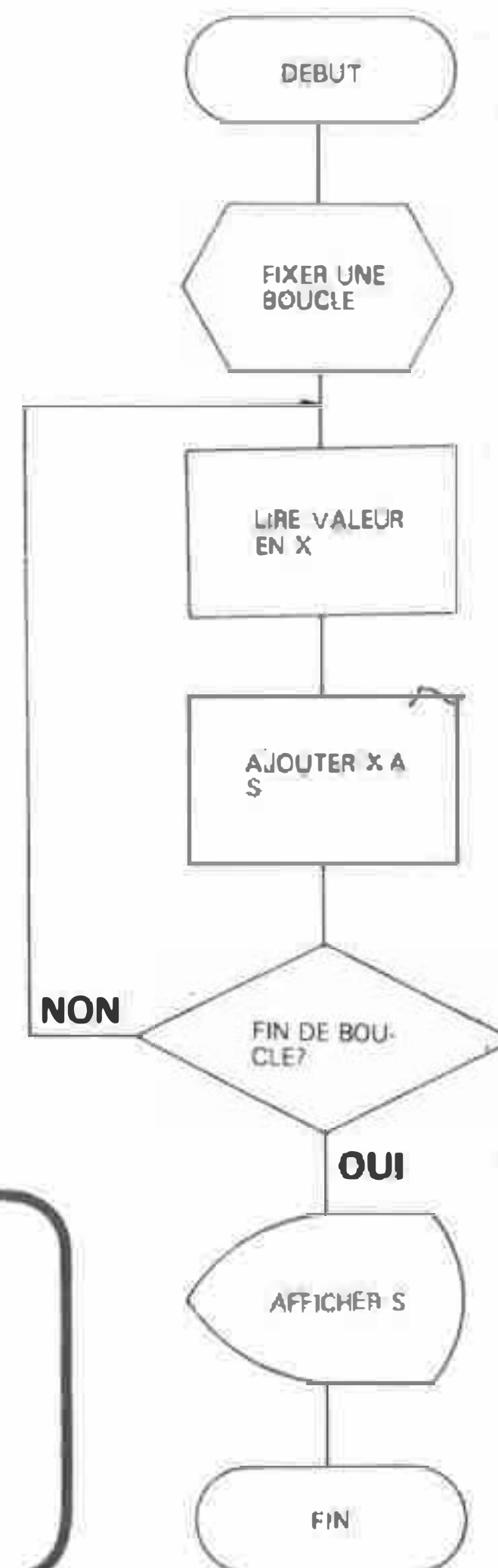
Nouvelles instructions

DATA

Spécifie des données

READ

Lit les données hors de l'instruction DATA



2.7 Tableaux de variables

A. Variables indicées

Dans le programme suivant, le numéro de l'élève est frappé sur le clavier, et la note correspondante est affichée.

```

10 REM NOTES
20 DATA 60,90,80,75
30 READ X1,X2,X3,X4
40 INPUT "ELEVE NO. ";N
50 IF N=1 THEN PRINT X1
60 IF N=2 THEN PRINT X2
70 IF N=3 THEN PRINT X3
80 IF N=4 THEN PRINT X4
90 END
  
```

Les notes des élèves sont spécifiées dans l'instruction DATA dans l'ordre du numéro de l'élève. Produisons la note de l'élève numéro 2



L'affichage est le suivant lorsque le programme est exécuté.

```
ELEVE NO.? 2
          90
          =
```

Vous avez peut-être remarqué que dans ce programme il faut autant de variable qu'il y a d'élèves. Ceci ne pose pas de problèmes tant que le nombre d'élèves est petit, comme dans ce programme où il y en a 4. Cependant, il ne convient pas s'il y a plus de 100 élèves, car il faudrait 100 variables.

Dans un tel cas, les variables indicées sont très pratiques.

Une seule donnée peut être attribuée à une variable ordinaire.

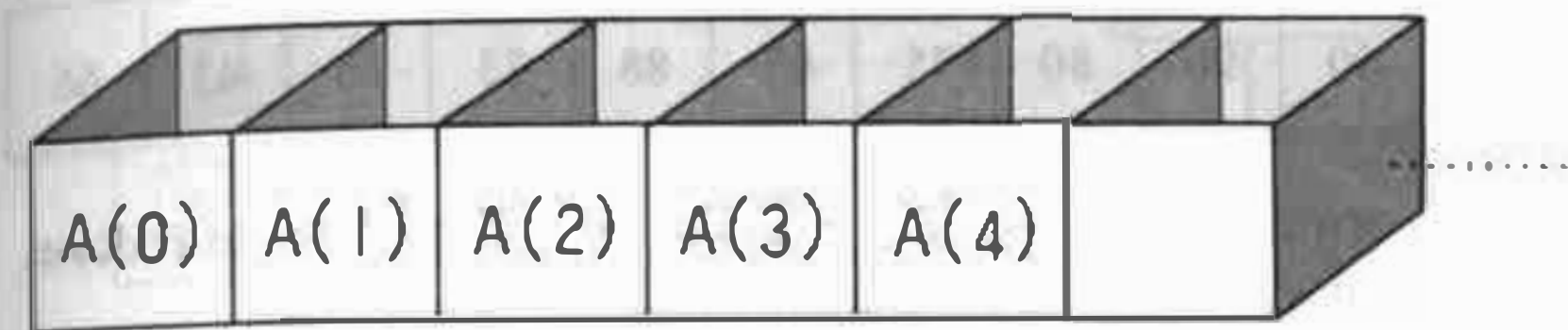
Toutefois, de nombreuses données peuvent l'être à une liste de variables. Une variable peut être représentée comme une maison à un seul habitant, mais une liste de variables comme un immeuble de nombreux habitants. Chaque appartement est identifié par son numéro. Il en va de même pour une variable indicée. Chacune possède un numéro pour identifier sa place en mémoire. Par exemple, la liste A reçoit les numéros suivants:

A(0), A(1) A(2), A(3), ...

Les chiffres entre parenthèses sont appelés "indices"



Liste de variables



Réécrivons le programme précédent en utilisant une liste de variables.

```
10 REM NOTES
20 DATA 60,90,80,75,65,88,23,76,43,66
30 DIM X(10)
40 FOR I=1 TO 10
50 READ X(I)
60 NEXT I
70 INPUT "ELEVE NO. ";N
80 PRINT X(N)
90 END
```

A la ligne 30, la liste de variables X est déclarée. Une telle liste doit toujours être déclarée avant de pouvoir être utilisée. Dans ce cas, une zone de X(0) à X(10) est réservée. cependant, si l'indice est inférieur à 10, une zone est automatiquement réservée, même si l'instruction DIM est omise. En conséquence, la ligne 30 peut être omise si vous le désirez.

A la ligne 50, les données sont attribuées aux variables de la liste X. Cependant, si vous fixez un indice à la variable, comme dans ce cas, le traitement de cette liste est plus aisé. Les données sont attribuées à la liste de variable X comme suit:

X(0)	X(1)	X(2)	X(3)	X(4)	X(5)	X(6)	X(7)	X(8)	X(9)	X(10)
0	60	90	80	75	65	88	23	76	43	66

Comme X(0) n'est pas utilisé, la valeur initiale de 0 est introduite.
Maintenant exécutons ce programme.

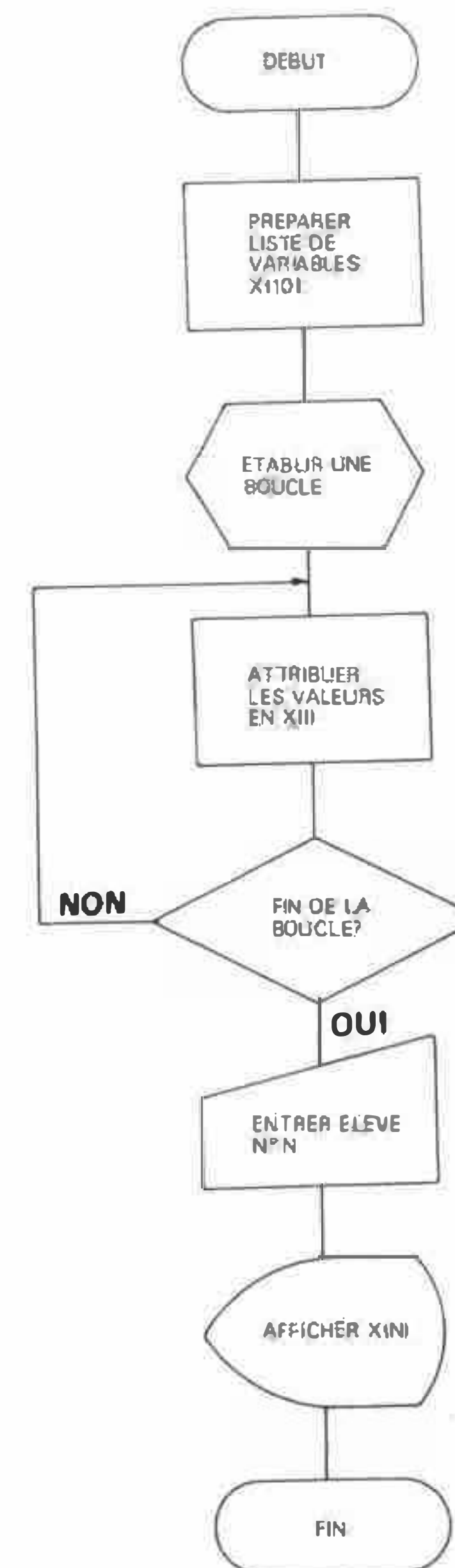
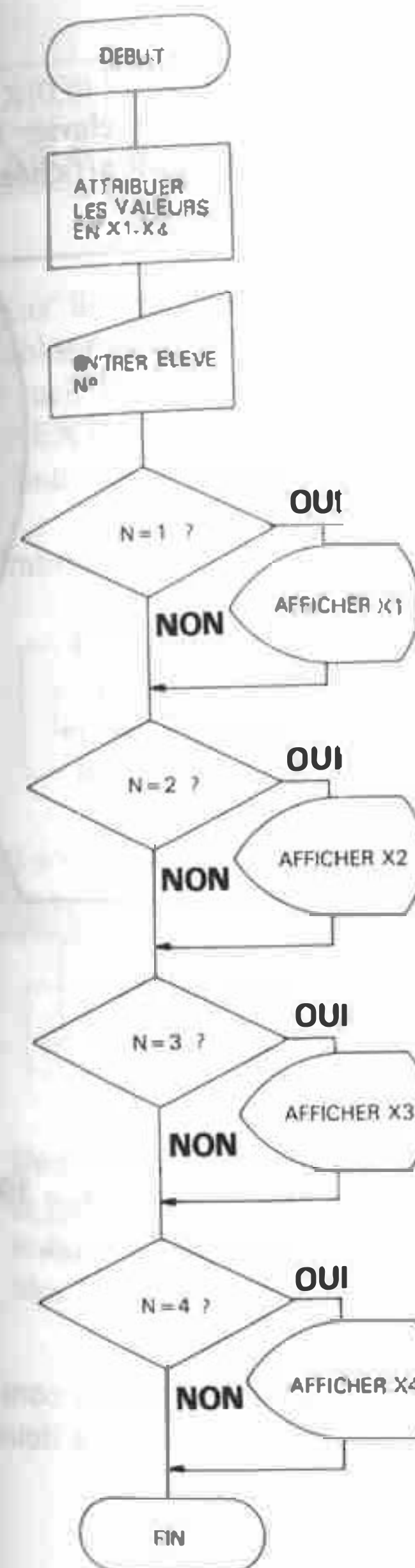
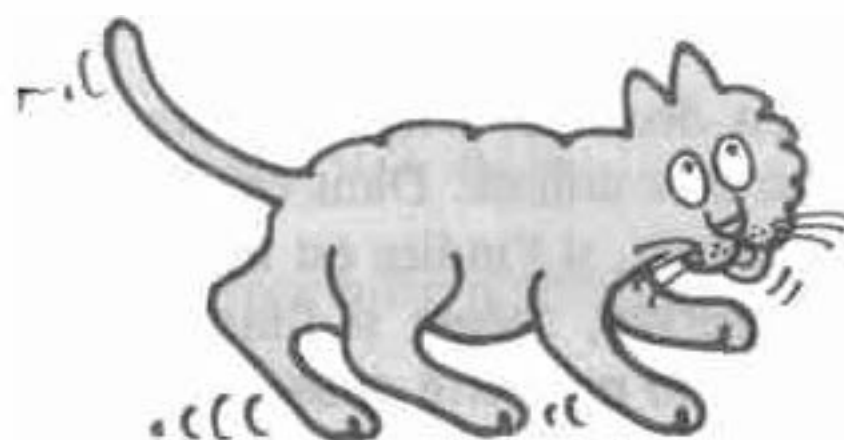
```

RUN
ELEVE NO.? 6
88

```

Nouvelles instructions

DIM Déclaration d'une liste de variables



B. Tableaux

Dans le prochain programme, on introduit à l'aide du clavier les numéros d'élève et les matières, et les notes des élèves sont affichées.

```
10 REM NOTES
20 DATA 56,67,66,98,45,55,65,87,32,19
30 DIM X(5,1)
40 FOR I=1 TO 5
50 FOR J=0 TO 1
60 READ X(I,J)
70 NEXT J
80 NEXT I
90 INPUT "ELEVE NO. ";N
100 INPUT "ANGLAIS (0) OU MATH (1)";M
110 PRINT X(N,M)
120 END
```

A l'instruction DATA de la ligne 20, les notes sont alignées ainsi:

	ANG.	MATH	ENG.	ANG.					
20 DATA	56,	67,	66,	98,	45,	55,	65,	87,	32, 19
	NO1		NO2		NO3		NO4		NO5

Le tableau est déclaré par l'instruction DIM de la ligne 30, qui comprend deux indices séparés par une virgule. Ce genre de liste à deux dimensions est appelé "tableau".

La zone réservée en mémoire pour de tableau est la suivante:

X(5,1)					
X(0,0)	X(1,0)	X(2,0)	X(3,0)	X(4,0)	X(5,0)
X(0,1)	X(1,1)	X(2,1)	X(3,1)	X(4,1)	X(5,1)

Aux lignes 40 et 80, les données sont attribuées aux variables du tableau à partir de l'instruction DATA. Veuillez remarquer qu'ici une boucle FOR - NEXT est placée à l'intérieur d'une autre boucle FOR - NEXT. Le placement d'une boucle à l'intérieur d'une autre est appelé "imbriquage".

Imbriquage

```
FOR I
  FOR J
  NEXT J
NEXT I
```

Il n'est pas admis d'imbriquer des boucles de la manière suivante:

```
FOR I
  FOR J
  NEXT I
NEXT J
```

Dans les boucles imbriquées FOR - NEXT, le contenu de la variable de la boucle intérieure change plus rapidement que celui de la variable de la boucle extérieure. Dans le présent programme, le contenu des variables I et J change ainsi:

I	1	1	2	2	3	3	4	4	5	5
J	0	1	0	1	0	1	0	1	0	1

$X(1,0)$ $X(1,1)$ $X(2,0)$ $X(2,1)$ $X(5,1)$

Lorsqu'un programme est exécuté, les données suivantes sont stockées dans le tableau X.

J	0	0	56	66	45	65	32	Anglais
	1	0	67	98	55	87	19	Math
		0	1	2	3	4	5	

I

Lorsque ce programme est exécuté, l'affichage est le suivant:

```

ELEVE NO.? 3
ANGLAIS (0) OU MATH (1)? 1
55

```



FOR I = ?

La variable utilisée avec l'instruction FOR est appelée un "compteur". Pourquoi I est-il si souvent utilisé? La raison en est qu'avant que le langage BASIC ne soit créé, il existait un langage couramment utilisé, le FORTRAN. Dans le langage FORTRAN, I, J, K, L, M et N sont utilisés pour les variables entières, et ceux qui avaient l'habitude d'utiliser I comme compteur en langage BASIC. Dans ce dernier, le nom d'une variable peut être n'importe quoi. Un débutant pourrait confondre "I" pour "1", de sorte que A ou N pourrait s'avérer plus pratique. Il semble toutefois préférable de continuer un usage consacré par l'expérience. Pour cette raison, lorsque vous étudiez un programme, il vous est aisé d'identifier I comme un compteur, X et Y comme coordonnées de l'écran, etc. En langage BASIC, vous commettrez moins d'erreurs du genre "FOR I = 0 TO 3", et "NEXT 1".

2.8 Traitement des chaînes de caractères

A. Liaison de caractères

Dans le programme ci-dessous, un prénom et un nom sont entrés à l'aide du clavier et affichés ensemble sur l'écran.

```
10 REM
20 INPUT "PRENOM";F$
30 INPUT "NOM";L$
40 NOM$=F$+" "+L$
50 PRINT NOM$
60 END
```

Comme les données entrées à l'aide du clavier sont des chaînes de caractères et non des valeurs numériques, les variables de l'instruction INPUT doivent être des chaînes. Le signe dollar (\$) est placé à la fin d'une chaîne de caractères.

A la ligne 40, les prénom et nom entrés sont reliés avec un espace entre eux. Comme vous le voyez, on utilise pour relier des chaînes de caractères le symbole (+), comme pour additionner des nombres.

Exécutez ce programme.

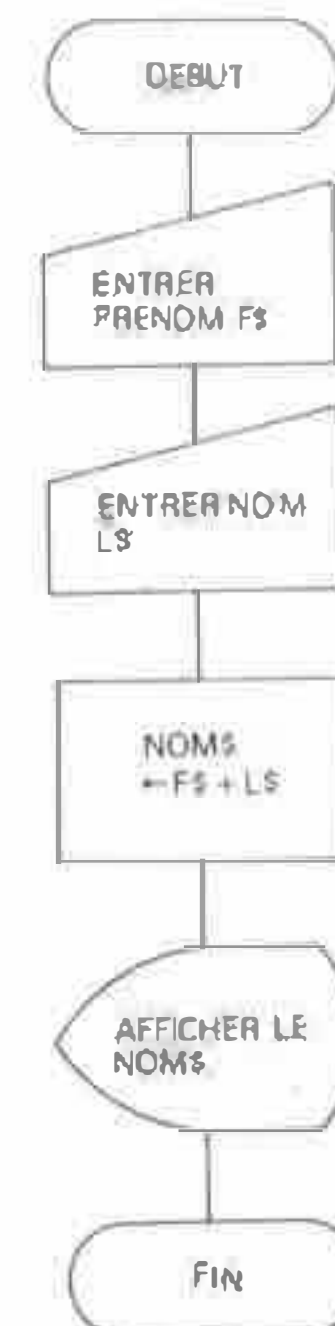
```
PRENOM? MARIE
NOM? DUPONT
MARIE DUPONT
```

Dans l'exemple ci-dessus, les chaînes de caractères ont été stockées dans les variables indiquées ci-dessous.

```
F$ MARY + " " + L$ SMITH
→ NOM$ MARY SMITH
```

Nouvelle instruction

+ Relie des caractères



B. Lecture de chaînes de caractères

Rédigons un programme dans lequel les notes d'élèves sont affichées lorsque leur nom est entré sur le clavier.

```

10 REM NOTES
20 DATA JULIE,FRANCOISE,PAUL
30 DATA 50,90,20
40 DIM A$(2),A(2)
50 FOR I=0 TO 2
60 READ A$(I)
70 NEXT I
80 FOR I=0 TO 2
90 READ A(I)
100 NEXT I
110 INPUT "NOM";N$
120 FOR I=0 TO 2
130 IF A$(I)=N$ THEN 160
140 NEXT I
150 GOTO 110
160 PRINT A$(I);" ";A(I)
170 END

```

Lire noms

Lire notes

A la ligne 60, les données tirées de l'instruction DATA sont placées dans la liste de variables A\$. Remarquez qu'ici les constantes de caractères sont spécifiées dans l'instruction DATA. De ce fait, la liste doit être composée de chaînes de caractères.

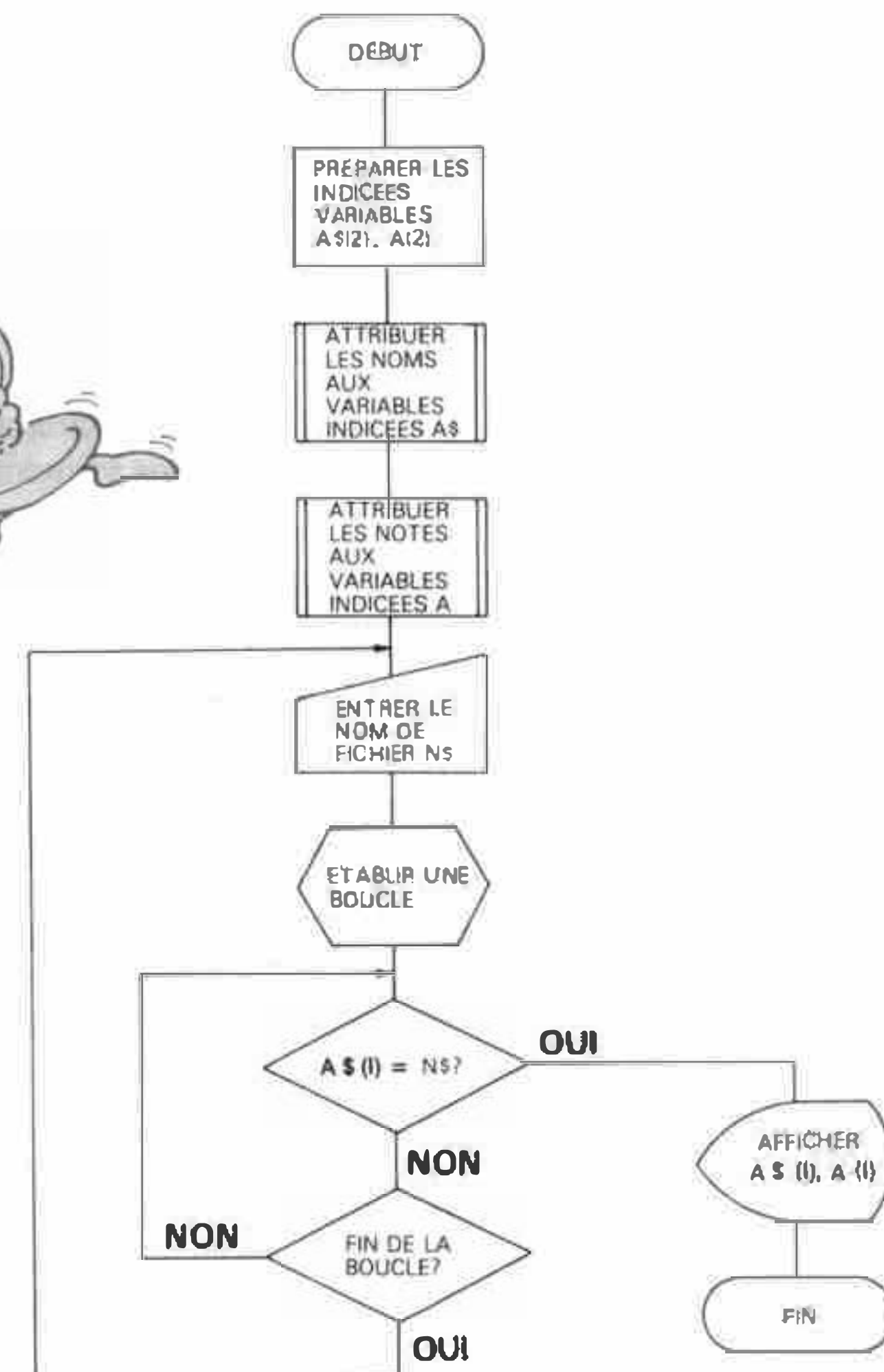
A la ligne 130, les chaînes de caractères sont comparées.

Si elles correspondent, le nom et la note sont affichés sur la ligne 160.

Voici ce qui se produit lorsque ce programme est exécuté.

NOM? FRANCOISE

FRANCOISE 90



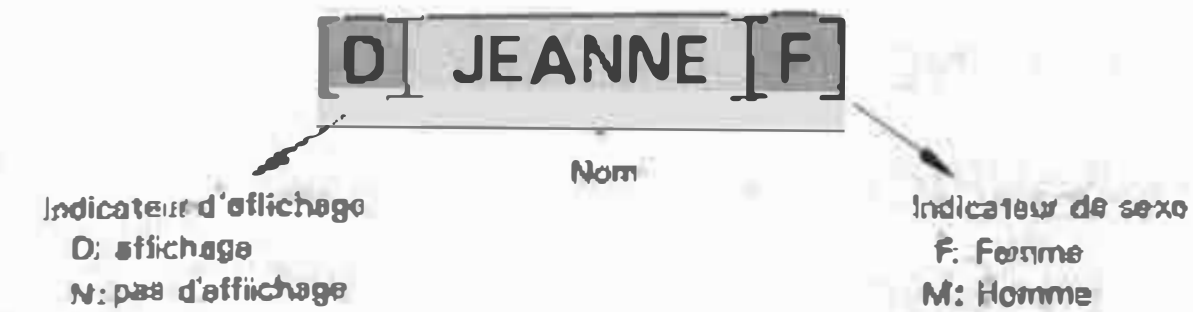
C. Fonctions traitant des chaînes de caractères

Etudiez le programme suivant.

```
10 REM
20 DATA DJEANNEF,DMAURICEH,NJACQUESH,DHE
LENEF,NJACQUES
30 FOR I=1 TO 5
40 READ NOM$
50 DSP$=LEFT$(NOM$,1)
60 IF DSP$="N" THEN 120
70 SEX$=RIGHT$(NOM$,1)
80 L=LEN(NOM$)-2
90 N$=MID$(NOM$,2,L)
100 IF SEX$="H" THEN S$=" " ELSE S$="*"
110 PRINT S$+N$
120 NEXT I
130 END
```

Dans ce programme, les données sont tirées de l'instruction DATA, les noms sont extraits puis affichés.

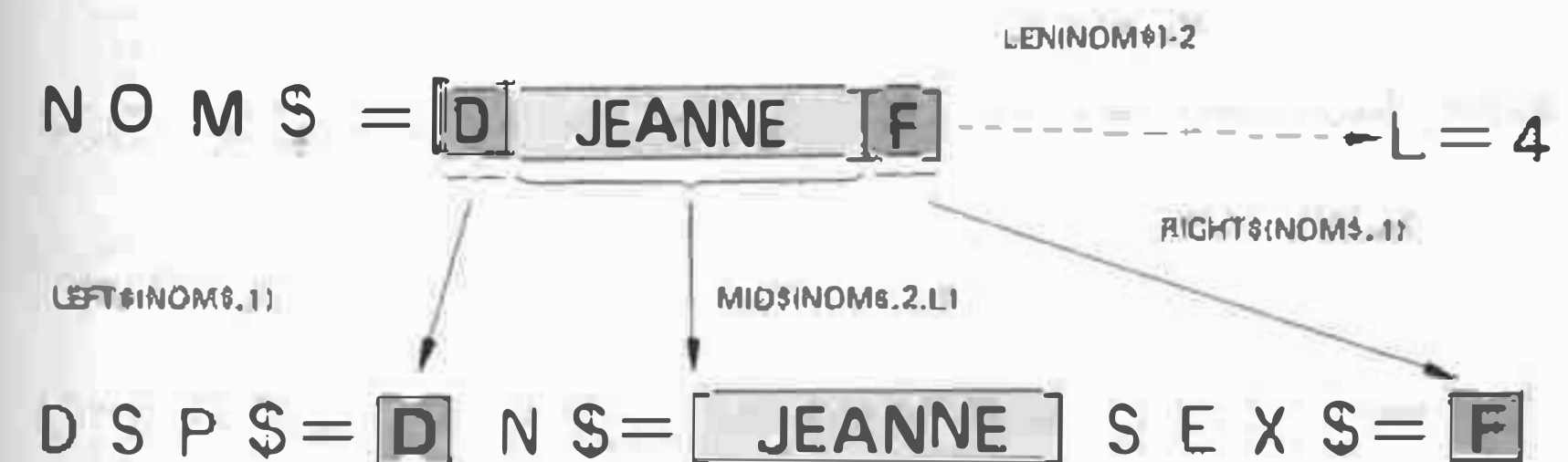
Les noms sont spécifiés sous la forme suivante dans l'instruction DATA



Lorsque le caractère de tête est "D", le nom est affiché. Le dernier caractère dénote le sexe. Lorsqu'il s'agit d'un nom masculin, un espace est placé au début du nom affiché, et un astérisque (*) lorsqu'il s'agit d'un nom féminin.

Dans ce programme, on utilise plusieurs fonctions traitant des données en chaîne. Les fonctions LEFT\$, RIGHT\$ et MID\$ extraient respectivement la partie gauche, droite et médiane des caractères d'une chaîne. La fonction LEN produit le nombre de caractères d'une chaîne.

Lorsque ce programme est exécuté, les caractères sont stockés dans les variables de la manière suivante:

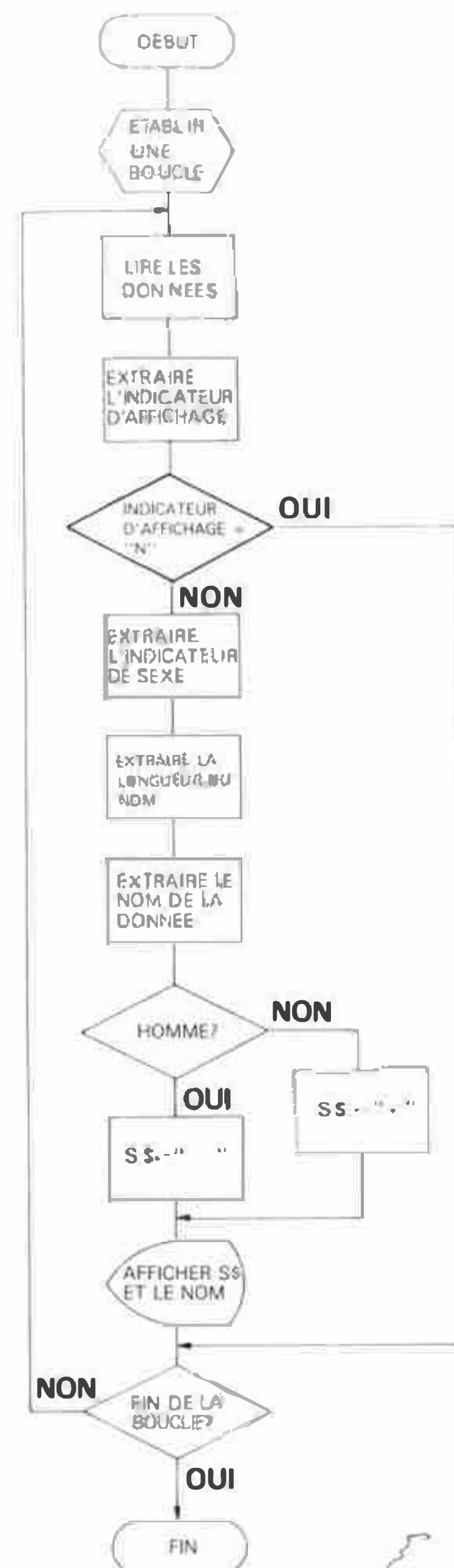


En d'autres termes, les caractères extrêmes gauches sont extraits du contenu de la variable NOM\$ à la ligne 50. Les caractères extrêmes droits sont extraits de NOM\$ à la ligne 70. A la ligne 80, la longueur du contenu de la variable NOM\$ moins 2 (c'est-à-dire la longueur du nom) est calculée. A la ligne 90, les caractères sont extraits de la variable NOM\$ à partir du second caractère.

Remarquez que ELSE est utilisé dans l'instruction IF de la ligne 100. Ce ELSE spécifie l'instruction qui doit être exécutée si la condition imposée n'est pas satisfaite.

Lors de l'exécution de ce programme, l'affichage est le suivant.

```
*JEANNE
MAURICE
*HELENE
```



Nouvelles instructions

- LEN** Produit la longueur d'une chaîne de caractères
- LEFT\$** Extrait les caractères extrêmes gauches d'une chaîne de caractères
- RIGHT\$** Extrait les caractères extrêmes droits d'une chaîne de caractères
- MID\$** Extrait une partie d'une chaîne de caractères

2.9 Définition de fonctions

Le programme suivant calcule la surface et la circonférence d'un cercle.

```
10 REM CALCUL D'UN CERCLE
20 DEFFN M(R)=3.14159*R*R
30 DEFFN N(R)=3.14159*2*R
40 INPUT I
50 X=FN M(I)
60 Y=FN N(I)
70 PRINT X,Y
80 END
```

Les lignes 20 et 30 définissent les fonctions M(R) et N(R) qui calculent la surface et la circonférence d'un cercle à partir de son rayon R. A la ligne 40, le rayon du cercle est entré sur le clavier, et aux lignes 50 et 60, la surface et la circonférence du cercle sont respectivement calculées à l'aide des fonctions définies ci-dessus.

En utilisant la fonction DEFFN de cette manière, il est possible de définir librement une fonction, et une fois qu'elle est définie dans un programme, une fonction peut être utilisée autant de fois que désiré dans le cadre de celui-ci. Par exemple, utilisez la fonction DEFFN pour les longues expressions souvent utilisées.

Lors de l'exécution de ce programme, l'affichage est le suivant:

```
? 7 Rayon du cercle
153.93791 Surface du cercle
43.98226 Circonférence
du cercle
≥
```



Nouvelle instruction

DEFFN Définition d'une fonction

le nom de la
fonction doit
être le même
dans DEFN et
FN (toutes les
lettres ont la même
signification)
4 lettres
au plus
+ une espace
renouvelable

2.10 Sous-programmes

Le programme suivant produit les valeurs maximum et minimum d'un jeu de nombres.

```
10 REM MAX MIN
20 INPUT "NOMBRE D'ENTREES";N
30 DIM X(N)
40 FOR J=1 TO N
50 INPUT "NOMBRE";X(J)
60 NEXT J
70 GOSUB 100
80 GOSUB 200
90 END
100 REM MAX
110 MAX=0
120 FOR K=1 TO N
130 IF MAX<X(K) THEN MAX=X(K)
140 NEXT K
150 PRINT "VALEUR MAX";MAX
160 RETURN
200 REM MIN
210 MIN=100
220 FOR K=1 TO N
230 IF MIN>X(K) THEN MIN=X(K)
240 NEXT K
250 PRINT "VALEUR MIN";MIN
260 RETURN
```

Affiche les ventes
totales dans l'ordre
décroissant de leur
importance.

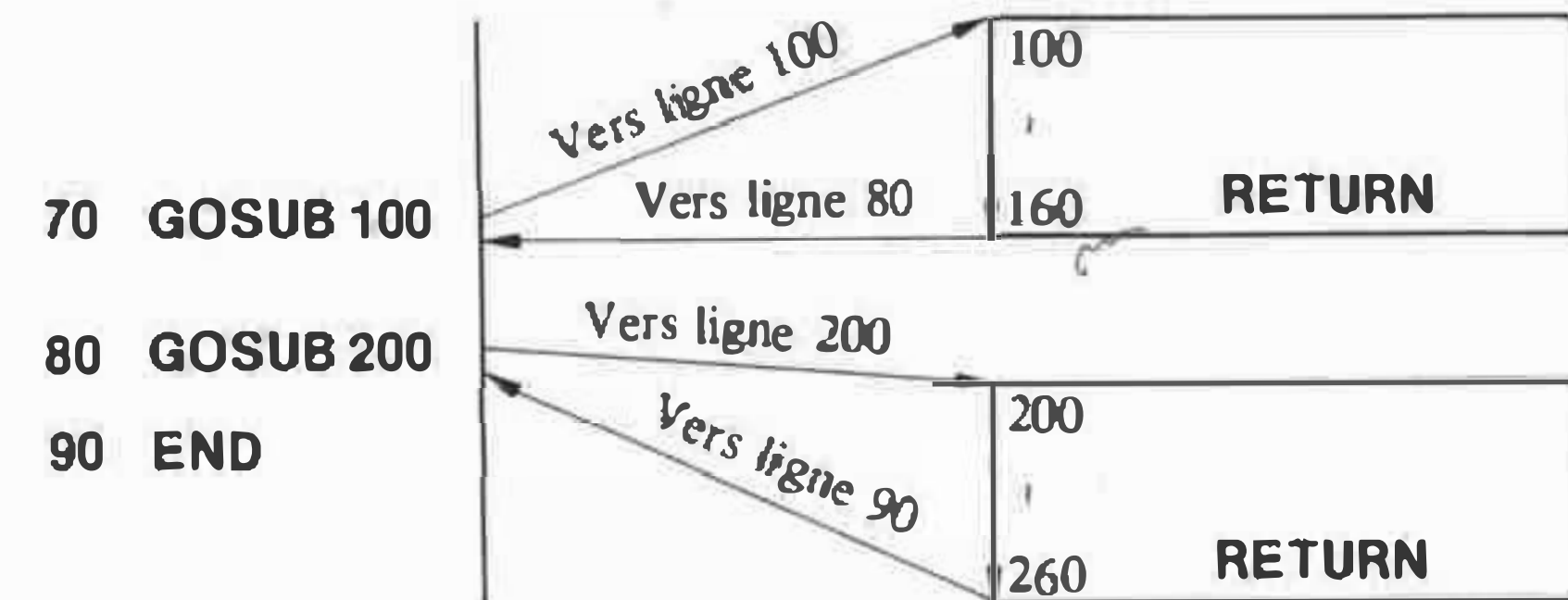
Affiche les ventes
totales dans l'ordre
décroissant de leur
importance.

Les valeurs maximum et minimum sont affichées lorsque des nombres compris entre 0 et 100 sont entrés sur le clavier.

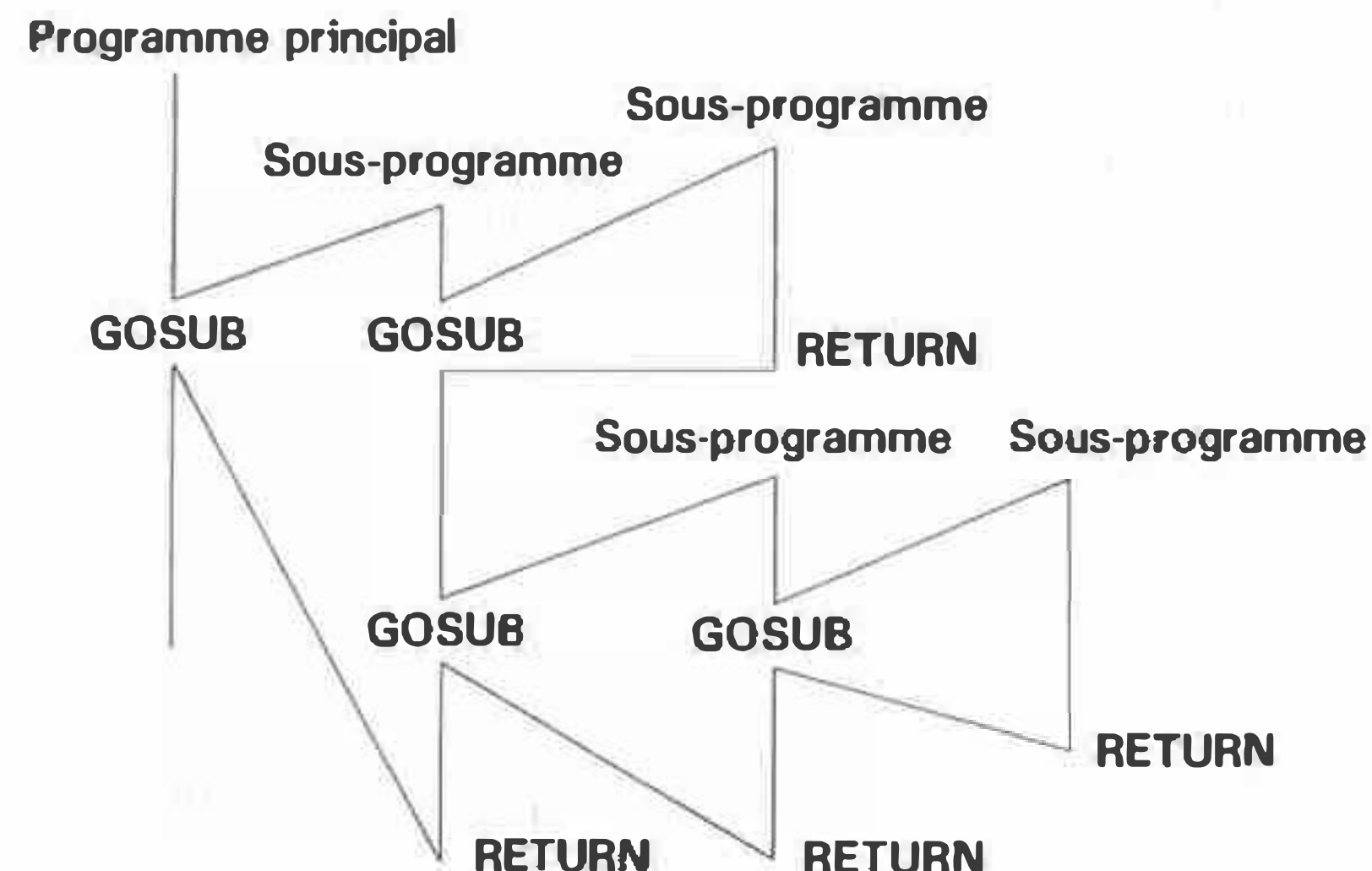
```
70 GOSUB 100
80 GOSUB 200
```

L'instruction GOSUB, tout comme l'instruction GOTO, transfère le contrôle au numéro de ligne spécifié. Toutefois, lorsque l'instruction RETURN est exécutée, le contrôle est retourné à l'instruction suivant GOSUB. Une partie de programme appelée de cette manière avec l'instruction GOSUB est appelée un "sous-programme".

Dans ce programme, les valeurs maximum et minimum sont calculées par les sous-programmes appelés par les instructions



Un sous-programme peut être appelé un nombre de fois quelconque. De même, un sous-programme peut en appeler un autre. Voici un exemple de sous-programmes imbriqués.



Lorsque le programme est exécuté, l'affichage est le suivant.

NOMBRE D'ENTREES? 5
 NOMBRE? 34
 NOMBRE? 76
 NOMBRE? 18
 NOMBRE? 90
 NOMBRE? 73
 VALEUR MAX 90
 VALEUR MIN 18
 ≥

Nouvelles instructions



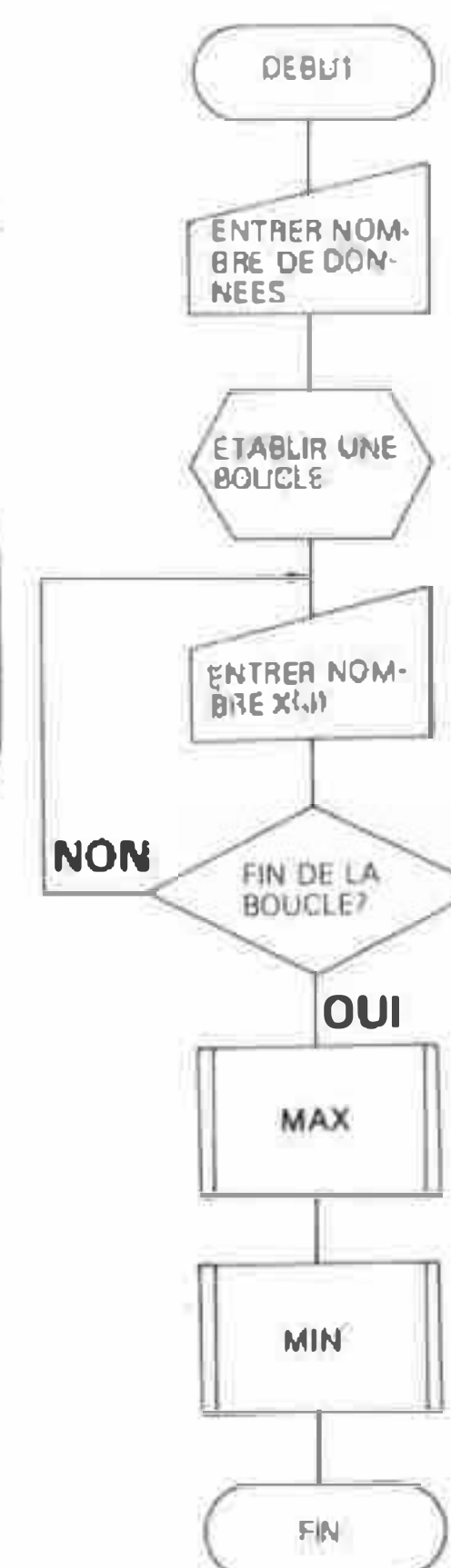
GOSUB Appelle un sous-programme
RETURN Termine un sous-programme

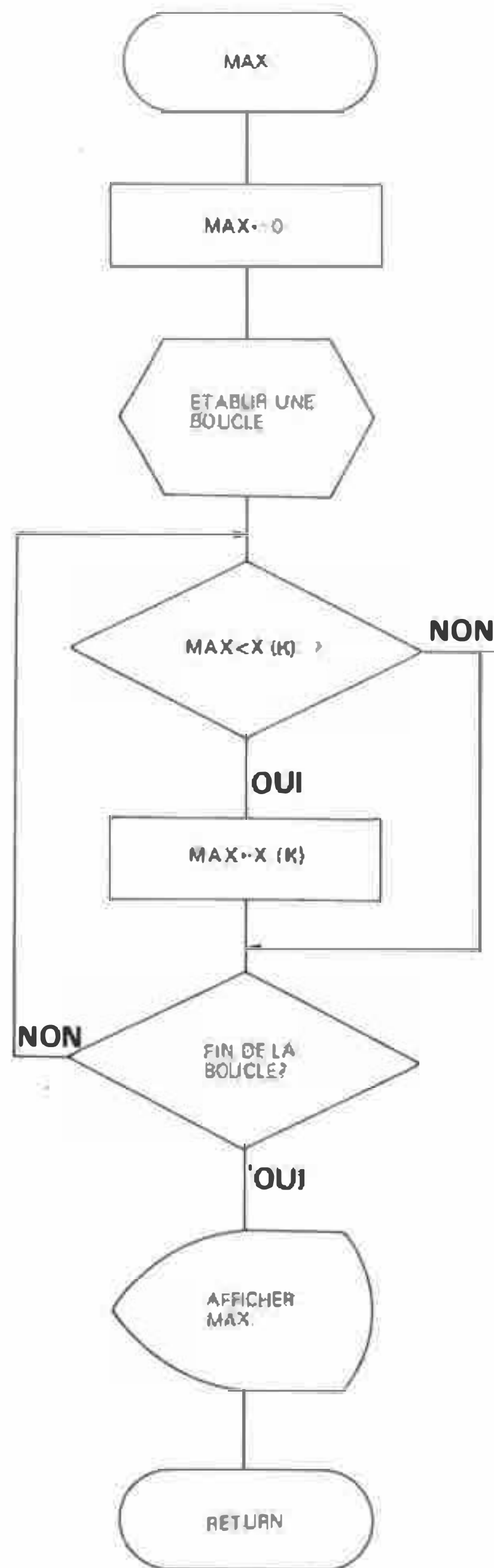
Imbrication de sous-programmes

Les sous-programmes sont utilisés pour clairement diviser des fonctions, et pour rassembler des sections similaires.

Les sous-programmes présentent les avantages suivants (avantages d'autant plus importants que le programme est long.)

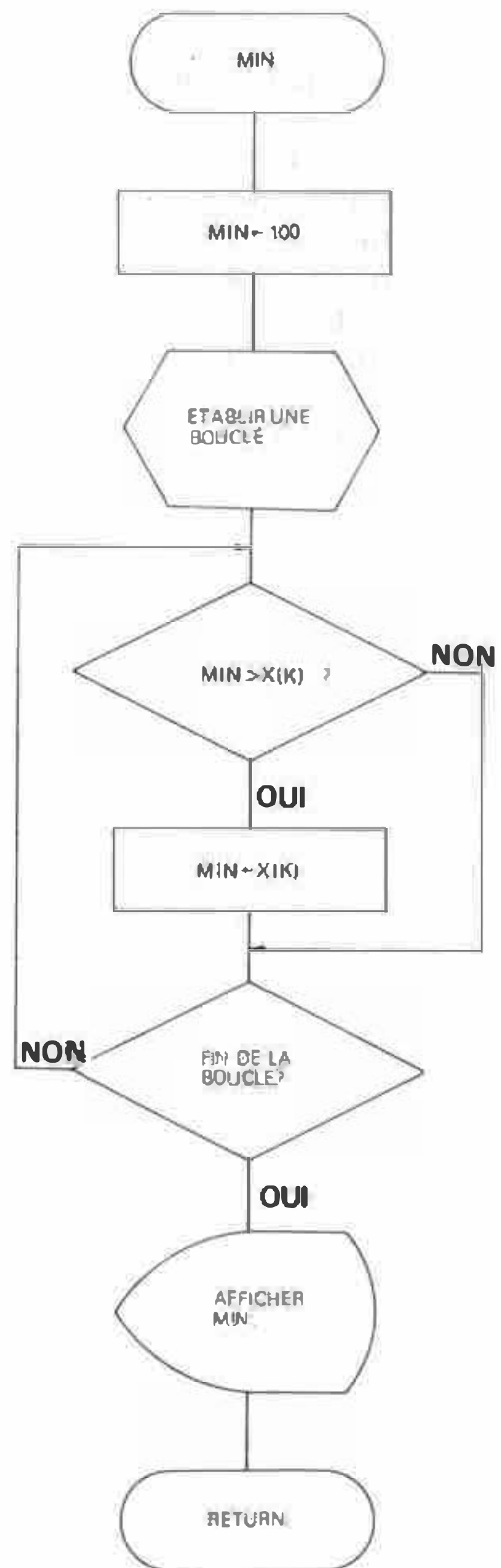
- Les fonctions peuvent être clairement séparées.
- La mise au point des programmes est facilitée.
- Rassemble des parties similaires d'un programme, ce qui permet de le raccourcir.
- Un programme peut être rédigé par plusieurs personnes





Chapitre 3

Programmes pratiques



Ce chapitre présente quelques programmes pratiques utiles pour les applications domestiques et commerciales.



3.1 Liste de chiffre d'affaire

Le programme suivant calcule les ventes totales, et les affiche dans l'ordre décroissant de leur importance.

```

10 DIM B$(50),C(50),D(50),E(50)
20 B$=""
30 INPUT "PRODUIT";B$
40 IF B$="" THEN 110
50 I=1+1
60 B$(I)=B$
70 INPUT "PRIX";C(I)
80 INPUT "QUANTITE";D(I)
90 E(I)=C(I)*D(I)
100 GOTO 20
110 N=I
120 SW=0
130 FOR I=1 TO N-1
140 IF E(I)>=E(I+1) THEN 200
150 SW=1
160 W$=B$(I):B$(I)=B$(I+1):B$(I+1)=W$
170 W=C(I):C(I)=C(I+1):C(I+1)=W
180 W=D(I):D(I)=D(I+1):D(I+1)=W
190 W=E(I):E(I)=E(I+1):E(I+1)=W
200 NEXT I
210 IF SW=I THEN 110

```

Entre les
données
et calcule
les ventes
totales

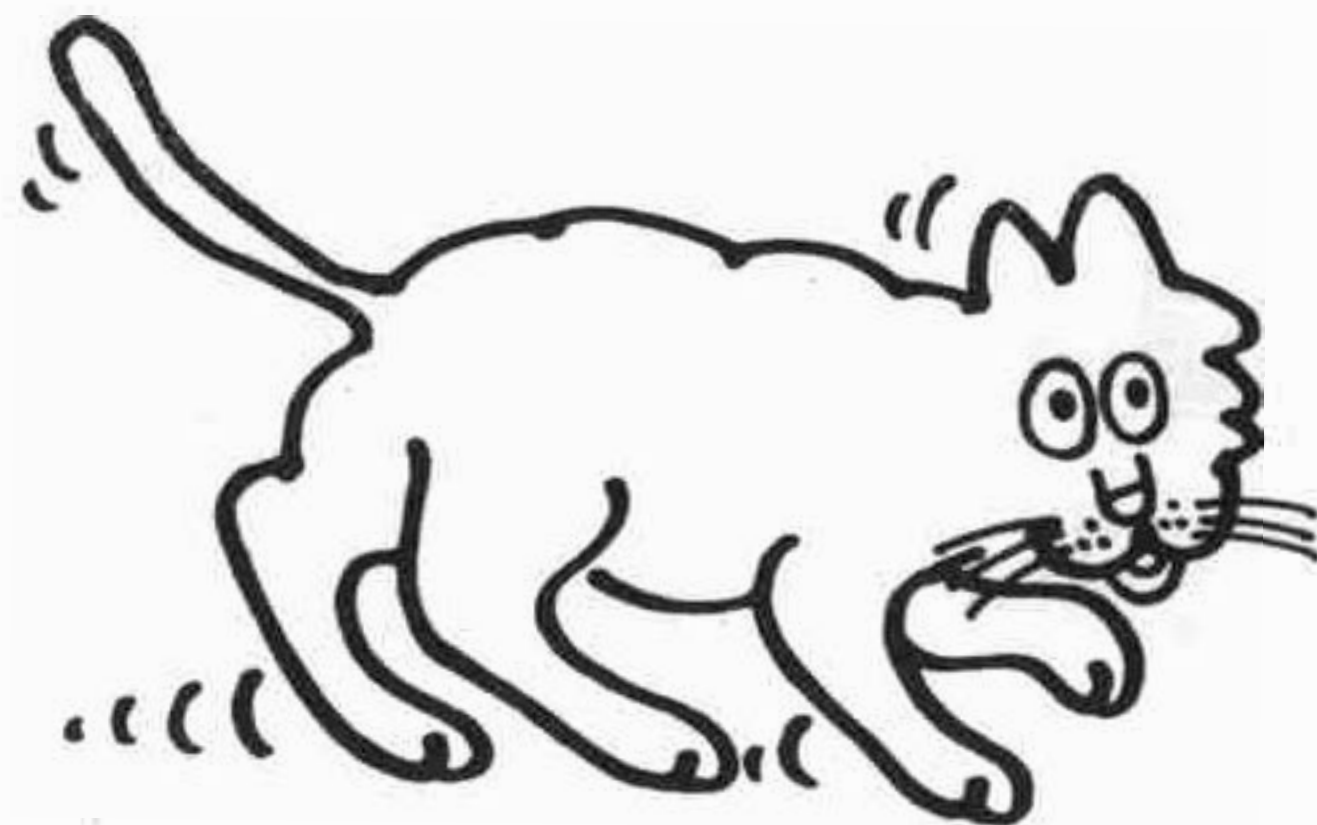
Tn

```

220 FOR I=1 TO N
230 PRINT
240 PRINT "RANG ";I
250 PRINT "PRODUIT: ";B$(I)
260 PRINT "PRIX ";
270 PRINT USING "F#.##";C(I)
280 PRINT "QUANTITE ";D(I)
290 PRINT "VENTES TOTALES ";
300 PRINT USING "F###.##";E(I)
310 NEXT I
320 END

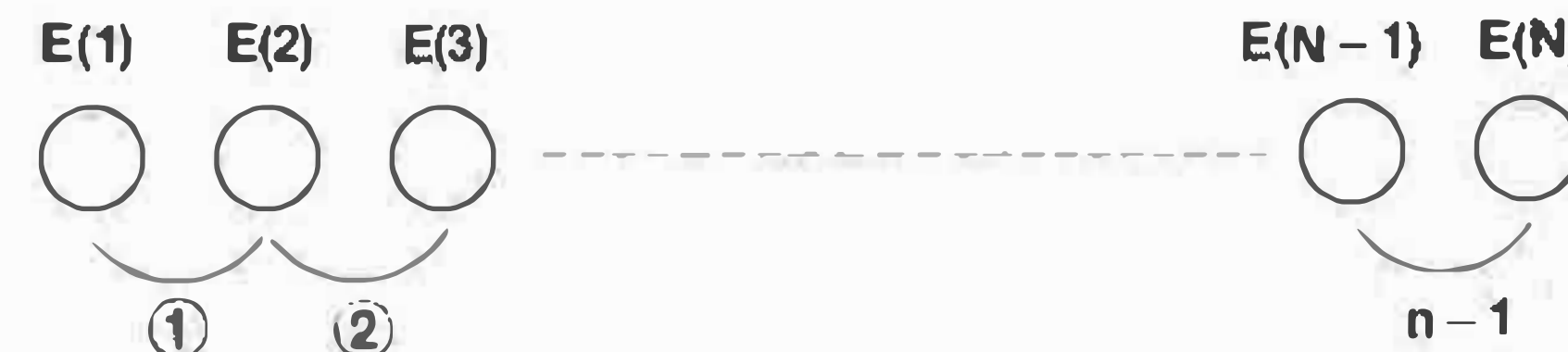
```

Affiche les ventes totales dans l'ordre décroissant de leur importance.



Après que les désignations de produit, prix et quantités aient été entrées sur le clavier et que les ventes totales aient été calculées, le tri est effectué et les résultats sont affichés en sortie.

Aux lignes 20 à 100, les désignations de produits, les prix et les quantités sont introduits sur le clavier. Le processus d'entrée est terminé lorsqu'une chaîne nulle (simple pression de la touche RETURN) est entrée comme désignation de produit. Le chiffre de vente de chaque produit est calculé à la ligne 90. Aux lignes 110 à 210, les chiffres de vente sont triés. A la ligne 140, les chiffres adjacents sont comparés, et s'ils ne sont pas dans l'ordre décroissant, ils sont permutés. Lorsque les chiffres sont permutés, la désignation de produit, le prix et la quantité correspondants le sont également. Les chiffres de vente sont comparés N-1 fois, et lorsqu'il y a permutation, la variable SW est fixée à 1. Dans un tel cas, il existe la possibilité que les permutations ne soient pas faites dans l'ordre décroissant, de sorte que la comparaison est à nouveau faite N-1 fois. Le tri est terminé lorsque la variable SW est fixée à 0.



Aux lignes 220 à 310, les rangs, désignations de produit, prix et quantités sont affichés dans l'ordre décroissant. Le format des prix et chiffres de vente affichés sont respectivement fixés aux lignes 270 et 300.

```
270 PRINT USING "F#.###";C(I)
300 PRINT USING "F#.###";E(I)
```

Les instructions ci-dessus spécifient qu'un symbole \$ est joint au prix et au chiffre de vente, que les deux derniers chiffres sont précédés d'une virgule, et que le nombre maximum de chiffres est de 3 (pour le prix) et de 5 (pour le chiffre de vente) Lorsque ce programme est exécuté, l'affichage devient le suivant:

(entrée de données)

```
PRODUIT? PLUME
PRIX? 1.00
QUANTITE? 150
PRODUIT? CAHIER
PRIX? 1.50
QUANTITE 90
PRODUIT? FICHER
PRIX? 3.00
QUANTITE? 55
PRODUIT? CRAYON
PRIX? 1.20
QUANTITE? 1.20
QUANTITE? 100
PRODUIT? [Appui sur la touche RETURN]
```

(Affichage de la liste dans l'ordre)

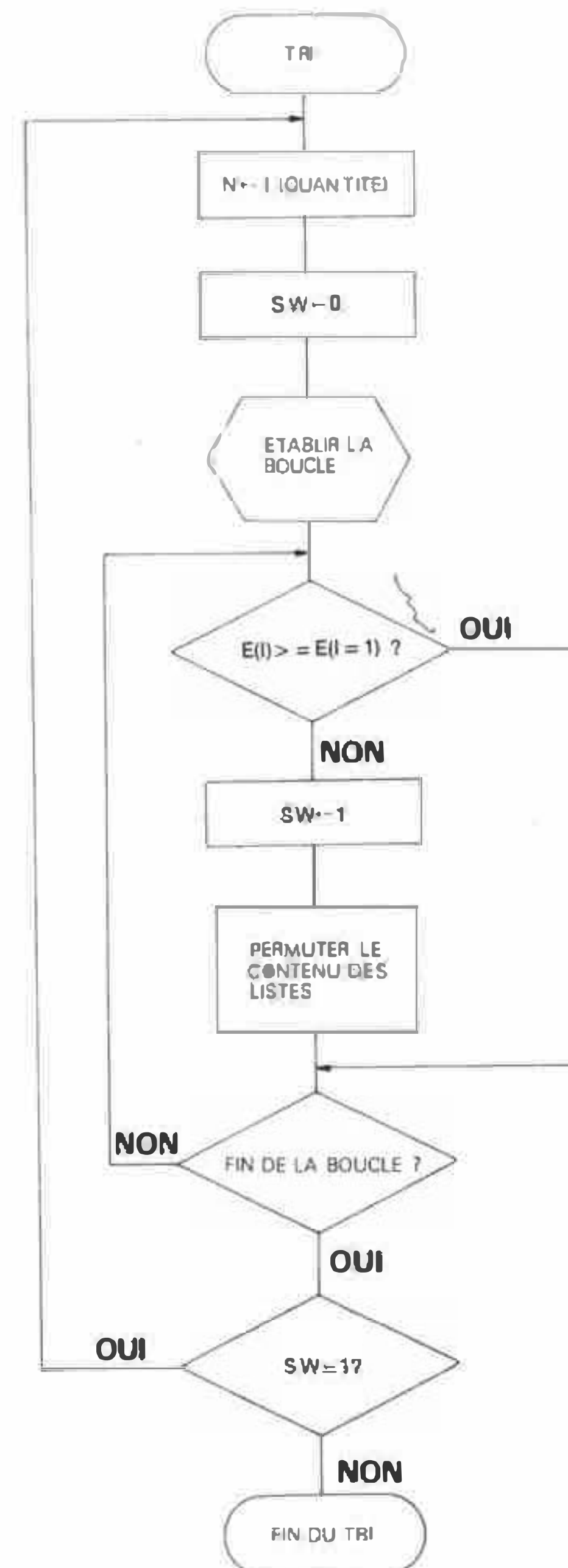
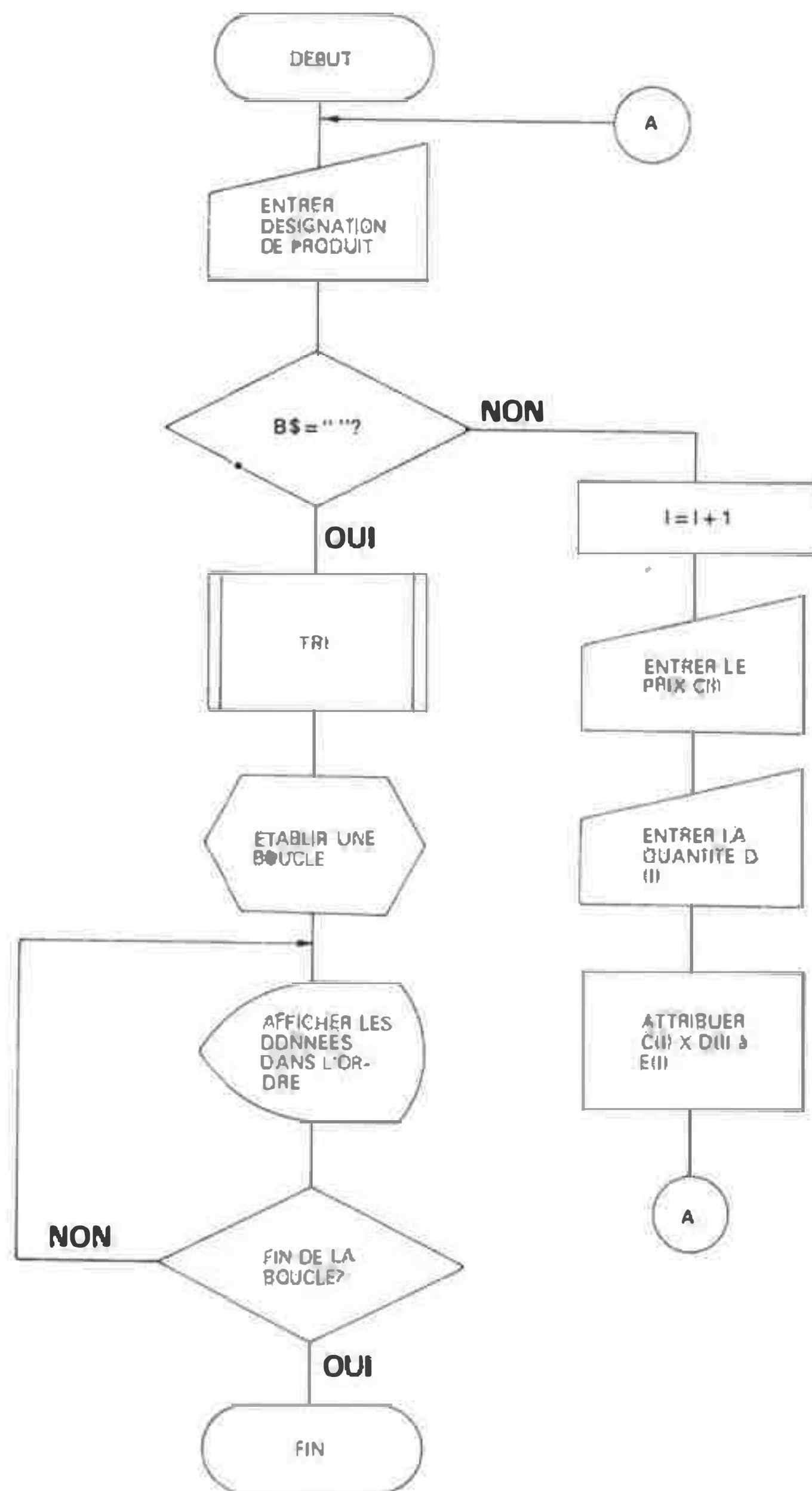
```
RANG 1
PRODUIT FICHER
PRIX: F3.00
QUANTITE 55
VENTES TOTALES F165.00
```

```
RANG 2
PRODUIT PLUME
PRIX F1.00
QUANTITE 150
VENTES TOTALES F150.00
```

```
RANG 3
PRODUIT CAHIER
PRIX F1.50
QUANTITE 90
VENTES TOTALES $135.00
```

```
RANG 4
PRODUIT CRAYON
PRIX F1.20
QUANTITE 100
VENTES TOTALES F120.00
```





Nouvelle instruction

PRINT USING
Spécifie le format
d'affichage

3.2 Remboursements égaux de capital et d'intérêt

Rédigeons un programme permettant de calculer le montant des remboursements égaux de capital et d'intérêt, le montant de capital et d'intérêt de chaque remboursement lorsqu'on introduit la période des remboursements, le taux annuel de l'intérêt et le capital.

Ces montants sont calculés à l'aide des expressions suivantes:

Montant des remboursements égaux de capital et d'intérêt

$$= \frac{\text{Capital} \times \text{taux d'intérêt} \times (1 + \text{taux d'intérêt})^{\text{ans}}}{(1 + \text{taux d'intérêt})^{\text{ans}} - 1}$$

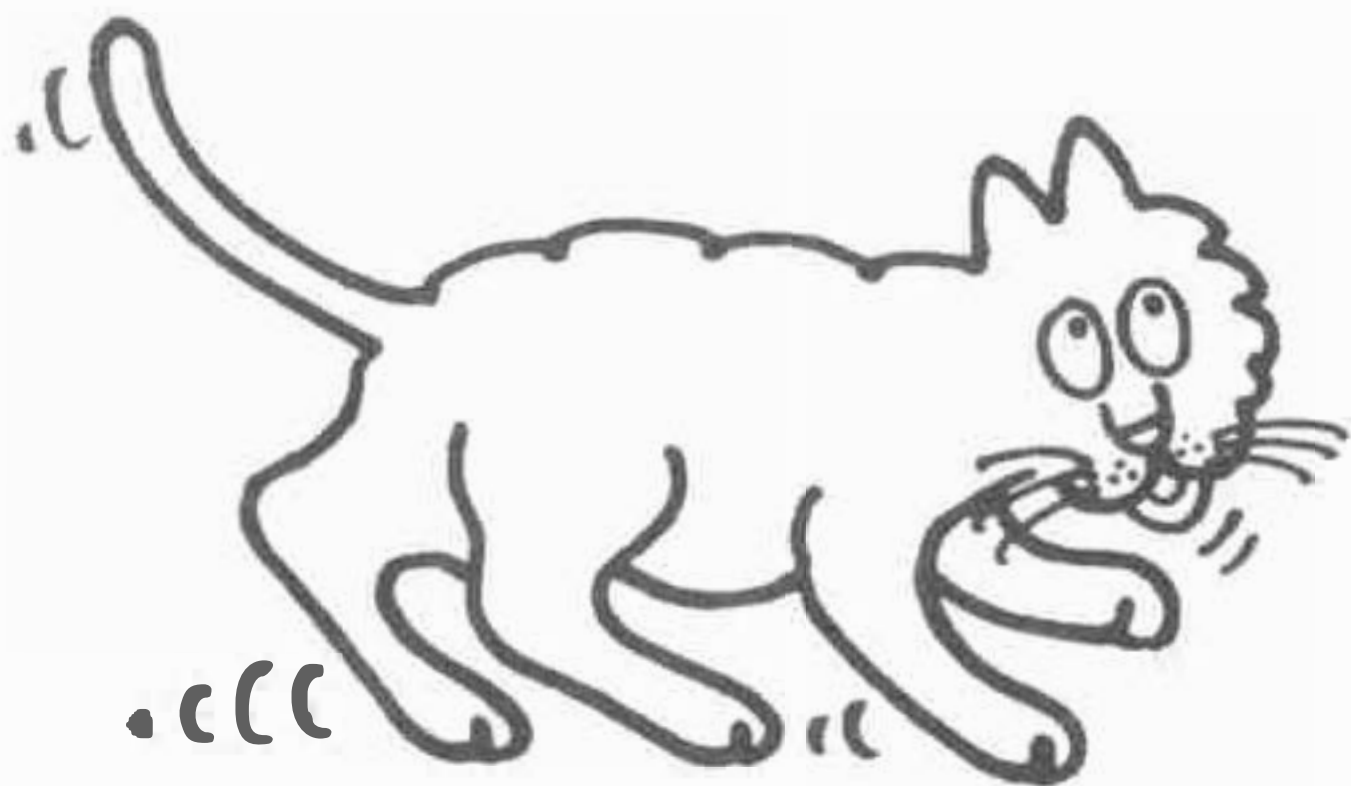
Intérêt par remboursement = Solde de capital $\times$ intérêt annuel

Montant de capital remboursé par remboursement

= Montant des remboursements égaux de capital et d'intérêt - intérêt pour la période envisagée

Bien que ce calcul produise une erreur sur le montant, cette erreur est compensée en l'ajoutant au premier remboursement.

Le programme est le suivant:



```

10 REM REMBOURSEMENT
20 DEFINT A,S
30 INPUT "PERIODE DE REMBOURSEMENT";Y
40 INPUT "INTERET ANNUEL %";R
50 INPUT "CAPITAL";X
60 R1=R/100
70 A=X*R1/((1+R1)^Y-1)*(1+R1)^Y
80 PRINT
90 PRINT "MONTANT DE REMBOURSEMENTS EGAU
X";A;"FRANC"
100 Z=X
110 H=0
120 FOR I=1 TO Y
130 PRINT
140 S=Z*R1+.9 ——— Intérêt
150 W=A-S ——— Montant de capital remboursé
160 Z=Z-W ——— Solde
170 H=H+W ——— Montant total de capital à rembourser
180 PRINT I;"REMBOURSEMENT"
190 PRINT "CAPITAL";A-S;"FRANC"
200 PRINT "INTERET";S;"FRANC"
210 NEXT I
220 P=X-H ——— Erreur
230 PRINT
240 PRINT "MONTANT DE REMBOURSEMENT A AJ
OUTER A LA 1ERE ECHEANCE";P;"FRANC"
250 END

```

20 DEFINT A,S

Ceci déclare que les variables commençant par A et S sont des variables entières. Il existe deux manières de déclarer le type de variable: à l'aide d'une instruction de déclaration de type, ou par addition d'un caractère de déclaration.

	Type d'instruction de déclaration	Type de caractère de déclaration (exemple)
Type entier	DEFINT	% (A%)
Type nombre réel à simple précision	DEFSNG	! (B!)
Type nombre réel à double précision	DEFDBL	# pas de caractère (C3 #, C)
Type chaîne	DEFSTR	\$ (A\$)

Rappelez-vous qu'il existe également la fonction INT qui tronque les décimales.

Exemple: PRINT INT(3.59)
3

Lorsque ce programme est exécuté, l'affichage est le suivant:

PERIODE DE REMBOURSEMENT? 5
INTERET ANNUEL%? 9
CAPITAL? 100000
MONTANT DE REMBOURSEMENTS EGAUX 25709
FRANC

} ENTREES

1 REMBOURSEMENT
CAPITAL 16709 FRANC
INTERET 9000 FRANC

2 REMBOURSEMENT
CAPITAL 18212 FRANC
INTERET 7497 FRANC

3 REMBOURSEMENT
CAPITAL 19851 FRANC
INTERET 5858 FRANC

4 REMBOURSEMENT
CAPITAL 21638 FRANC
INTERET 4071 FRANC

5 REMBOURSEMENT
CAPITAL 23585 FRANC
INTERET 2124 FRANC
MONTANT DE REMBOURSEMENT
A AJOUTER A LA
1ERE ECHEANCE 5 FRANC

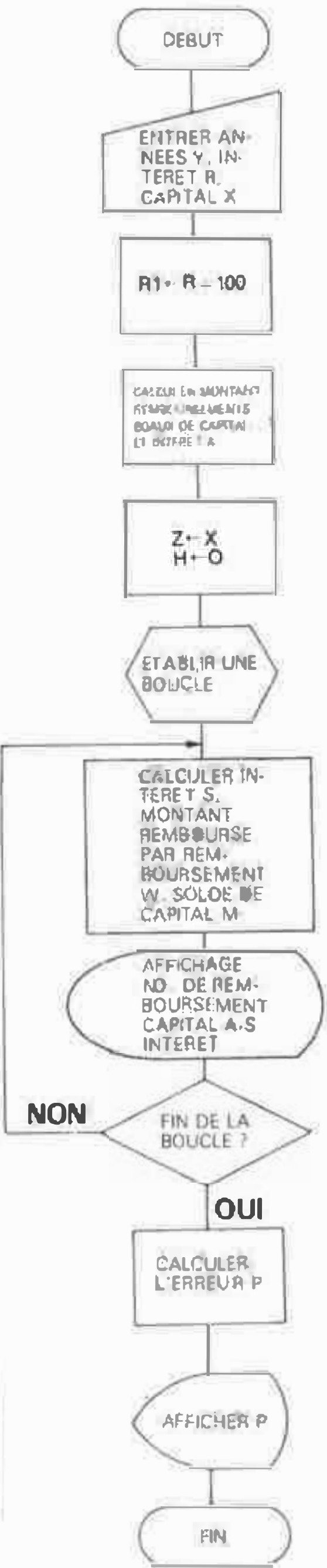
Nouvelles instructions

DEFINT Déclare les variables de type entier

DEFSNG Déclare les variables de type nombre entier à simple précision

DEFDBL Déclare les variables de type nombre entier à double précision

DEFSTR Déclare les variables de type chaînes de caractères



3.3 Marque d'un concours de golf

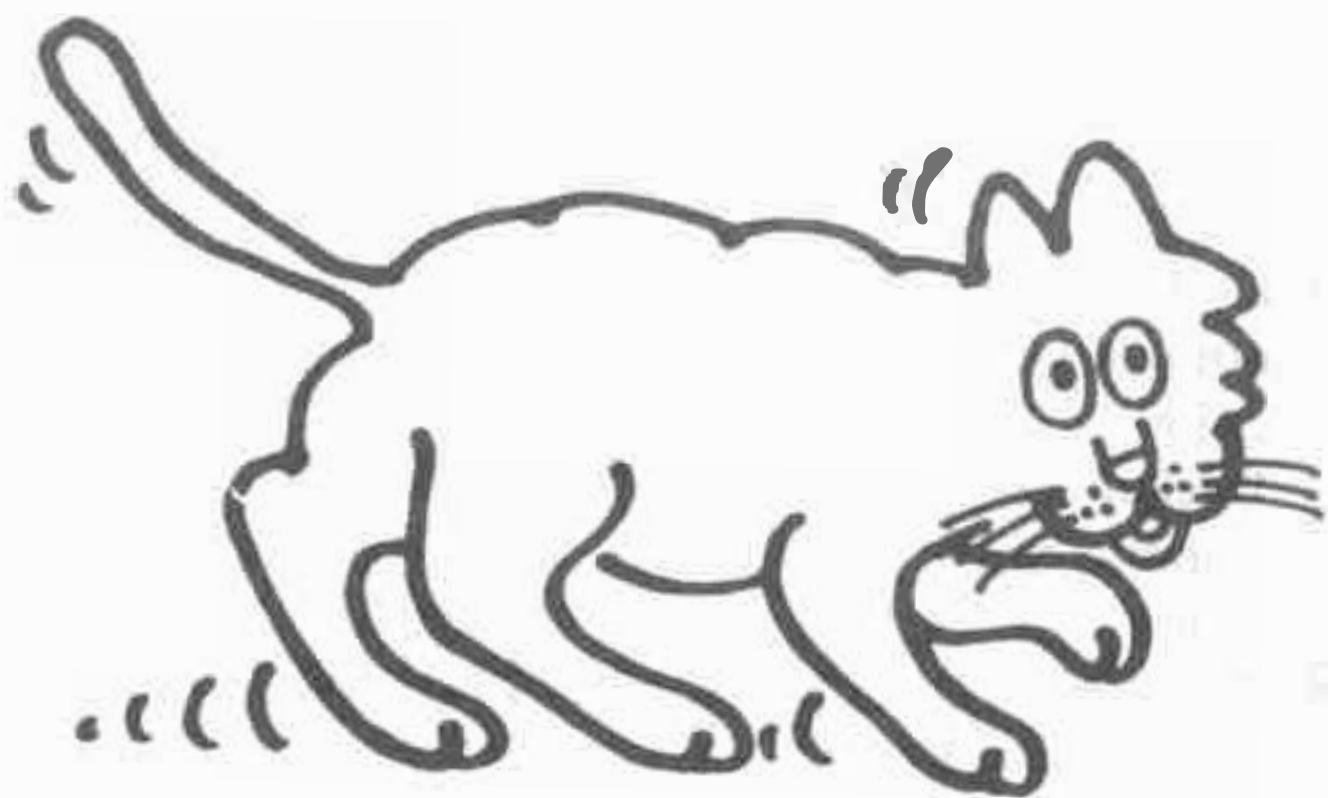
Rédigeons un programme pour calculer les scores bruts et nets de chaque joueur en introduisant les noms, les handicaps et les scores aller et retour de chaque joueur.

Les scores bruts et nets se calculent de la manière suivante:

Score brut = score aller + score retour

Score net = score brut - handicap

Voici un tel programme



```
10 REM GOLF
20 DIM N$(10),G(10,5)
30 N$=""
40 INPUT "NOM";N$
50 IF N$="" THEN 170
60 A=A+1
70 INPUT "SCORE ALLER";N1
80 INPUT "SCORE RETOUR";N2
90 INPUT "HCP";N3
100 N$(A)=N$
110 G(A,1)=N1
120 G(A,2)=N2
130 G(A,3)=N1+N2
140 G(A,4)=N3
150 G(A,5)=N1+N2-N3
160 GOTO 30
170 GOSUB 300
180 INPUT "CORRECTION NO.";X
190 IF X=0 THEN 270
200 INPUT "NOM ";N$(X)
210 INPUT "SCORE ALLER ";G(X,1)
220 INPUT "SCORE RETOUR ";G(X,2)
230 INPUT "HCP ";G(X,4)
240 G(X,3)=G(X,1)+G(X,2)
250 G(X,5)=G(X,3)-G(X,4)
260 GOTO 180
270 PRINT CHR$(12)
280 GOSUB 300
290 END
```



```

300 REM AFFICHER
310 FOR I=1 TO A
320 PRINT
330 PRINT I
340 PRINT "NOM ";N$(I)
350 PRINT "SCORE ALLER ";G(I,1)
360 PRINT "SCORE RETOUR ";G(I,2)
370 IF INKEY$="" THEN 370
380 PRINT "BRUT ";G(I,3)
390 PRINT "HCP ";G(I,4)
400 PRINT "NET ";G(I,5)
410 IF INKEY$="" THEN 410
420 NEXT I
430 RETURN

```

Aux lignes 30 à 160, les scores bruts et nets sont calculés lorsque les données sont introduites sur le clavier, et ces valeurs sont stockées dans le tableau.

Aux lignes 170 à 260, les données entrées peuvent être corrigées.

Les résultats finaux sont affichés à la ligne 280.

Les lignes 300 à 430 forment le sous-programme pour l'affichage des résultats. La fonction INKEY\$ de la ligne 370 lit un caractère composé sur le clavier. Lorsqu'aucune entrée n'est faite sur le clavier, il ne se produit rien (CHR\$(0): il n'y a rien). De même, jusqu'à ce qu'une touche soit pressée, l'exécution du programme reste arrêtée à la ligne 370. Ceci vaut également pour la ligne 410. Ces lignes sont composées de

telle sorte qu'après que 3 lignes aient été affichées sur l'écran, celui-ci ne roule pas avant qu'une touche soit enfoncée.

Exécutons maintenant ce programme.

Entrée de données

NOM? CHARLES

SCORE ALLER? 50

SCORE RETOUR? 48

HCP? 5

NOM? MARC

SCORE ALLER? 55

SCORE RETOUR? 52

HCP? 10

NOM? ANNE

SCORE ALLER? 43

SCORE RETOUR? 50

HCP? 2

NOM? [←pression de la touche RETURN]

CORRECTION NO. ? 0

Résultat final (S'il n'y a pas de correction, on entre 0 pour "CORRECTION NO.")

1

NOM CHARLES

SCORE ALLER 50

SCORE RETOUR 48 [←frapper une touche ici]

BRUT 98

HCP 5

NET 93 [←frapper une touche ici]

2

NOM MARC

SCORE ALLER 55

SCORE RETOUR 52 [←frapper une touche ici]

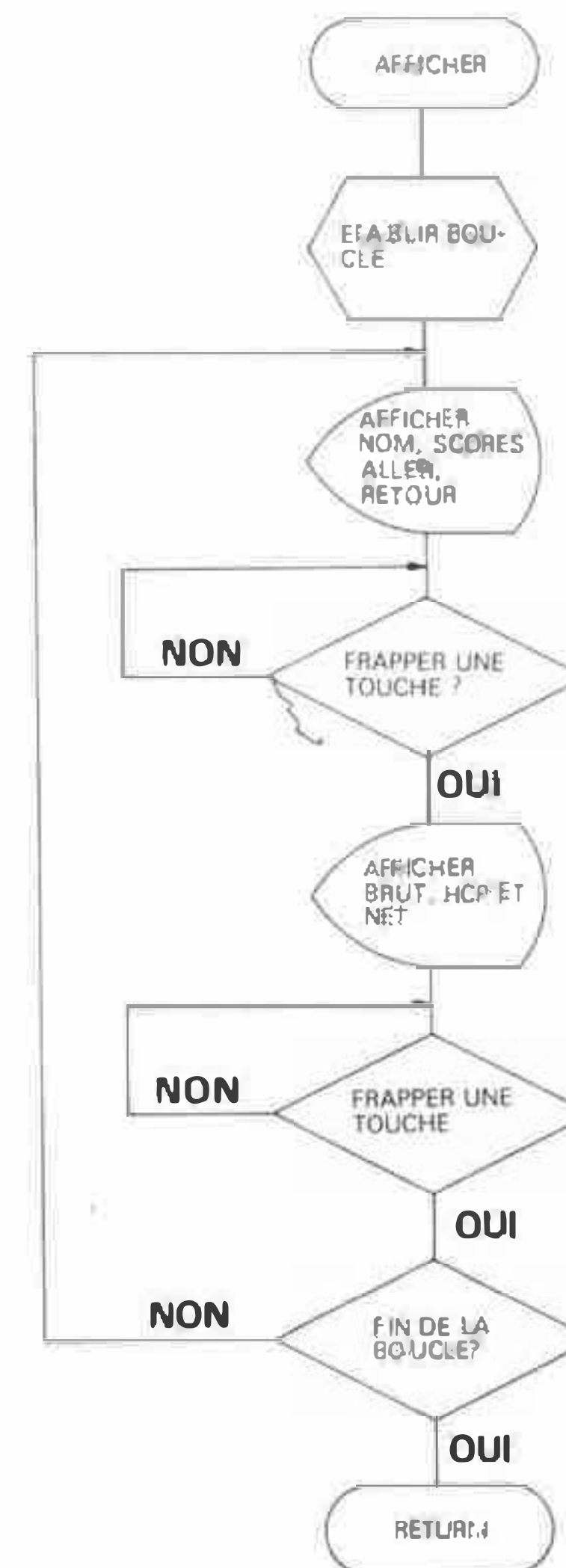
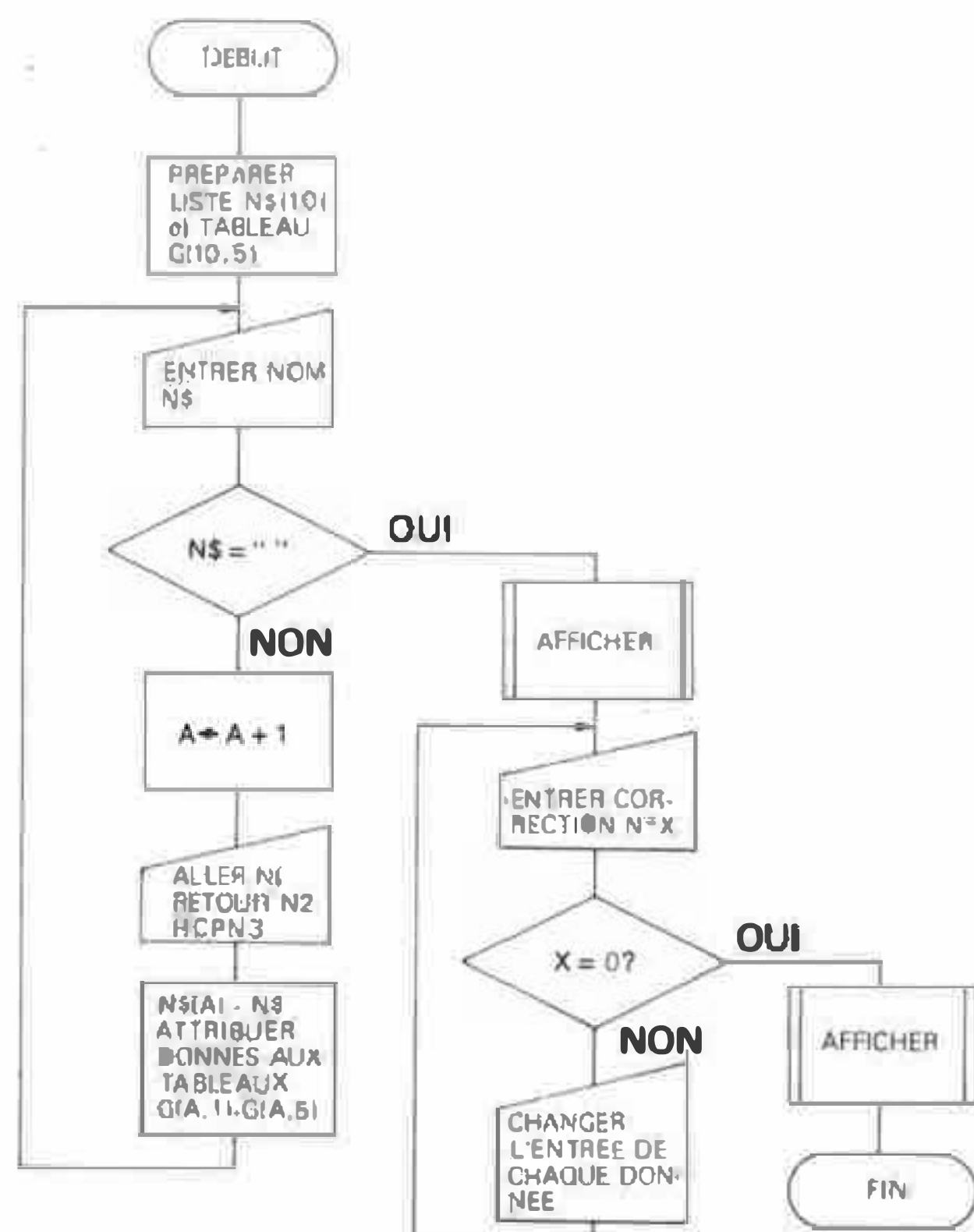
BRUT 107

HCP 10

NET 97 [←frapper une touche ici]

3

NOM ANNE
 SCORE ALLER 43
 SCORE RETOUR 50 (←frapper une touche ici)
 BRUT 93
 HCP 2
 NET 91 (←frapper une touche ici)



Nouvelle instruction

INKEY\$

Fonction lisant un caractère frappé sur le clavier.

3.4 Gestion d'un journal

Créons un programme permettant de sauvegarder un fichier de journal sur bande cassette ou de lire ce journal stocké sur bande cassette.

A. Comment créer un fichier de journal

Le programme suivant permet de créer un fichier de journal sur bande cassette.




```

10 REM CONTROLE
20 INPUT "NOMBRE DE DONNES";N [Entrée no. de données]
30 DIM A$(N),B(N)
40 FOR I=1 TO N
50 INPUT "NOM";A$(I) [Entrée nom]
60 INPUT "NOTE";B(I) [Entrée note]
70 NEXT I
80 INPUT "CASSETTE OK";O$ [Ouvrir magnéto-cassette]
90 INPUT "NOM DE FICHIER";N$ [Entrée désignation de fichier]
100 INIT #1,"CASO:" [Ouvrir magnéto-cassette en mode écriture]
110 PRINT #1,N$ [Sort désignation de fichier]
120 PRINT #1,N [Entrée no. de données]
130 FOR I=1 TO N
140 PRINT #1,A$(I),B(I) [Ecrit noms et notes]
150 NEXT I
160 END

```

Premièrement, le nombre de données N est composé sur le clavier puis lu. Ensuite, le nombre de noms et de notes N est composé sur le clavier, puis est lu pour être enregistré sur bande cassette. Examinons maintenant les lignes relatives à la création du fichier sur bande cassette.

100 INIT #1, CASO:"

Un fichier de données est ouvert sur la bande cassette en mode de sortie. La cadence de transmission est pré réglée à 1200 baud et le mode de transmission est fixé sur B.

110 PRINT #1, N\$

120 PRINT #1, N

Le contenu de la variable N\$ (soit désignation de fichier, que nous appellerons SEI), et le nombre de données N sont placés dans le fichier de données qui vient d'être ouvert.

140 PRINT #1, A\$(I),B(I)

De la même manière que ci-dessus, les noms et notes sont inscrits dans le fichier SEI.

La procédure générale de création d'un fichier est la suivante:

INIT #n, "CASO: <désignation de fichier>"	Ouverture
PRINT #n	Lecture

Nota: <désignation de fichier> est fictif, et n'est pas inscrit sur la bande cassette. Par conséquent, il peut être omis. De ce fait, le vrai nom du fichier (SEI dans l'exemple de programme ci-dessus) est inscrit à son début, et il sert à vérifier le fichier lors de sa lecture ultérieure.

De même, il va de soi que pour enregistrer sur la bande cassette, il faut que l'enregistreur à cassette se trouve en mode d'enregistrement. Pour cette raison, la ligne 80 demande si tout est en ordre.

Exécutons ce programme de cette manière:

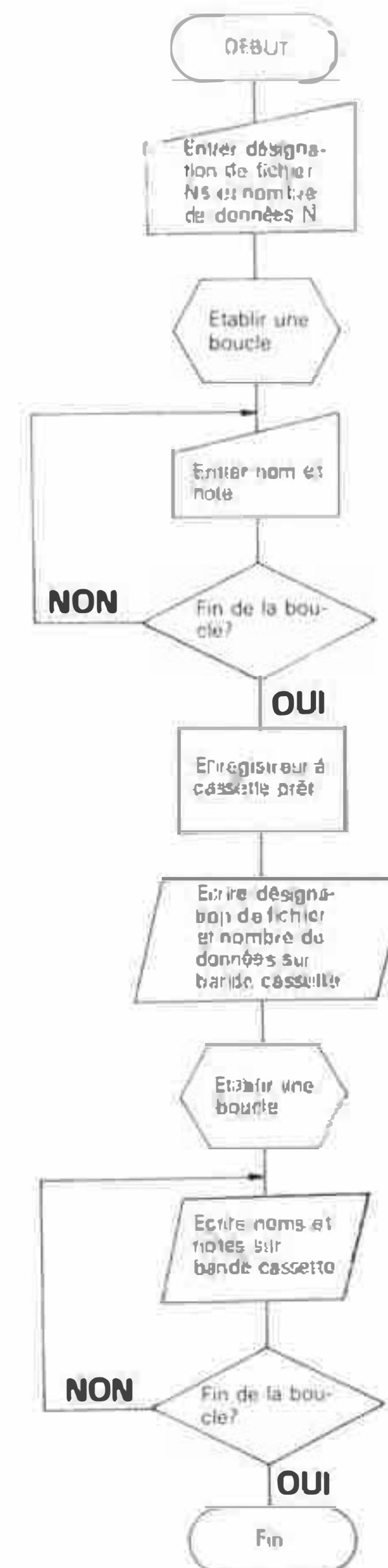
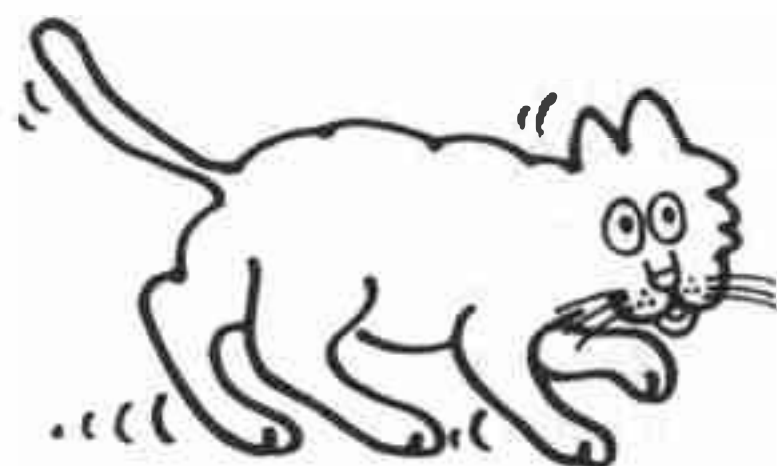
NOMBRE DE DONNEES? 4
 NOM? ROBERT
 NOTE? 76
 NOM? JEAN
 NOTE? 32
 NOM? ANNE
 NOTE? 90
 NOM? RICHARD
 NOTE? 55
 CASSETTE OK?.....
 NOM DE FICHIER? SEI

Réglez l'enregistreur à cassette pour l'enregistrement, et frappez la touche RETURN.

Le fichier SEI est crée sur la bande cassette de la manière suivante:

SEI

4	ROBERT		JEAN		ANNE		RICHARD	
---	--------	--	------	--	------	--	---------	--



Nouvelle instruction

INIT #n Ouvre un fichier
 PRINT #n Ecrit un fichier

B. Calcul de statistiques à partir de journaux

Le programme ci-après lit les données à partir de fichiers sur cassette précédemment créés, puis calcule le total, la moyenne et l'écart standard de ces données.

Les expressions pour le calcul du total, de la moyenne et de l'écart standard sont les suivantes:

n = nombre d'élèves

$x_1, x_2, \dots, x_n$ = notes des élèves

$S = x_1 + x_2 + \dots + x_n$

$\bar{x} = S/n$

$V = (x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \dots + (x_n - \bar{x})^2$

$SD = \sqrt{V/n}$

```
10 REM CONTROLE DE LECTURE
20 INPUT "DESIGNATION DE FICHIER";N$
30 INPUT "REBOBINAGE DE LA BANDE";O$
40 PRINT "PRESSER TOUCHE DE LECTURE"
50 INIT#1,"CASI:"
60 INPUT#1,F$
70 IF F$=N$ THEN PRINT "Trouve":N$ ELSE
  60
80 INPUT#1,N
90 DIM B(N)
100 FOR I=1 TO N
110 INPUT#1,A$,B(I)
120 S=S+B(I)
130 PRINT A$;" " :B(I)
140 NEXT I
```

```
150 AU=S/N
160 FOR I=1 TO N
170 U=U+(B(I)-AU)^2
180 NEXT I
190 SD=SQR(U/N)
200 PRINT "TOTAL";S
210 PRINT "MOYENNE":
220 PRINT USING "###.##";AU
230 PRINT "ÉCART STANDARD":
240 PRINT USING "###.##";SD
250 END
```

Moyenne

Ecart standard

Premièrement, rebobiner la bande, et mettre l'enregistreur à cassette en mode de lecture. Lorsque SEI est entré en correspondance à la désignation de fichier F\$, ce programme trouve le fichier commençant par "SEI", puis lit les noms et les notes qui sont affichés sur l'écran. Les lignes destinées à la lecture des données sur la bande cassette sont les suivantes:

```
20 INPUT "DESIGNATION DE FICHIER";N$
50 INIT #1, "CASI:"
```

Le fichier de données sur cassette est ouvert par le mode d'entrée.

```
60 INPUT #1, F$
80 INPUT #1, N
110 INPUT #1, A$, B(I)
```

Les lignes 60 et 70 recherchent le fichier désiré. A la placée ligne 80, le nombre de données est placé dans la variable N.

A la ligne 110, les noms et notes sont respectivement placées dans les variables A\$ et B.

La procédure générale de lecture de données à partir d'une bande cassette est la suivante:

Nota: Comme auparavant, <désignation de fichier> n'a qu'une signification fictive.

INIT #n, "CASI: <désignation de fichier>" INPUT #n	Ouverture Lecture
---	------------------------------------

L'instruction USING fait que la moyenne et l'écart standard sont affichés avec deux chiffres après la virgule.

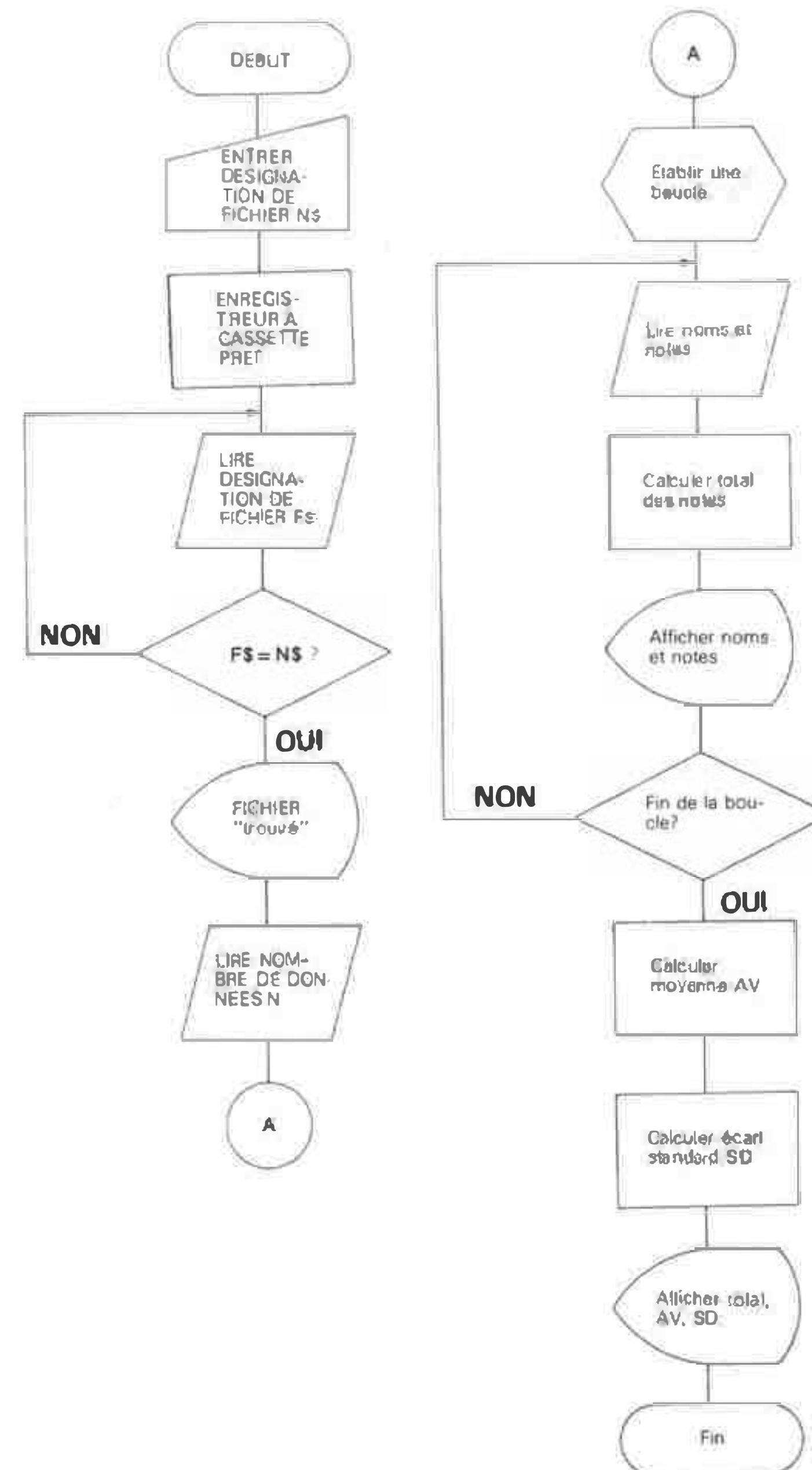
Lorsque le programme est exécuté, l'affichage est le suivant;

```

ROBERT 76
JEAN 32
ELIANE 90
RICHARD 55
TOTAL 253
MOYENNE 63.25
ECART STANDARD 21.92
    
```

Nouvelles instructions

INPUT #n SQR	Lit un fichier Fonction produisant la racine carrée
-------------------------------	--





Programmes pratiques

Au premier abord, il pourrait sembler difficile de créer des programmes pratiques, mais ils sont souvent bien plus faciles que les programmes de jeux. Lors de la création d'un tel problème, rappelez-vous simplement de ce que vous désirez entrer et sortir, et le reste viendra automatiquement pour autant que vous restiez dans la simplicité. Créez le programme de la même manière qu'un organigramme, avec une suite correcte de calculs, affichages, mises en mémoire et décision.

D'autre part, bien que les jeux soient pris à la légère, ils exigent des connaissances approfondies de programmation. D'une part, ils doivent délivrer du "plaisir", et aucune notion n'est plus ambiguë et personnelle que "plaisir".

Traduire cela clairement en un programme, cela ressemble à de l'acrobatie. Cependant, la création d'un tel programme autorise certaines imprécisions. En d'autres termes, il n'est pas gênant si à mi-chemin du programme le contenu du jeu change, car son objet a changé.

En conclusion, les programmes pratiques, qui doivent avoir une gamme d'application étendue, sont difficiles à créer, car ils n'admettent aucune imprécision ou erreur. Cependant, les programmes les plus faciles à créer sont ceux qu'on va utiliser soi-même, pour des tâches spécifiques (qu'on a choisi soi-même). Mais trêve de bavardage, essayons de créer un programme.

3.5 Annuaire téléphonique

La mémoire RAM accepte aussi bien des fichiers de programme que des fichiers de données.

Cependant, seuls des fichiers séquentiels sont admis. Dans cette section, créons un programme destiné à former un annuaire téléphonique.

Lors de la création d'un fichier en mémoire RAM, l'espace réservé au fichier doit être fixé au moyen d'un ordre FSET. Ici, fixons l'espace réservé pour fichiers à 1024 octets (FSET 1024). Si vous rencontrez une difficulté quelconque pour cette opération, retournez à "1.5 Mémoire du X-07" du guide de l'utilisateur, ou à "FSET" du manuel de référence BASIC.



A. Comment créer un fichier d'annuaire téléphonique

Ce programme crée un fichier d'annuaire téléphonique en mémoire RAM pour fichiers.

```
10 REM TEL A
20 INIT #1,"TEL",100,"P"
30 INPUT "NO. DE DONNEE";N
40 FOR I=1 TO N
50 INPUT "NOM";N$(I)
60 INPUT "TEL. NO.";T$(I)
70 NEXT I
80 PRINT #1,N
90 FOR I=1 TO N
100 PRINT #1,N$(I),T$(I)
110 NEXT I
120 END
```

Les noms et numéros de téléphone sont lus au clavier puis inscrits dans la mémoire RAM pour former le fichier TEL.

Examinons une ligne traitant de la mémoire RAM.

20 INIT #1, "TEL", 100, "P"

La ligne 20 ouvre le fichier. Concrètement, nous venons de réserver en mémoire RAM un fichier dont la désignation est TEL, d'une dimension de 100 octets (valable pour 100 caractères environ). Si l'on désire traiter beaucoup de noms, il faudra fixer un fichier plus grand.

Utilisez la commande FSET pour augmenter la longueur du document de la mémoire RAM.

30 INPUT "NO. DE DONNEE";N

Le nombre de personnes traitées est fixé à la ligne 30, et placé dans la variable N. Après la ligne 30, chaque ligne est exécutée N fois. Ici, si l'on désire traiter les données pour plus de 10 personnes, il faut déclarer des listes de variables comme après la ligne 30 au moyen de DIM N\$(N), T\$(N).

80 PRINT #1, N
100 PRINT #1, N\$(I), T\$(I)

La ligne 80 écrit le nombre de données (N) au début du fichier TEL. La ligne 100 écrit consécutivement les noms N\$(I) et numéros de téléphone dans le fichier TEL. #1 définit le numéro de fichier, qui doit être identique au numéro de fichier spécifié par l'instruction INIT#. La procédure générale de création d'un fichier de données en mémoire RAM est la suivante:

INIT # n	"désignation de fichier",	nombre d'octets,
	"type de fichier"	
PRINT #n		Ouverture Ecriture

Exécutons ce programme

NO. DE DONNEE? 4
 NOM? MARIE
 NO.DE TEL.? 987-4543
 NOM? GEORGES
 NO.DE TEL.? 345-5432
 NOM? ANITA
 NO. DE TEL? 423-1298
 NOM? SAM
 NO. DE TEL? 110-1111

Le fichier TEL est établi comme suit:

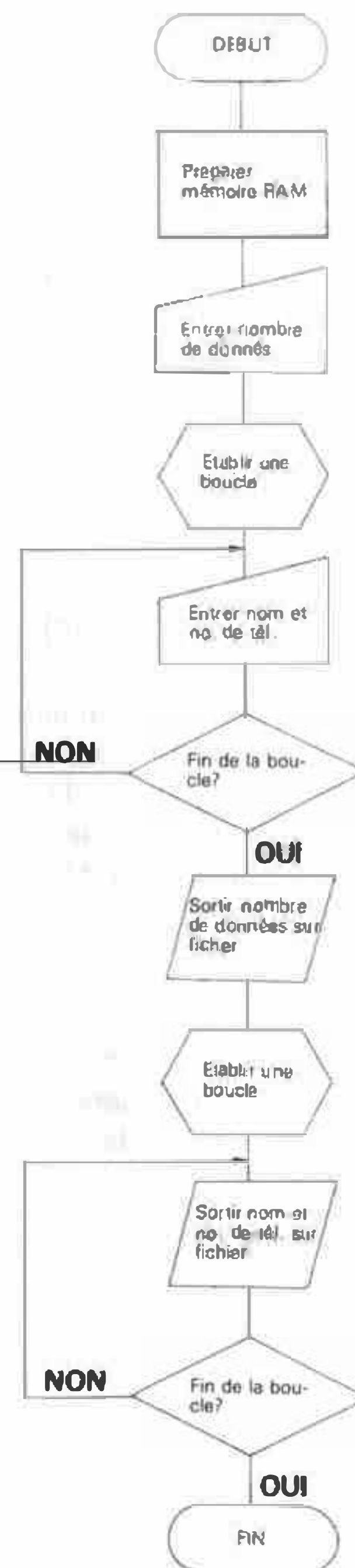
TEL

4 MARIE 987-4543 GEORGES

345-5432 ANITA 423-1298

SAM 110-1111

Assurez-vous que le fichier TEL ait été créé en mémoire RAM au moyen de l'ordre DIR.



B. Comment lire un annuaire téléphonique

Rédigeons maintenant un programme destiné à lire le fichier d'annuaire téléphonique créé précédemment.

```

10 REM TEL B
20 INIT #1,"TEL"
30 INPUT #1,N
40 FOR I=1 TO N
50 INPUT #1,N$,T$
60 PRINT N$;" ";T$
70 IF INKEY$="" THEN 70
80 NEXT I
90 END
  
```

Ceci ouvre le fichier TEL RAM

```

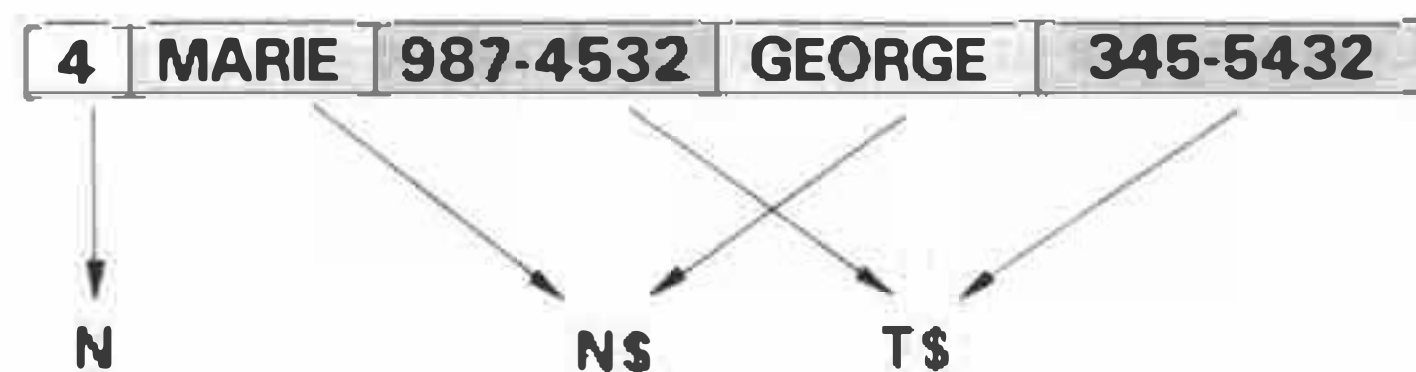
30 INPUT #1, N
50 INPUT #1, N$, T$
  
```

A la ligne 30, le nombre de données en fichier TEL est lu, et à la ligne 50, les noms et numéros de téléphone sont placés dans les variables N\$ et T\$.

```

70 IF INKEY$="" THEN 70
  
```

La ligne 70 est établie de telle manière que le prochain affichage ne soit pas obtenu avant qu'une touche soit frappée.



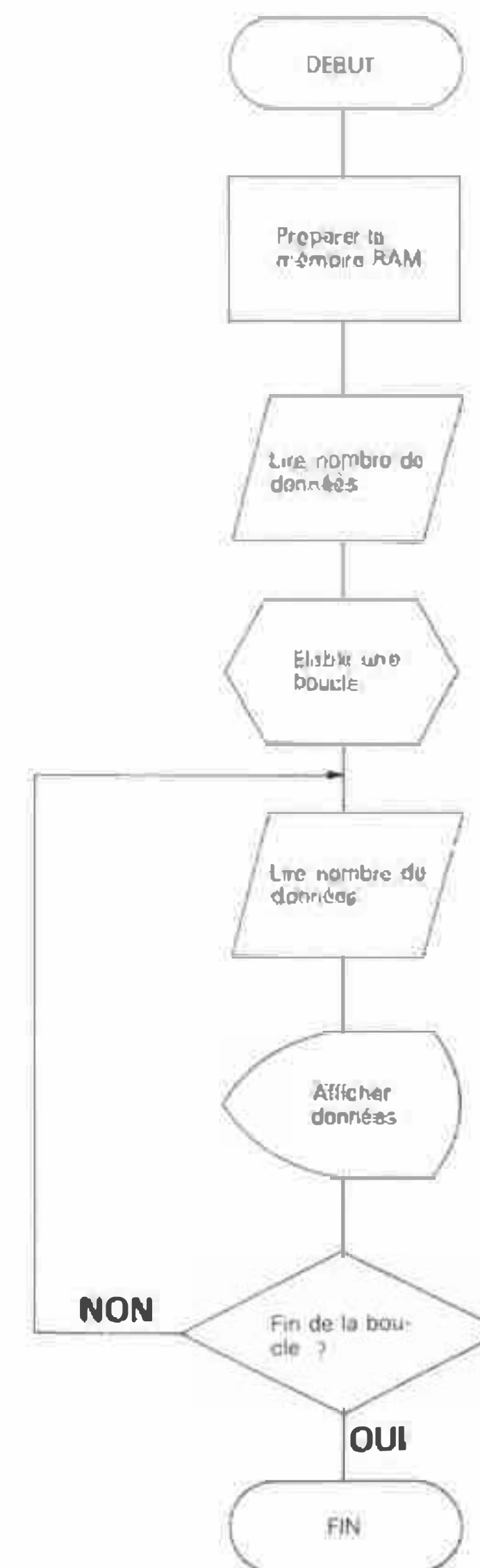
La procédure générale de lecture d'un fichier en mémoire RAM est la suivante:

INIT # n''désignation de fichier''
INPUT # n

Ouverture
Lecture

L'affichage obtenu lors de l'exécution de ce programme est le suivant.
Frappez une touche après affichage de chaque ligne.

MARIE 987-4534
GEORGES 345-5432
ANITA 432-1298
SAM 110-1111



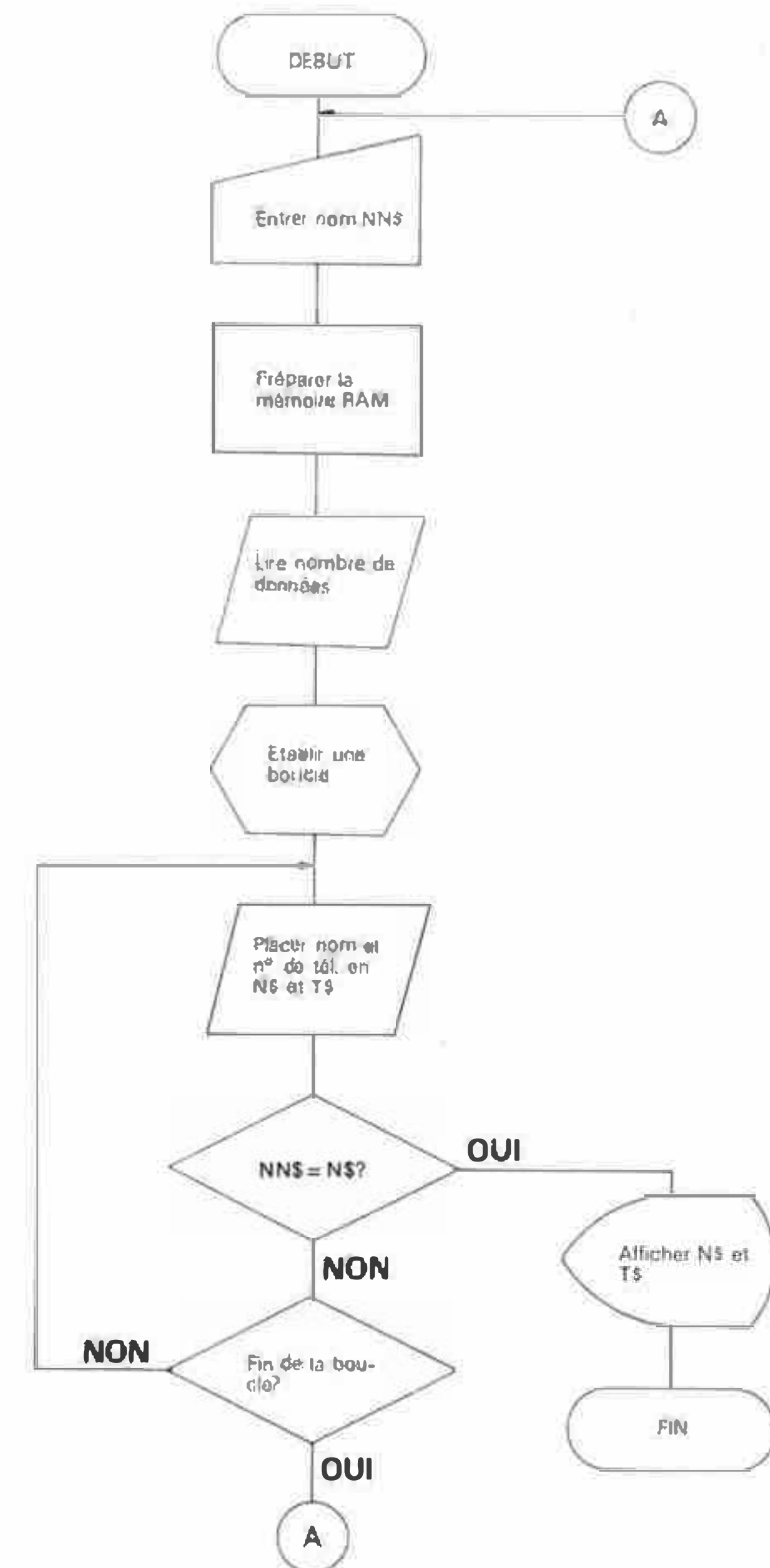
C. Comment retirer des données d'un annuaire téléphonique

Le programme suivant permet d'afficher un numéro de téléphone lorsqu'un nom est frappé sur le clavier.

```
10 REM TEL C
20 INPUT "NOM";NN$
30 INIT #1,"TEL"
40 INPUT #1,N
50 FOR I=1 TO N
60 INPUT #1,N$,T$
70 IF N$=NN$ THEN 100
80 NEXT I
90 GOTO 20
100 PRINT N$;" ";T$
110 END
```

L'affichage est le suivant lorsque ce programme est exécuté.

NOM? GEORGES
GEORGES 345-5432



3.6 Gestion d'inventaire

Autre exemple de programme utilisant la mémoire RAM pour fichiers, créons un programme de gestion d'inventaire doté des caractéristiques suivantes:

- Ajoute de nouvelles données d'inventaire
- Met à jour le nombre de marchandises en inventaire

Le programme est le suivant:

```

10 REM GESTION D'INVENTAIRE — Nombre en l'absence de fichier
20 ON ERROR GOTO 500
30 DIM A$(2,100)
40 INIT#1,"INVENTAIRE",100,"0"
50 INPUT#1,N — L'inv. de produit
60 IF N=0 THEN 100
70 FOR I=1 TO N
80 INPUT#1,A$(0,I),A$(1,I),A$(2,I) L'inv. produit
90 NEXT I
100 PRINT "1 AJOUT"
110 PRINT "2 MISE A JOUR"
120 PRINT "0 FIN"
130 INPUT F
140 IF F=1 THEN GOSUB 220
150 IF F=2 THEN GOSUB 300
160 IF F<>0 THEN 100
170 PRINT #1,N
180 FOR I=1 TO N
190 PRINT #1,A$(0,I),A$(1,I),A$(2,I) Ecrit en fichier
200 NEXT I
210 END
220 REM AJOUT

```

Affiche menu

```

230 N=N+1
240 INPUT "CODE DE PRODUIT";A$(0,N)
250 INPUT "NOM DE PRODUIT";A$(1,N)
260 INPUT "QUANTITE";A$(2,N)
270 RETURN
300 REM MISE A JOUR
310 INPUT "NOM DE PRODUIT";SC$
320 FOR I=1 TO N
330 IF A$(1,I)=SC$ THEN GOSUB 380:I=N:M=
    1
340 NEXT I
350 IF M=1 THEN 370
360 PRINT "INCONNU"
370 M=0:RETURN
380 PRINT A$(0,I),A$(1,I),A$(2,I)
390 INPUT "AUGMENTATION(+):DIMINUTION(-):"
    QUANTITE";L
400 X=VAL(A$(2,I))
410 X=X+L
420 A$(2,I)=STR$(X)
430 RETURN
500 REM TRAITEMENT D'ERREUR
510 IF ERR<>22 THEN END
520 PRINT "PAS DE FICHIER"
530 PRINT "ENTRE DONNEE (AJOUT 1)"
540 FOR I=0 TO 500:NEXT
550 RESUME 100

```

Traitement en l'absence de donnée

on s'arrête
(10 en 300)

Ce programme est exécuté comme suit.

1. Choisissez la fonction désirée lorsque l’affichage suivant est obtenu:

1 AJOUT

2 MISE A JOUR

0 FIN

?

1. Pour ajouter de nouvelles données à l’inventaire
2. Pour mettre à jour le nombre de marchandises
0. Pour terminer le programme
2. Pour ajouter de nouvelles données, entrer le code de produit, le nom de produit et la quantité.

CODE DE PRODUIT?
NOM DE PRODUIT?
QUANTITE?

3. Pour mettre à jour la quantité d’un produit, entrer le le code de produit en question.

CODE DE PRODUIT?

Ensuite, ajouter ou soustraire de la quantité existante. Pour soustraire, placer le signe “-” devant le nombre.

AUGMENTATION(+)DIMINUTION(-)QUANTITE?

4. Une erreur IO se produit lorsque l’ordinateur essaie de lire une donnée qui n’existe pas. Dans ce cas seulement, vous devez indiquer s’il n’y a pas de donnée après la ligne 500, et transférer le contrôle au menu. A partir de ce point, choisir “1” et entrée la donné. Cette procédure de traitement d’erreur est nécessaire par exemple lorsque ce programme est exécuté pour la première fois.

Le fichier INVENTAIRE est créé en mémoire RAM, et sa structure est la suivante.

No. de produit	Code de produit	Nom de produit	Quantité	Code produit	Nom de produit	Quantité	
----------------	-----------------	----------------	----------	--------------	----------------	----------	--

Le nombre de produits dans le fichier INVENTAIRE est entré au début, suivi de la donnée de produit.

Exécutons ce programme.

Ajout

1 AJOUT

2 MISE A JOUR

0 FIN

? 1

CODE DE PRODUIT? 10

NOM DE PRODUIT? PLUME

QUANTITE? 30

Ajout

1 AJOUT

2 MISE A JOUR

0 FIN

? 1

CODE DE PRODUIT? 20

NOM DE PRODUIT? CAHIER

QUANTITE? 30

Mise à jour

1 AJOUT

2 MISE A JOUR

0 FIN

? 2

CODE DE PRODUIT? 10

10 PLUME

30

AUGMENTATION(+)DIMINUTION(-)

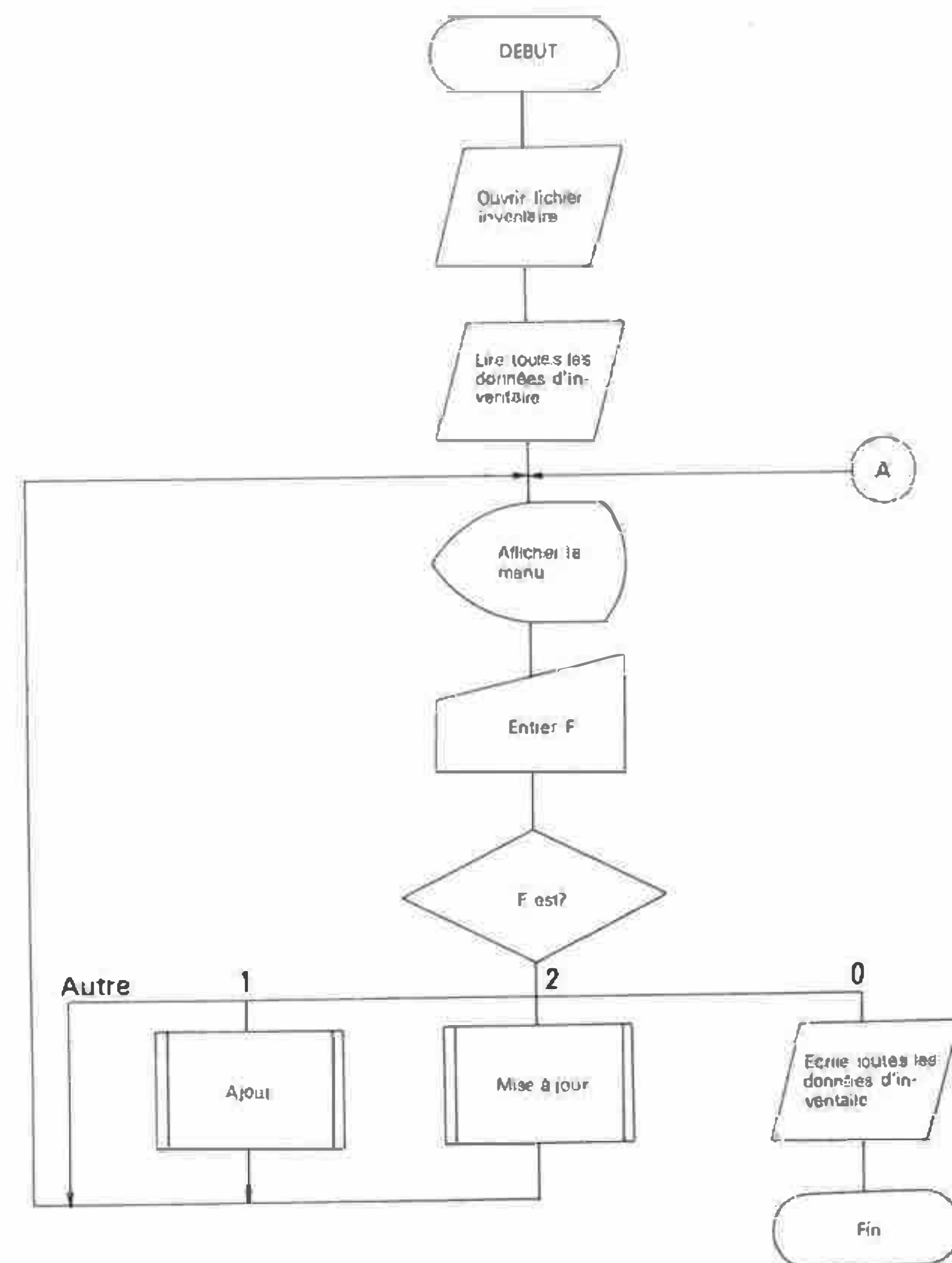
QUANTITE? 5 Augmentation

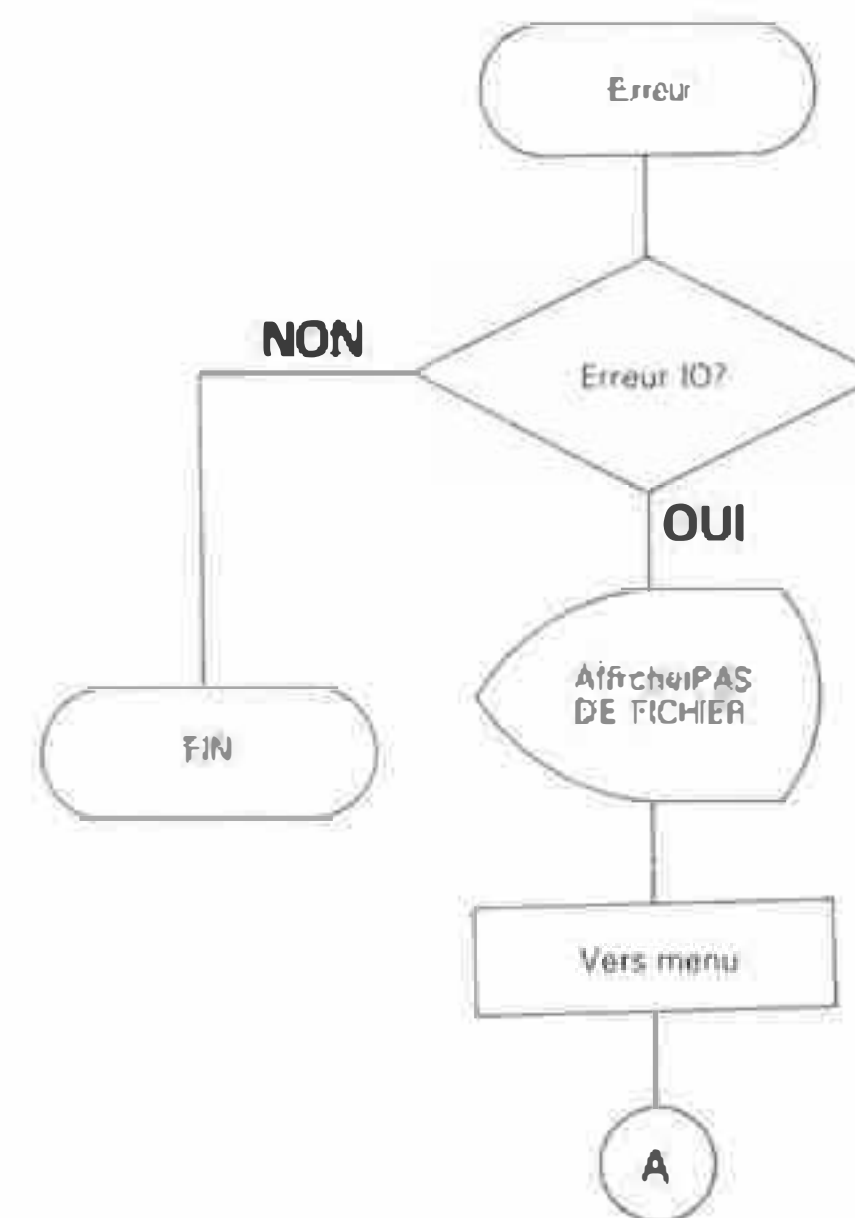
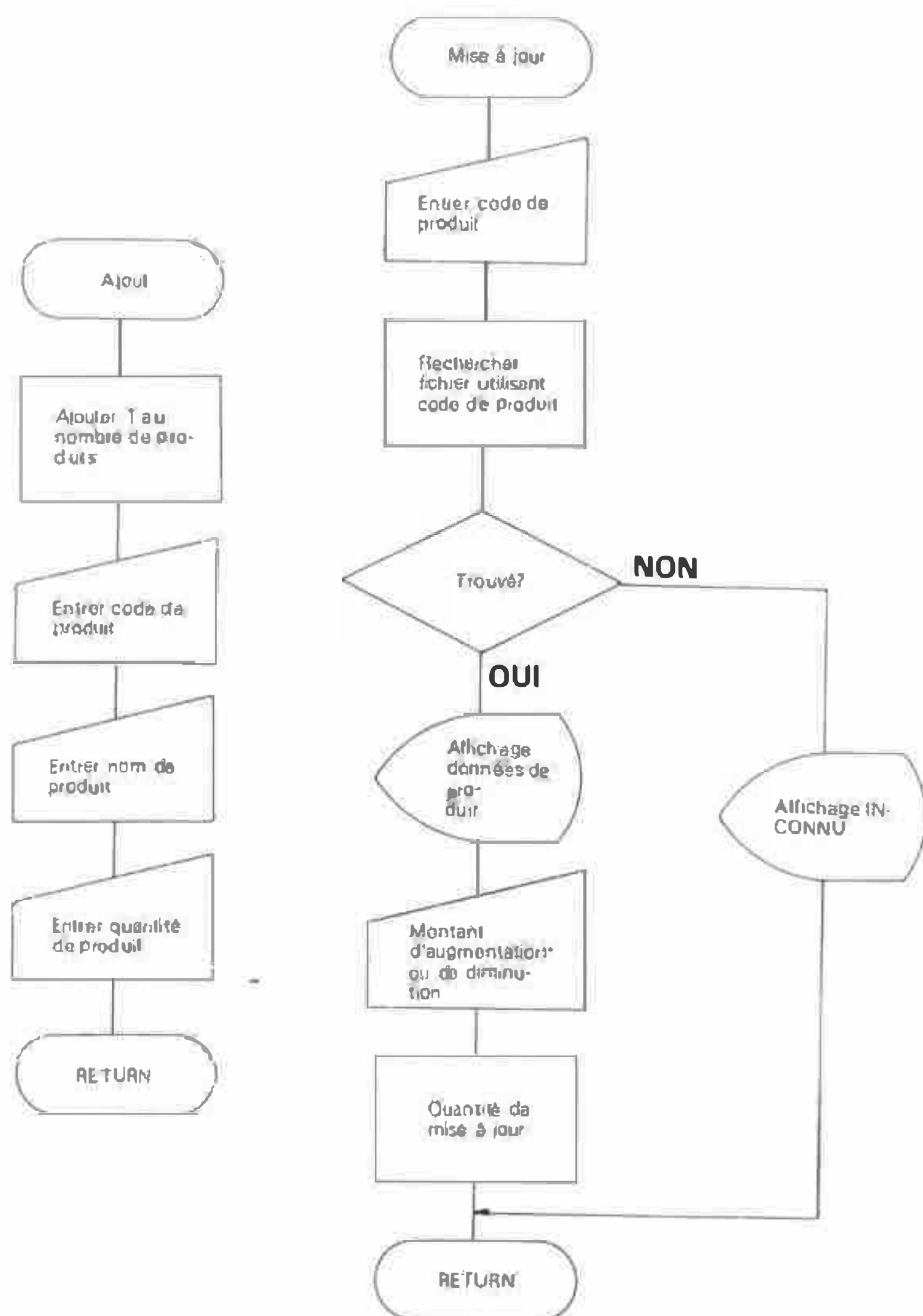
Mise à jour

1 AJOUT
2 MISE A JOUR
3 FIN
? 2
CODE DE PRODUIT? 10
10 PLUME
35
AUGMENTATION(+)DIMINUTION(-)
QUANTITE? - 10 Diminution

Fin

1 AJOUT
2 MISE A JOUR
0 FIN
? 0





Nouvelles instructions

ON ERROR GOTO	Spécifie le sous-programme de traitement d'erreur
RESUME	Rappel du sous-programme de traitement d'erreur
ERR	C'est une variable réservée avec code d'erreur (indique quelle erreur s'est produite)
STR\$	Cette fonction convertit une valeur numérique en une chaîne numérique
VAL	Cette fonction convertit une chaîne numérique en valeur numérique

Chapitre 4

Programmes tirant le meilleur parti des caractéristiques du X-07



Le X-07 possède les caractéristiques suivantes:

- **Affichage de 4×20 caractères**
- **Haut-parleur pour la reproduction musicale**
- **Affichage graphique de 32×120 points**
- **Branchement à une imprimante graphique**
- **Branchement à un coupleur optique**

Ce chapitre introduit des programmes faisant appel à ces caractéristiques.

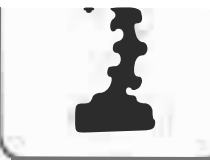


4.1 Affichage de caractères

L'affichage à cristaux liquides comprend 20 blocs x 4 lignes et ses coordonnées sont placées comme suit:

		X →																							
Y ↓		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19				
	0																								
	1																								
	2																								
	3																								

Cet écran peut sembler petit, mais il permet de créer des programmes intéressants.



A. Horloge-réveil numérique

Le programme suivant affiche une horloge numérique sur l'écran.

```
.0 REM REVEIL
20 CONSOLE@1:CLS
30 LOCATE 5,0:PRINT DATE$
40 LOCATE 5,2:PRINT TIME$
50 A$=INKEY$:I=I+1
60 IF I=10000 THEN I=0:SLEEP
70 IF A$<>"A" THEN 30
80 PRINT "HEURE DE REVEIL ACTUELLE"
   ;ALM$
90 INPUT "HEURE DE REVEIL(H,M)";B$,C$
100 ALM$="//://" + B$ + ":" + C$
110 CLS
120 GOTO 30
```

L'écran affiche l'année, le mois et le jour de la semaine. Ces indications avancent avec le passage du temps. Lorsque la touche A est frappée, le réveil peut être réglé; il sonnera pendant 1 minute à l'heure spécifiée.

20 CONSOLE @1:CLS

L'instruction CONSOLE @1 enclenche l'alarme, et l'instruction CLS efface l'écran.

30 LOCATE 5,0:PRINT DATE\$

L'instruction LOCATE déplace le curseur aux coordonnées spécifiées. Ici, il est déplacé aux coordonnées (5,0). Le prochain mot d'instruction PRINT affiche les caractères à partir de la position du curseur. La variable DATE\$ contient la date. De cette manière, la date est affichée à partir des coordonnées (5,0).

40 LOCATE 5,2: PRINT TIME\$

Semblable à ci-dessus. La variable TIME\$ contient l'heure présente, qui est affichée à partir des coordonnées (5,2).

50 A\$=INKEY\$:I=I+1
70 IF A\$ < > "A" THEN 30

La fonction INKEY\$ lit le caractère de la touche frappée. Contrairement à l'instruction INPUT, la valeur de cette touche est lue sans qu'il faille presser RETURN. Les lignes 50 et 70 lisent la valeur de la touche frappée, et vérifient s'il s'agit de [A] ou non.

60 IF I=10000 THEN I=0:SLEEP

A la ligne 60, si le résultat du compteur, qui a été incrémenté de 1 dans la dernière moitié de la ligne 50, est supérieur à 10.000, il est remis à 0 et l'instruction SLEEP est exécutée. Cette ligne coupe automatiquement l'alimentation du X-07 peu après que le déroulement du programme soit commandé par RUN, puis rouvre automatiquement le programme dès que le X-07 est mis sous tension.

80 PRINT "HEURE DE REVEIL ACTUELLE", ALM\$

La variable ALM\$ contient l'heure à laquelle le réveil est actuellement réglé; cette heure est affichée lorsque la ligne 80 est exécutée.

90 INPUT "HEURE DE REVEIL(H,M)";B\$,C\$
100 ALM\$="//://" + B\$ + ":" + C\$

A la ligne 90, les heures et minutes sont entrées, et à la ligne 100 sont placées dans la variable ALM\$ en tant que nouvelle heure de réveil. Lorsque l'heure de réveil est atteinte, la sonnerie retentit. Lors de l'exécution de ce programme, l'affichage est le suivant:

```

83/02/09, WED
11 : 39 : 46
  
```

La date et l'heure sont avancées avec le passage du temps. Lorsque la touche A est frappée à l'heure désirée, on obtient l'affichage suivant:

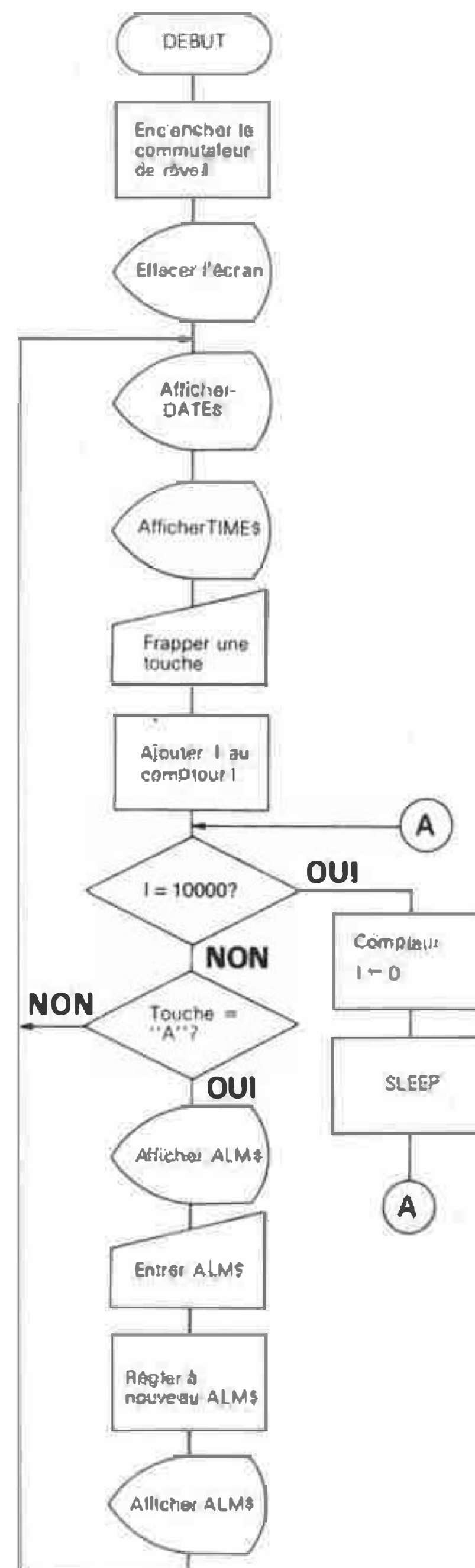
```

11 : 39 : 46
HEURE DE REVEIL ACTUELLE
**/**/**, ****, 11:20
HEURE DE REVEIL(H, M)?
  
```

Ici par exemple, si l'on introduit

15,6

le réveil sonnera à 15:06.



Nouvelles instructions

LOCATE X,Y

Déplace le curseur à (X,Y)

CLS

efface l'écran

CONSOLE @

Règle le commutateur de réveil, etc.

SLEEP

Coupe l'alimentation du X-07 tout en conservant l'état actuel.

DATE\$

Cette variable est changée selon la date

TIME\$

Cette variable est changée selon l'heure

ALM\$

Cette variable prend l'heure de réveil



B. Vidage de mémoire

Vidage de mémoire signifie lecture et affichage du contenu de la mémoire.

```
10 REM VIDAGE DE MEMOIRE
20 INPUT "DEBUT, FIN"; X, Y
30 AD$=HEX$(X) ——— [Convertit l'adresse en nombre hexadécimal]
40 PRINT AD$;" " ; — [Affiche l'adresse]
50 A=PEEK(X) ——— [Lit les données en mémoire]
60 A$=HEX$(A) ——— [Convertit les données en nombre hexadécimal]
70 PRINT A$," " ; — [Affiche les données (hexadécimal)]
80 IF A<32 THEN A=32
90 B$=CHR$(A) ——— [Convertit les données en caractères]
100 PRINT B$ ——— [Affiche les données (caractères)]
110 X=X+1
120 IF X<=Y THEN 30
130 END
```

Le contenu de la zone spécifiée de la mémoire est affiché sous la forme suivante:

Adresse (hexadécimal) Donnée (hexadécimal)
Donnée (caractère)

20 INPUT "DEBUT, FIN"; X, Y

A la ligne 20, l'adresse de début et l'adresse de fin de la mémoire à vider sont composés sur le clavier. Lorsque les variables sont séparées par une virgule, de nombreuses données peuvent être entrées avec une seule instruction INPUT.

30 AD\$=HEX\$(X)

La fonction HEX\$ convertit un nombre décimal en une chaîne de caractères hexadécimaux. A ce point, l'adresse est convertie en sa forme hexadécimale.

50 A=PEEK(X)

La fonction PEEK lit le contenu d'un octet de mémoire. Le contenu ainsi lu est stocké dans la variable A. Pour inscrire une donnée en mémoire, au lieu de lire une donnée (fonction PEEK), on utilise la fonction POKE.

PEEK
Mémoire ← Variable
POKE

```
80 IF A < 32 THEN A = 32
90 B$ = CHR$(A)
```

La fonction CHR\$ convertit le code du caractère en caractère. A la ligne 80, si la variable A est inférieure à 32, la variable B\$ est fixée vide.

Ceci est exécuté parce qu'un code de caractère de moins de 32 représente un code de contrôle, et le caractère désiré ne serait pas affiché sur l'écran. A ce point, le contenu lu en mémoire est converti en caractères. Pour convertir un caractère en son code, et non le contraire (fonction CHR\$), on utilise la fonction ASC.

Code de caractère $\xleftrightarrow[\text{ASC}]{\text{CHR\$}}$ Caractère

```
40 PRINT AD$;" ";
70 PRINT A$;" ";
100 PRINT 8$
```

Ces lignes affichent chaque adresse (nombre hexadécimal) et leur contenu (nombre hexadécimal et chaîne de caractères). Remarquez que les lignes 40 et 70 se terminent par un point-virgule (;). Dans un tel cas, le retour du curseur/interligne ne se produit pas. De ce fait, le résultat de l'instruction PRINT à la suite de celle-ci est affiché sur la même ligne. Ainsi, le contenu des variables AD\$, A\$ et B\$ est affiché sur une même ligne.

L'affichage est le suivant lorsque ce programme est exécuté.

Exemple

Pour entrée décimale

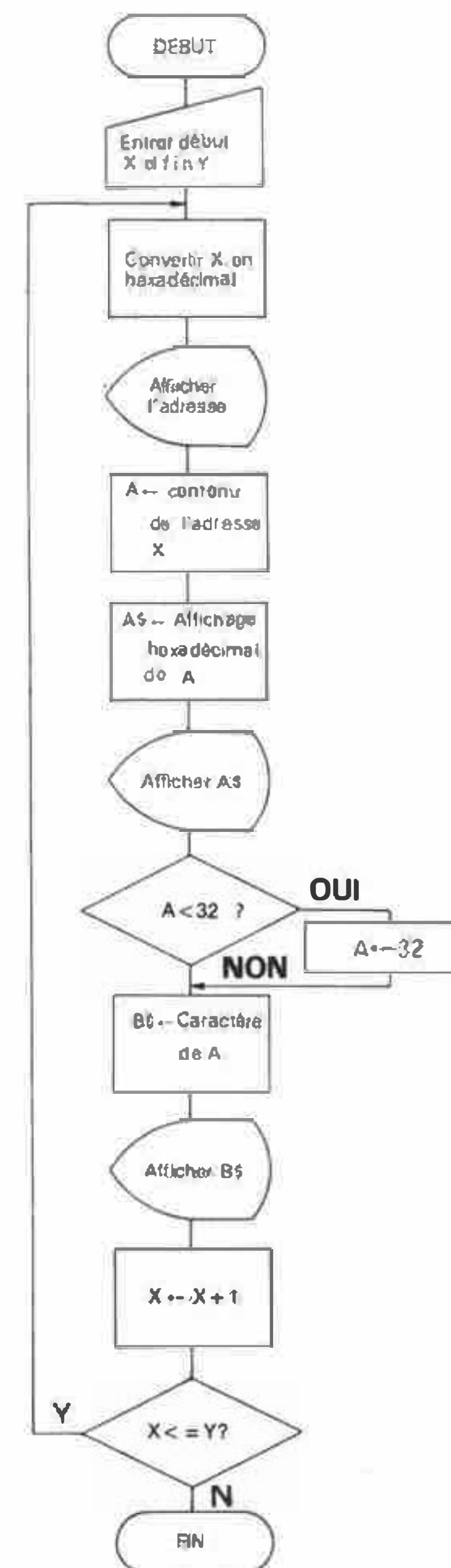
Depart, Fin? 10, 20

```
A  F5
B  64 4
C  0
D  0
E  0
F  0
10 C3 テ
11 37 7
12 F 5
13 0
14 0
```

Pour entrée hexadécimale

Depart, Fin? &H10, &H20

```
10 C3 テ
11 37 7
12 F 5
13 0
14 0
15 0
16 3
17 0
18 C3 テ
19 C7 ヌ
1A C9 ノ
1B 3A :
1C 0
1D 0
1E 0
1F 0
20 C3 テ
```



Nouvelles instructions

HEX\$ (n)	Convertit un nombre décimal n en nombre hexadécimal
PEEK (A)	Lit le contenu de l'adresse A
POKE A,n	Ecrit n à l'adresse A
CHR\$(C)	Convertit le code de caractère C en caractère
ASC("A")	Convertit le caractère "A" en code de caractère

Le cœur de l'ordinateur

Le cœur de l'ordinateur est un LSI sur 1 "puce" appelé "CPU" (unité centrale). Cette puce est si petite qu'elle tient sur le bout d'un doigt. Elle est constituée de sorte que les tensions sur plusieurs broches établissent des tensions sur d'autres broches. Ces broches en elles-mêmes n'effectuent rien; elles sont reliées au clavier. Lorsqu'une touche est frappée, des tensions sont appliquées aux broches en fonction de la touche donnée. Un affichage à cristaux liquides est également relié à cette puce; celui-ci combine les tensions résultantes et affiche le résultat. Un affichage à cristaux liquide peut être représenté comme un assemblage complexe d'ampoules lumineuses miniatures.

On trouve ensuite une puce qui stocke les tensions générées par la pression d'une touche. (La présence et l'absence de tension sont disposées dans l'ordre.) Un circuit est ajouté pour décider de ce qui va être exécuté en fonction de la combinaison de touches frappées. Tous ces éléments constituent le cœur du X-07.

Cependant, le X-07 est bien plus compliqué, et il faudrait un livre entier pour l'expliquer correctement. La simple explication ci-dessus s'applique à tout ordinateur. Si l'on connaît ce qui est décrit ci-dessus, il est plus facile de comprendre un terme technique tel que "extension à un périphérique". Par exemple, une imprimante et un circuit RS-232C sont des "unités additionnelles pour étendre les fonctions de l'unité centrale", qui réagissent aux signaux provenant de cette dernière et des broches des circuits intégrés qui la constituent.

C. Image miroir

Ce programme présente une image miroir de l'affichage.

```
10 REM INVERSION
20 CLEAR 200: DIM A$(19,2)
30 FOR Y=0 TO 2
40 FOR X=0 TO 19
50 A=SCREEN(X,Y)
60 A$(X,Y)=CHR$(A)
70 NEXT X
80 NEXT Y
90 CLS
100 FOR Y=0 TO 2
110 FOR X=19 TO 0 STEP -1
120 PRINT A$(X,Y);
130 NEXT X
140 NEXT Y
150 IF INKEY$="" THEN 150
160 GOTO 30
```

Place les lignes 0 à 2
sur l'écran en A\$

inverse
l'affichage

L'instruction **CLEAR** est nécessaire lorsqu'on doit utiliser un grand espace pour chaînes. Ici, on réserve un espace pour 200 caractères. Seuls 60 caractères sont affectés à la fois à la variable de caractères **A\$**; il peut donc sembler qu'un espace aussi grand n'est pas nécessaire. De la place est toutefois nécessaire pour l'opération de chaîne (ligne 60), et un espace plus grand permet une exécution plus rapide.


```

50 A = SCREEN(X,Y)
60 A$(X,Y) = CHR$(A)

```

L'instruction SCREEN lit le code de caractère se trouvant aux coordonnées spécifiées sur l'écran. A la ligne 60, le code de caractère ainsi lu est converti en un caractère et placé dans la liste de variables A\$. En d'autres termes, aux lignes 30 à 80, les caractères affichés de la ligne 0 à la ligne 2 sont placés dans la liste de variables A\$.

Aux lignes 80 à 120, les caractères placés dans la liste de variables A\$ sont affichés sur l'écran dans l'ordre droite-gauche inversé. Le terme STEP-1 de la ligne 110 diminue le compteur de la boucle FOR - NEXT d'une unité à chaque fois que cette boucle est exécutée.

L'affichage est le suivant lorsque ce programme est exécuté:

```

ABCD
EFG
RUN

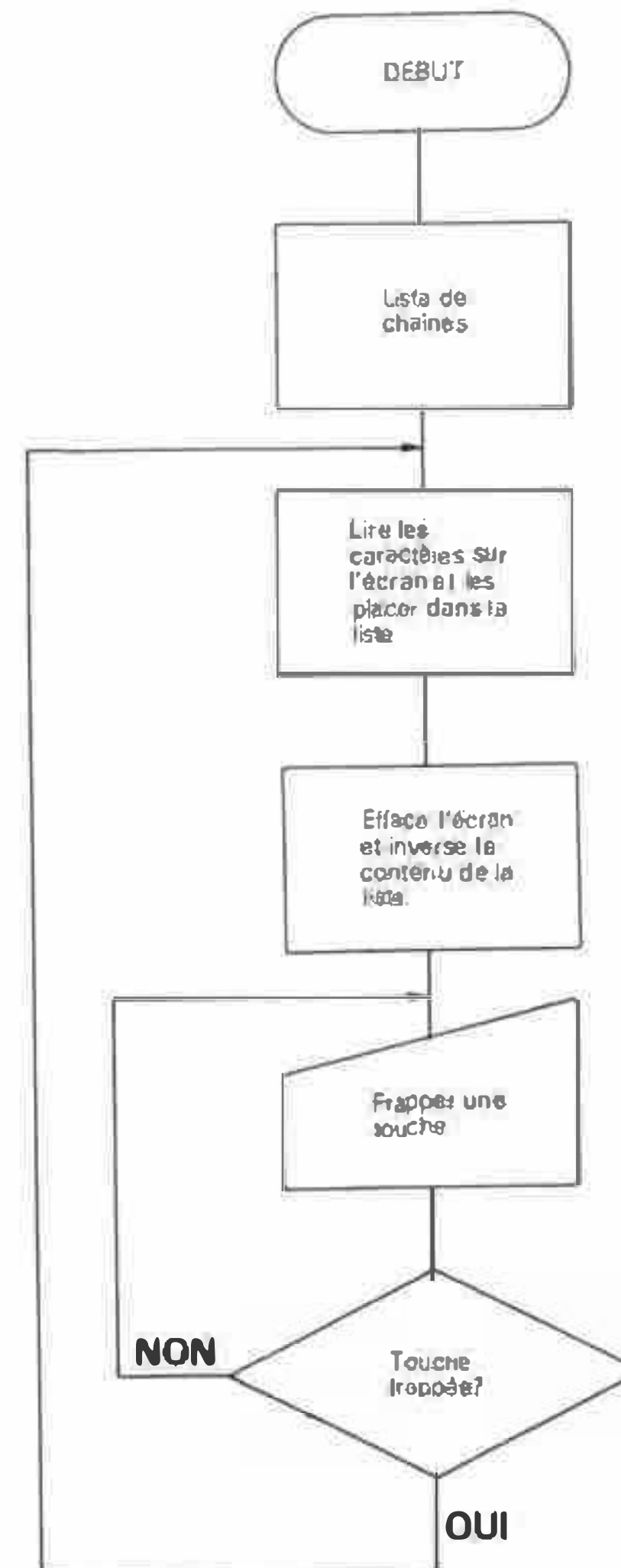
```

```

DCBA
GFE
NUR

```

Le programme effectue une boucle à la ligne 150.
L'écran est inversé chaque fois qu'une touche est frappée.



Nouvelles instructions

CLEAR

Réserve un espace pour chaîne, etc.

SCREEN (X,Y)

Lit les codes de caractères à partir de celui se trouvant à (X,Y)



D. Fixation d'un caractère

Ce programme régle le déplacement de coureurs sur l'écran

```

10 FONT$(130)=""&H38,&H38,&H10,&H7C,&H90,
&H28,&H44,&H84"
20 CLS:D=RND(0)
30 FOR Y=0 TO 2
40 A(Y)=A(Y)+RND(1)*2
50 IF A(Y)>=18 THEN A(Y)=18:SW=1
60 LOCATE A(Y),Y
70 PRINT CHR$(130)
80 NEXT Y
90 IF SW=1 THEN END
100 FOR Y=0 TO 2
110 LOCATE A(Y),Y
120 PRINT " "
130 NEXT Y
140 GOTO 30

```

La coordonnée Y des coureurs est limitée à 0, 1 et 2. La coordonnée X est changée par des nombres aléatoires générés qui sont placés en A(0), A(1) et A(2). Lorsque le coureur atteint $X = 18$, le jeu est terminé.

**10 FONT\$(130) = &H38,&H38,&H10,&H7C,&H90,
&H28,&H44,&H84''**

La fonction FONT\$ fixe un caractère. Ici, le code de caractère 130 est fixé comme indiqué ci-dessous.

					— &H38
					— &H38
					— &H10
					— &H7C
					— &H90
					— &H28
					— &H44
					— &H84

```
20 CLS:D=RND (0)
```

Ici, `RND(0)` produit une nouvelle séquence de nombres aléatoires. Si ce terme est omis, les coureurs traversent l'écran de la même manière, à chaque fois que le programme est exécuté. Pour toujours produire une séquence différente de nombre aléatoires, rappelez-vous les points suivants:

1. Utiliser $D = \text{RND}(0)$ près du début du programme.
2. Utiliser $\text{RND}(1)$ par la suite.

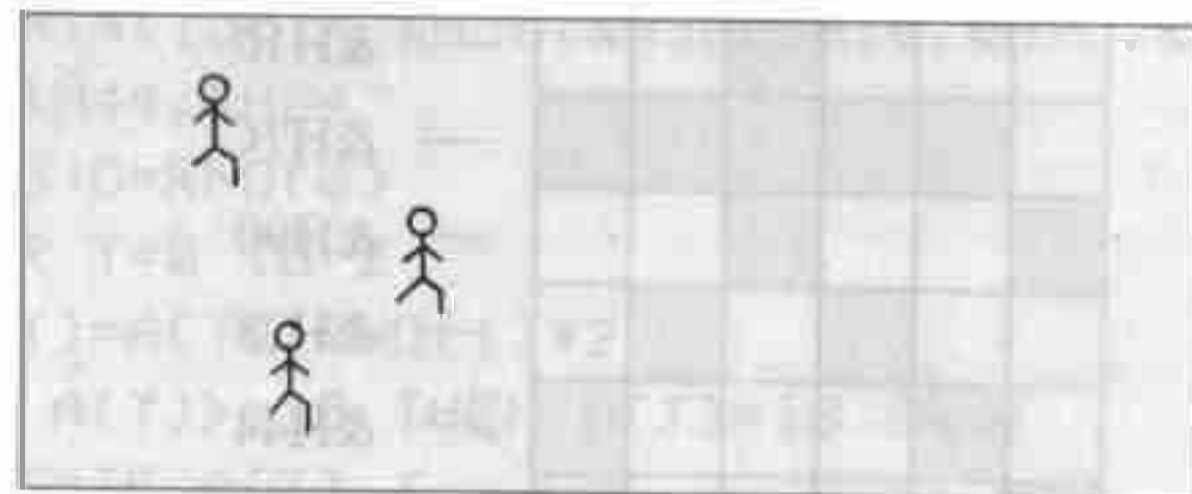
```
70 PRINT CHR$(130)
```

La ligne 70 affiche le caractère fixé à la ligne 10. Aux lignes 30 à 80 sont fixées les nouvelles coordonnées des trois coureurs, auxquelles sont affichés les caractères fixés à la ligne 10. A la ligne 40, des nombres aléatoires sont utilisés pour ajouter un nombre de 0 à 2 aux coordonnées X des trois coureurs. A la ligne 50, lorsqu'un coureur atteint la coordonnée X 18, le commutateur SW est placé à 1. Les caractères fixés à la ligne 10 sont à nouveau affichés aux lignes 60 et 70.

La ligne 90 vérifie si le jeu est arrivé à sa fin. Lorsque la fin du jeu est atteinte, le programme est terminé.

Aux lignes 100 à 130, les caractères affichés sont effacés.

Lorsque ce programme est exécuté, les trois coureurs traversent l'écran.



La ligne d'arrivée est à 2 blocs de la fin de l'écran.

La course est terminée lorsqu'un coureur atteint celle-ci.

Le gagnant est différent à chaque fois, de sorte que ce programme peut être exécuté de nombreuses fois avec plaisir.

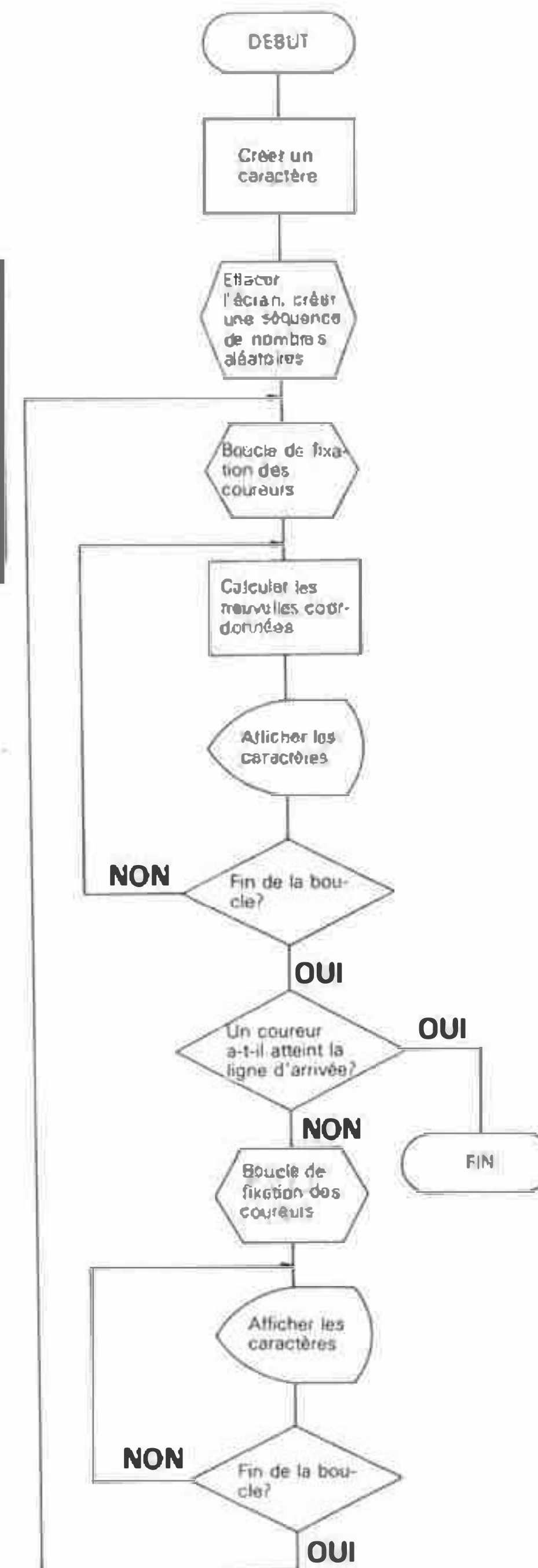
Nouvelles instructions

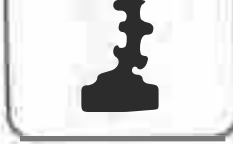
FONT\$ (code de caractère)

Fixe un caractère

RND

Produit des nombres aléatoires





4.2 Reproduction d'un passage musical

L'instruction BEEP peut être utilisée pour produire un son.

Il est également possible de reproduire un morceau de musique entier. Dans cette section, nous présentons deux programmes utilisant l'instruction BEEP qui permet de faire de la musique.

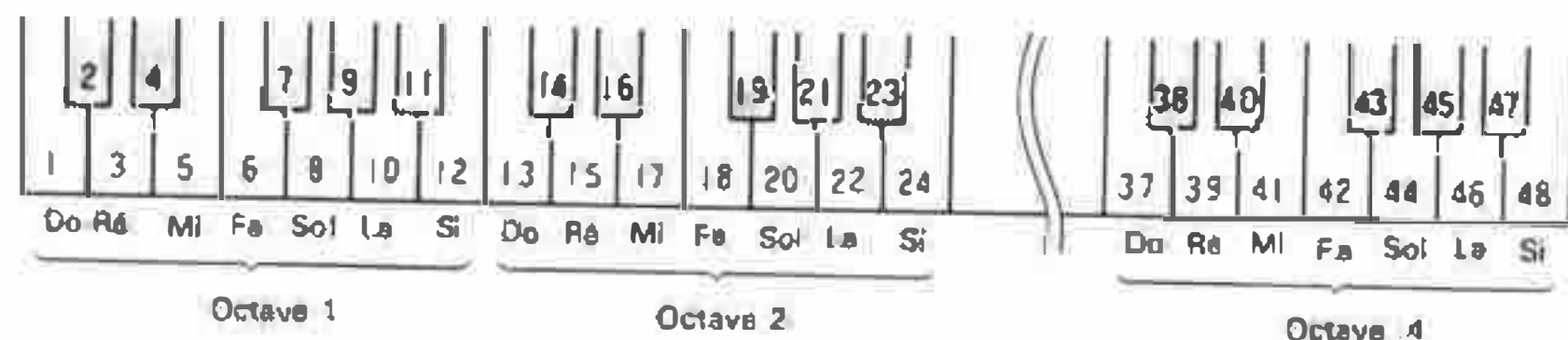
A. Quand trois poules vont aux champs

Reproduisons "Quand trois poules vont aux champs".

Un son et sa longueur est fixé au moyen de l'instruction BEEP de la manière suivante:

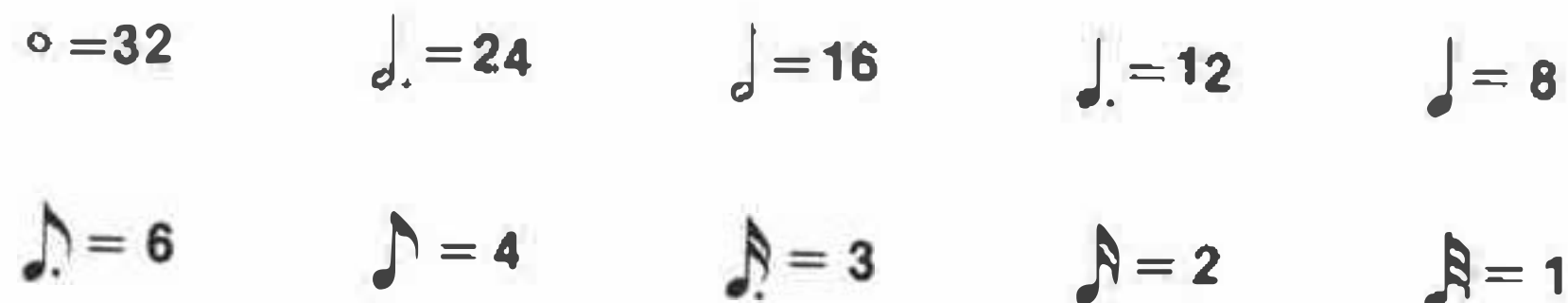
BEEP I,J

Le son est produit sur 4 octaves par un nombre compris entre 0 et 48.

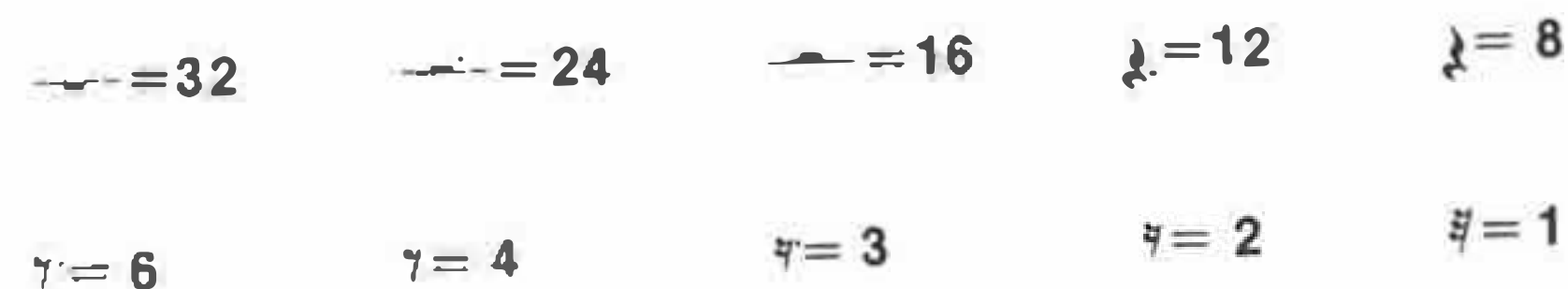


De plus, lorsqu'une note est fixée à 0, aucun son n'est produit.

Cela permet d'annuler une note. La longueur d'une note peut être comprise entre 0 et 255. Les notes et leur longueur peuvent être fixées comme suit:



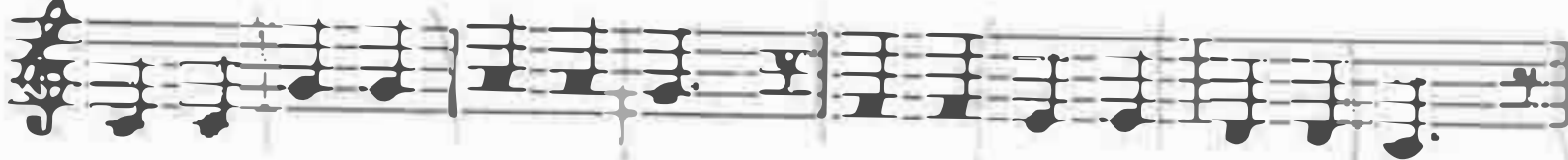
La longueur des silences est également réglable.



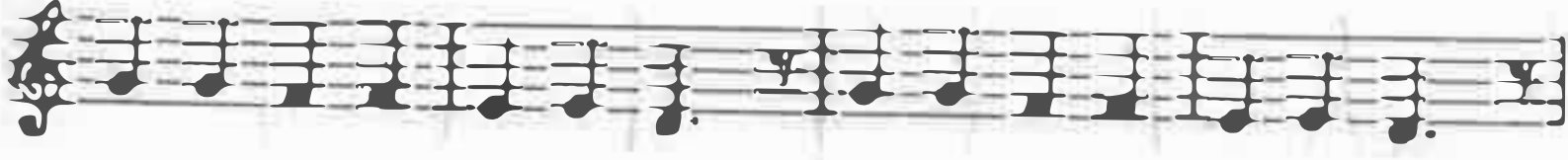
Lorsque ces valeurs sont doublées, la longueur des notes et des silences l'est également. En d'autres termes, le tempo de la musique est réduit de moitié.

La partition de "Quand trois poules vont aux champs" est donnée ci-dessous. La note et la longueur de chaque son et silence sont indiqués au-dessous.

Quand trois poules vont aux champs



Note	1	1	8	8	10	10	8	0	6	6	5	5	3	3	1	0
Longueur J	8	8	8	8	8	8	12	4	8	8	8	8	8	8	12	4



Note	8	8	6	6	5	5	3	0	8	8	6	6	5	5	3	0
Longueur J	8	8	8	8	8	8	12	4	8	8	8	8	8	8	12	4



Note	1	1	8	8	10	10	8	0	6	6	5	5	3	3	1
Longueur J	8	8	8	8	8	8	12	4	8	8	8	8	8	8	16



Le programme de "Quand trois poules vont aux champs" est le suivant:

```

10 DIM A(20),B(20),C(20)
20 DATA 1,1,8,8,10,10,8,0,6,6,5,5,3,3,1,0
30 DATA 8,8,6,6,5,5,3,0,8,8,6,6,5,5,3,3,0
40 DATA 8,8,8,8,8,8,12,4,8,8,8,8,8,8,12,4
50 FOR I=1 TO 16
60 READ A(I)
70 NEXT I
80 FOR I=1 TO 16
90 READ B(I)
100 NEXT I
110 FOR I=1 TO 16
120 READ C(I)
130 NEXT I
140 FOR I=1 TO 16
150 BEEP A(I),C(I)
160 NEXT I
170 FOR I=1 TO 16
180 BEEP B(I),C(I)
190 NEXT I
200 FOR I=1 TO 16
210 BEEP A(I),C(I)
220 NEXT I
230 END
    
```

Lit les Instructions DATA

Reproduit la première mesure

Reproduit la seconde mesure

Reproduit la troisième mesure

Les sons sont spécifiés par l'instruction DATA des lignes 20 et 30, et leur longueur par l'instruction DATA de la ligne 40. Les instructions BEEP des lignes 150, 180 et 210 reproduisent respectivement la première, seconde et troisième mesures.



Nouvelle instruction

BEEP I,J Produit le son I pendant la longueur J

B. Clavier

Ce programme permet d'utiliser le clavier comme un piano.

```

10 REM PIANO
20 I$=INKEY$
30 IF I$="" THEN 20
40 I=INSTR("ZSXDCVGBHJNM",I$)
50 BEEP I,4
60 GOTO 20
  
```

Le son est placé en I selon la touche frappée.

Un son est délivré lorsqu'une touche est frappée. Les touches correspondent aux sons suivants.

	2	4		7	9	11	
	S	D		G	H	J	
1	3	5	6	8	10	12	13
Z	X	C	V	B	N	M	,

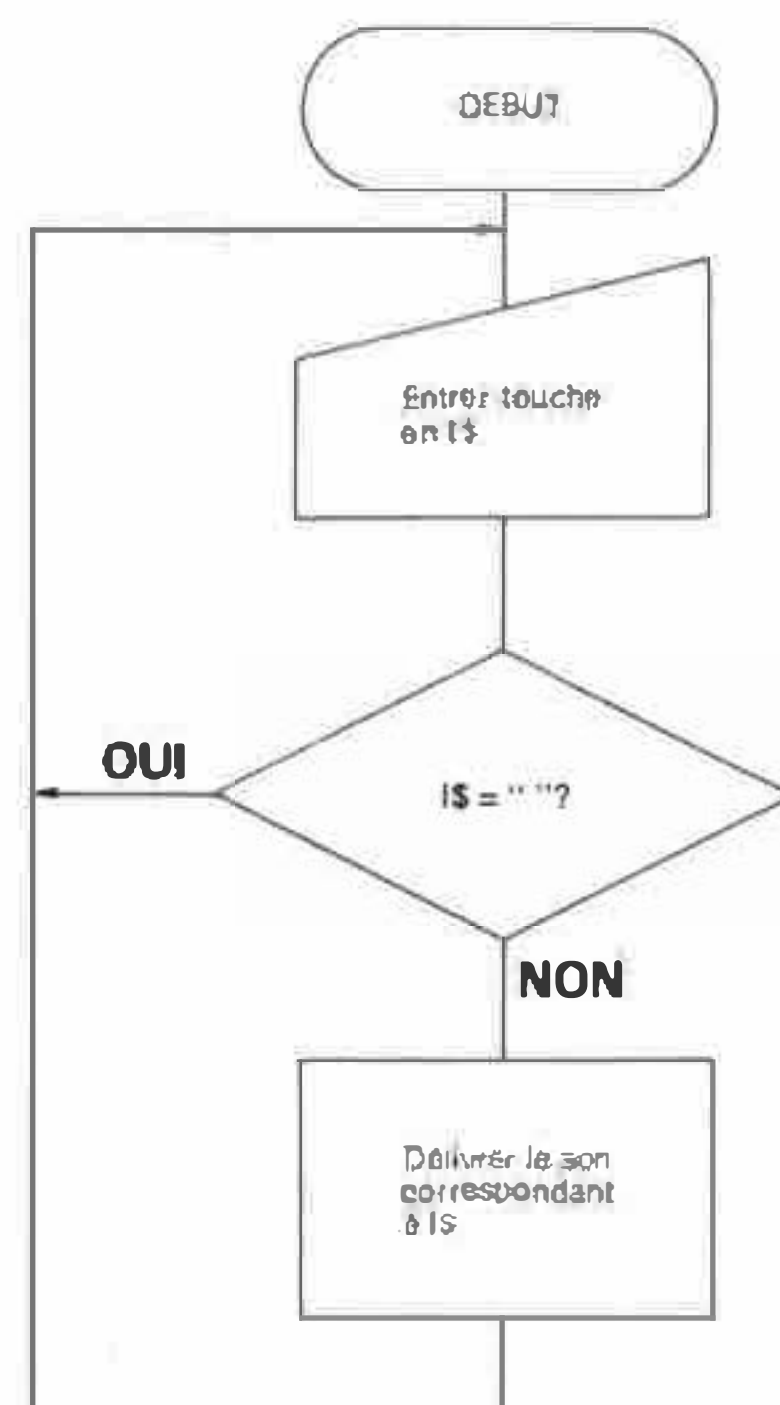
Son I = 1 ~ 13

A la ligne 20, une touche est frappée. La ligne 30 vérifie si une touche a été frappée, et si ce n'est pas le cas, la ligne 20 est reprise. La boucle ligne 20-ligne 30 est exécutée continuellement jusqu'à la pression d'une touche. La ligne 40 détermine quelle touche a été frappée, et le son correspondant est placé dans la variable I. Si l'on frappe une touche ne correspondant pas à une note, il n'est pas produit de son et l'exécution se remet en attente de la frappe d'une touche.



A la ligne 50, le son correspondant à la touche pressée est reproduit. La longueur du son est fixée à 4. Avant d'exécuter ce programme, il est préférable d'annuler le déclic des touches en exécutant `CONSOLE,,,0`.

Maintenant, utilisez le clavier pour jouer de la musique.



Nouvelle instruction

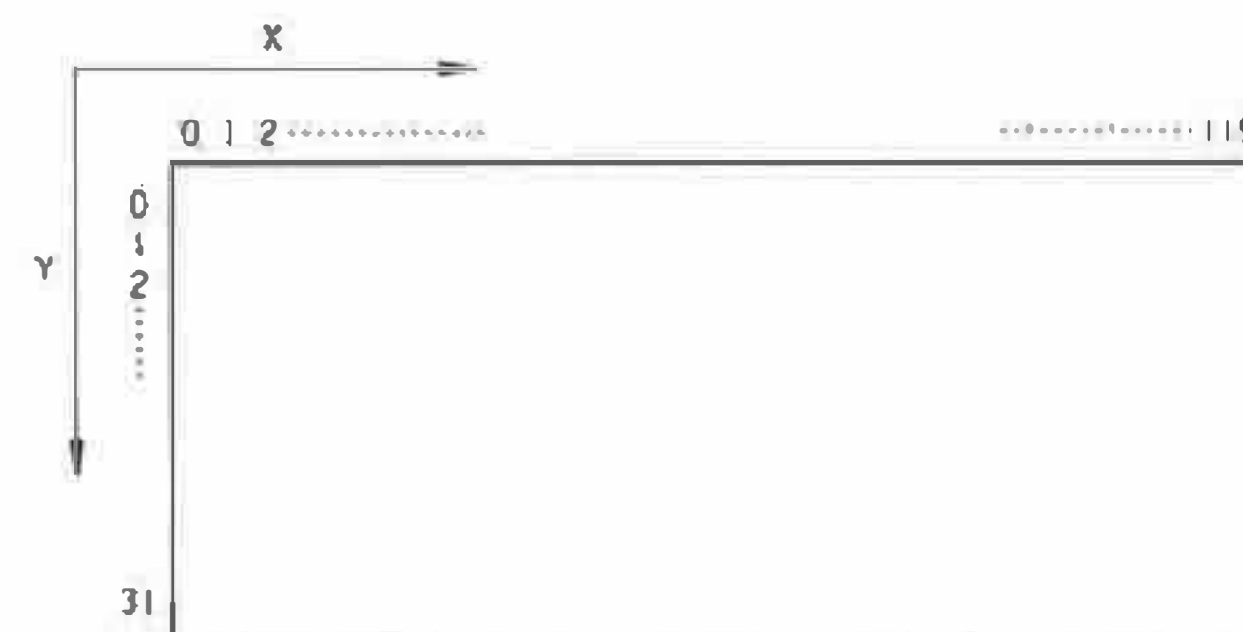
INSTR (N\$, I\$)

Fonction produisant la position I\$ en N\$

4.3 Affichage graphique

Il est possible d'utiliser le X07-BASIC pour tracer des cercles, traits et points sur l'écran.

L'écran réalise un affichage graphique de 120×32 points (largeur × longueur), et ses coordonnées sont placées comme suit.



A. Axes de coordonnées et points

Le programme ci-dessous trace les axes de coordonnées puis des points aux coordonnées introduites sur le clavier.

```
10 REM Coordonnees
20 CLS
30 LINE(0,16)-(120,16)
40 LINE(60,0)-(60,32)
50 INPUT "X,Y";X,Y
60 PSET(X+60,16-Y)
70 END
```

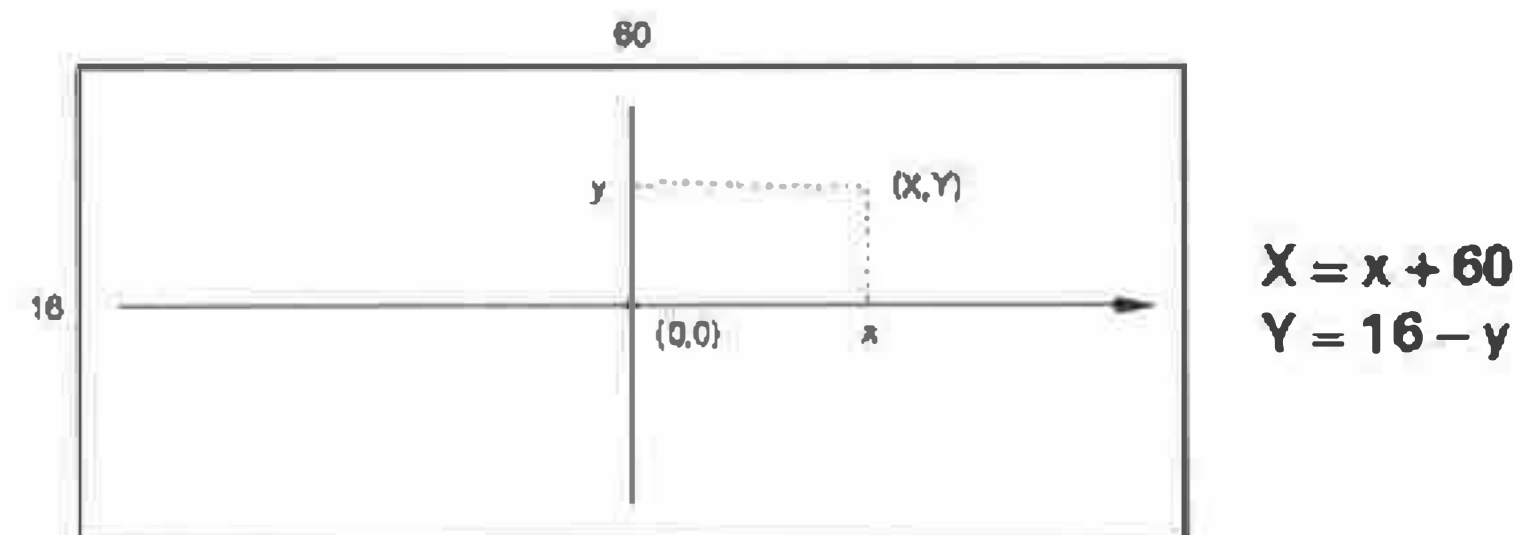
```
30 LINE (0, 16) - (120, 16)
40 LINE (60, 0) - (60, 32)
```

L'instruction **LINE** trace un point d'un jeu de coordonnées à un autre. La ligne 30 trace l'axe X des coordonnées (0,16) aux coordonnées (120,16). La ligne 40 trace l'axe Y des coordonnées (60,0) aux coordonnées (60,32)

A la ligne 50, on introduit les coordonnées du point d'origine des axes ci-dessus sur le clavier.

```
60 PSET (X+ 60, 16 - Y)
```

L'instruction **PSET** trace un point aux coordonnées spécifiées. Les coordonnées introduites à la ligne 50 sont relatives aux axes de coordonnées tracés. Cependant, l'instruction **PSET** ne permet de spécifier que des coordonnées absolues sur l'écran, et c'est pourquoi les coordonnées introduites sont converties à leur position absolue.



Essayons ce programme

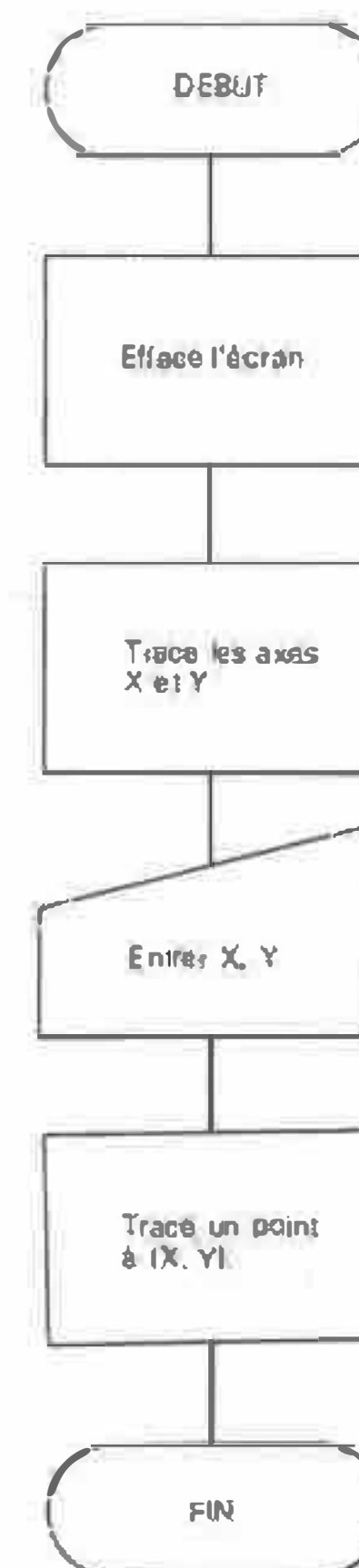
Nouvelles instructions

PSET (X,Y)

Trace un point en (X, Y)

LINE (X1, Y1) - (X2, Y2)

Trace un trait de (X1, Y1) à (X2, Y2)





B. Déplacement de points

Lorsque le programme suivant est exécuté, un point se déplace à travers tout l'écran.

```

10 REM DEPLACEMENT DE POINTS
20 CLS
30 I=1:J=1
40 X=0:Y=0
50 PSET(X,Y)
60 FOR P=1 TO 5:NEXT P
70 PRESET(X,Y)
80 X=X+I:Y=Y+J
90 IF X=119 OR X=0 THEN I=-I
100 IF Y=31 OR Y=0 THEN J=-J
110 GOTO 50

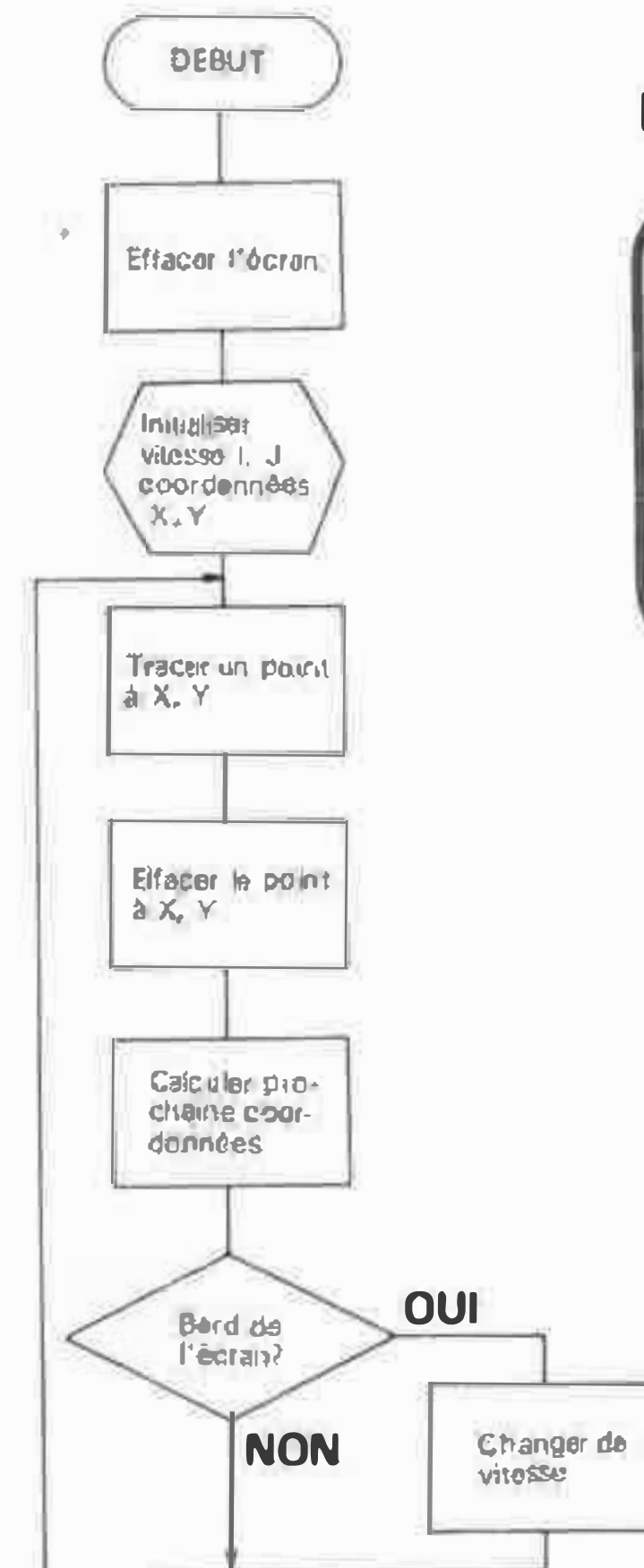
```

A partir du coin supérieur gauche, un point se déplace à travers l'écran, en rebondissant comme une boule de billard contre ses bords. Le point est tracé au moyen d'instructions PSET aux lignes 50 et 60. Il aurait été possible de n'utiliser que l'instruction PSET, mais alors le point se déplacerait trop rapidement pour qu'il soit visible. Par conséquent, on a utilisé ici l'instruction PRESET.

70 PRESET (X, Y)

L'instruction PRESET efface le point aux coordonnées spécifiées. En d'autres termes, cette instruction efface le point tracé à la ligne 60. La ligne 80 avance les coordonnées du point.

Les lignes 90 et 100 vérifient si le point a atteint le bord de l'écran. Si c'est le cas, sa direction est changée. Essayons maintenant ce programme. Vous verrez une balle qui rebondit contre chaque bord. Pour terminer l'exécution, frappez la touche ON/BREAK.



Nouvelle instruction

PRESET (X, Y)
Efface le point en (X, Y)



C. Tracé de cercles

Le programme ci-après trace un cercle au hasard.

```
10 REM CERCLE
20 CLS :R=RND(0)
30 X=RND(1)*120
40 Y=RND(1)*32
50 R=RND(1)*10
60 CIRCLE(X,Y),R
70 FOR I=1 TO 50:NEXT I
80 GOTO 30
```

Les lignes 30 à 50 recherchent la coordonnée X, la coordonnée Y et le rayon R du cercle. RND produit des nombres aléatoires compris entre 0 et 1. De ce fait, les gammes de X, Y et R sont les suivantes:

$$0 \leq X < 120$$

$$0 \leq Y < 32$$

$$0 \leq R < 10$$

60 CIRCLE (X,Y),R

L'instruction CIRCLE trace un cercle dont le centre est aux coordonnées (X, Y) et le rayon est R.

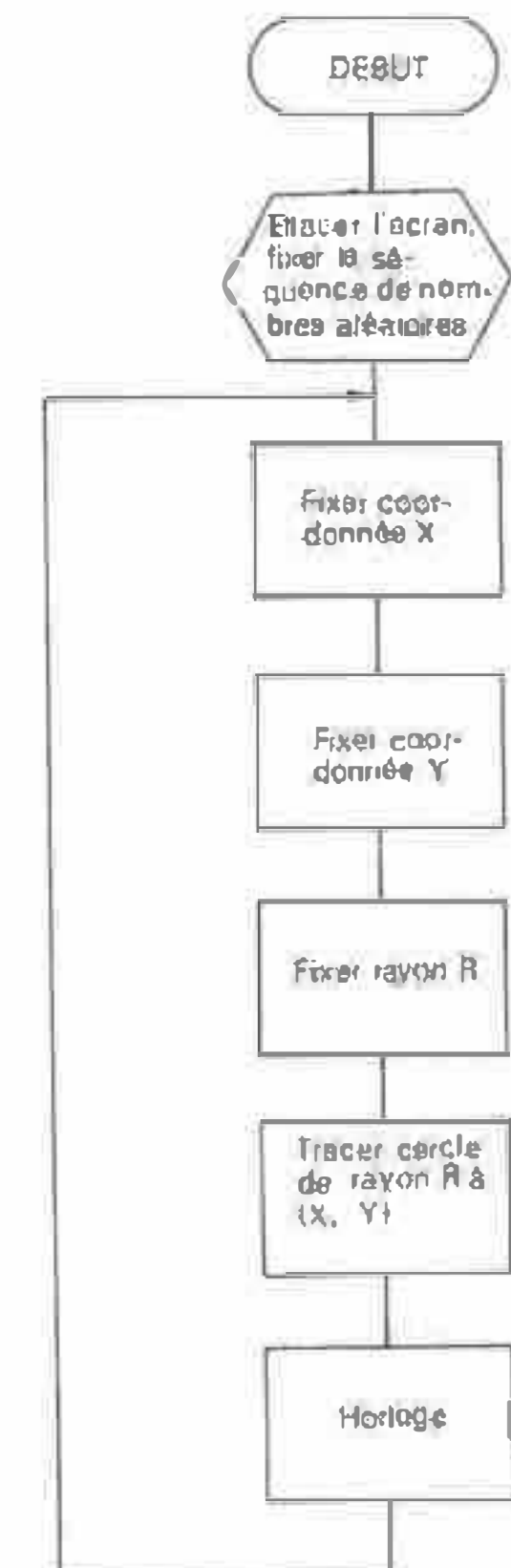
La ligne 70 est utilisée pour ralentir la vitesse d'exécution, qui est fixée à une durée raisonnable.

Lorsque ce programme est exécuté, des cercles de diverse grandeur sont tracés sur l'écran. Pour terminer l'exécution, pressez la touche ON/BREAK.

Nouvelle instruction

CIRCLE (X,Y),R

Trace un cercle de rayon R centré sur les coordonnées (X,Y).



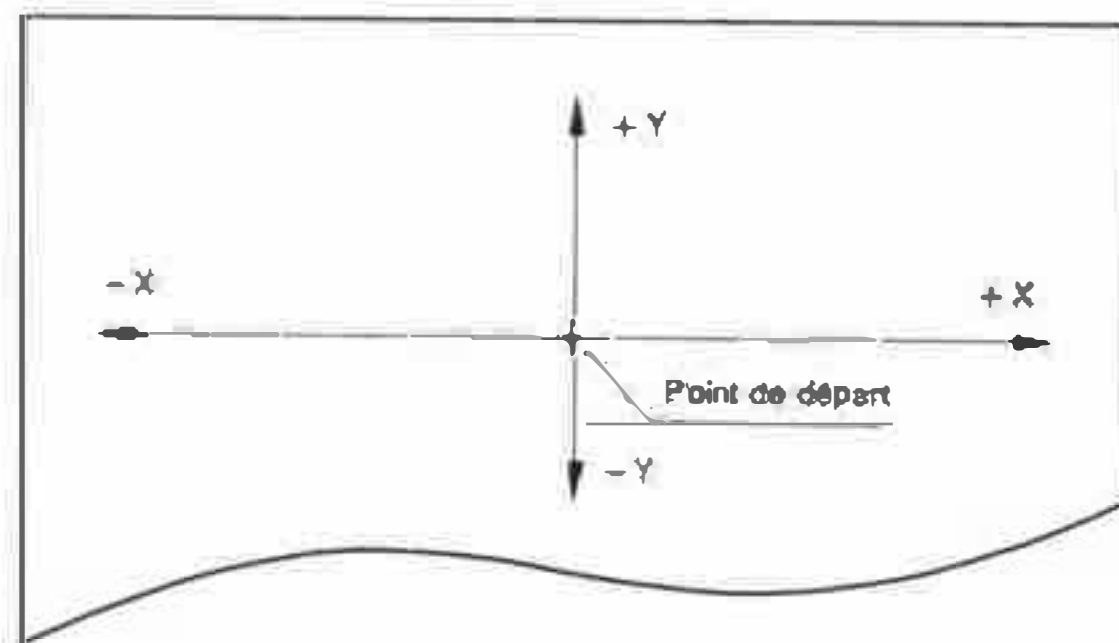
4.4 Imprimante

L'imprimante graphique couleur/X-710 peut être reliée au X-07.

Cette imprimante peut être utilisée aussi bien en mode texte qu'en mode graphique. Le texte mode est utilisé pour imprimer des lettres et des chiffres de la manière habituelle.

En d'autres termes, pour imprimer des programmes, données, etc., on utilise le mode texte. Il est possible d'imprimer jusqu'à 40 caractères/ligne en format standard.

En mode graphique, il est possible de librement tracer des traits et caractères de différentes couleurs, dimensions et sous différents angles. Lorsque l'imprimante est utilisée en mode graphique, les coordonnées suivantes sont fixées.



Sens d'avance du
papier

$-999 \leq X, Y \leq 999$

La position du point de départ peut être définie par une instruction. En général, le coin supérieur gauche est la position de départ du stylo.

A. Impression de caractères

Le programme suivant imprime le jeu complet de caractères fixé par les codes de caractères.

```
10 REM IMPRIMANTE
20 C=&H20
30 C$=CHR$(C)
40 LPRINT C$;
50 C=C+1
60 IF C<&HDF THEN 30
70 LPRINT
80 END
```

Les codes de caractères 20 (hexadécimal) à DF (hexadécimal) sont convertis en caractères correspondants et imprimés par l'imprimante. Dans ce cas, l'imprimante doit se trouver en mode de texte (c'est son mode habituel).

40 LPRINT C\$;

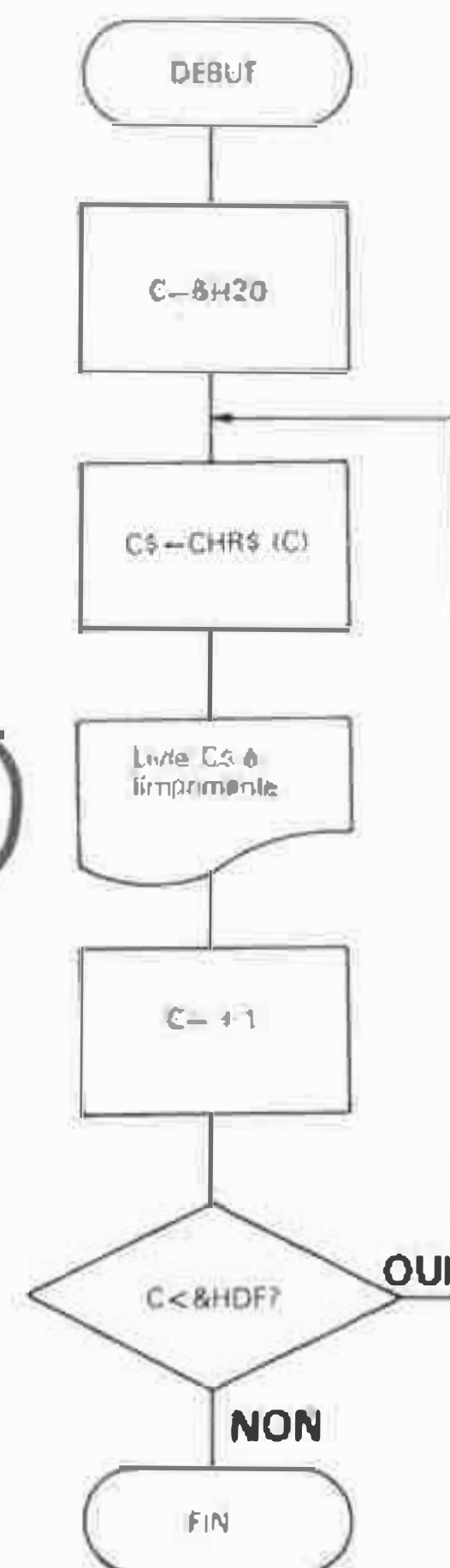
L'instruction LPRINT est utilisée pour imprimer des données sur l'imprimante. Cette instruction est similaire à PRINT. Ici, les caractères sont imprimés horizontalement un par un. Lorsque ce programme est exécuté, on obtient l'affichage suivant:

```
0"#$%&'()*+,-./0123456789:;<=>?@ABCDEF
HIJKLMNOPQRSTUVWXYZ[\]^_`ab cde fgh ijk lmn
opqrstu vwxyz{|}~ ¨ . ' , .
```



Nouvelle instruction

LPRINT Imprime les données à l'aide de l'imprimante.



B. Impression graphique de caractères

Le programme suivant imprime "GRAPHIC" en différentes dimensions, couleurs et sous différents angles.

```

10 LPRINT CHR$(18)
20 LPRINT "P"
30 FOR I=0 TO 3
40 LPRINT "S";I+1
50 LPRINT "C";I
60 LPRINT "Q";I
70 LPRINT "PGRAPHIC"
80 NEXT I
90 LPRINT "S1";CHR$(17)
100 END
  
```

Annotations du programme :

- 10 LPRINT CHR\$(18) : Place l'imprimante en mode graphique
- 20 LPRINT "P" : imprime plusieurs espaces
- 40 LPRINT "S";I+1 : Fixe la dimension
- 50 LPRINT "C";I : Fixe le couleur
- 60 LPRINT "Q";I : Fixe l'angle
- 70 LPRINT "PGRAPHIC" : Imprime GRAPHIC
- 90 LPRINT "S1";CHR\$(17) : Annule la dimension et replace l'imprimante en mode texte.

10 LPRINT CHR\$(18)

CHR\$(18) correspond à DC2 (code de commande d'unité).

Lorsque DC2 est envoyé à l'imprimante, elle est placée en mode graphique.

20 LPRINT "P"

Lorsque la ligne 20 est exécutée en mode texte, "P" est imprimé au début, puis plusieurs espaces sont créés.

Cependant, en mode graphique P, S, C, Q, etc. après le retour du chariot ont une signification particulière. P ordonne que les caractères qui le suivent doivent être imprimés. De ce fait, plusieurs espaces sont imprimés sans "P", et la plume est laissée à la dernière position.

40 LPRINT "S"; I+1

S fixe le format des caractères. I est variable de 0 à 15; de ce fait, le format de caractère est variable de S0 à S15.

50 LPRINT "C"; I

C fixe la couleur des caractères. Il est variable de C0 (noir), C1 (bleu), C2 (vert) à C3 (rouge).

60 PRINT "Q"; I

Q fixe l'angle des caractères. Ainsi, les caractères sont imprimés en séries à un angle de 90°.

70 LPRINT "PGRAPHIC"

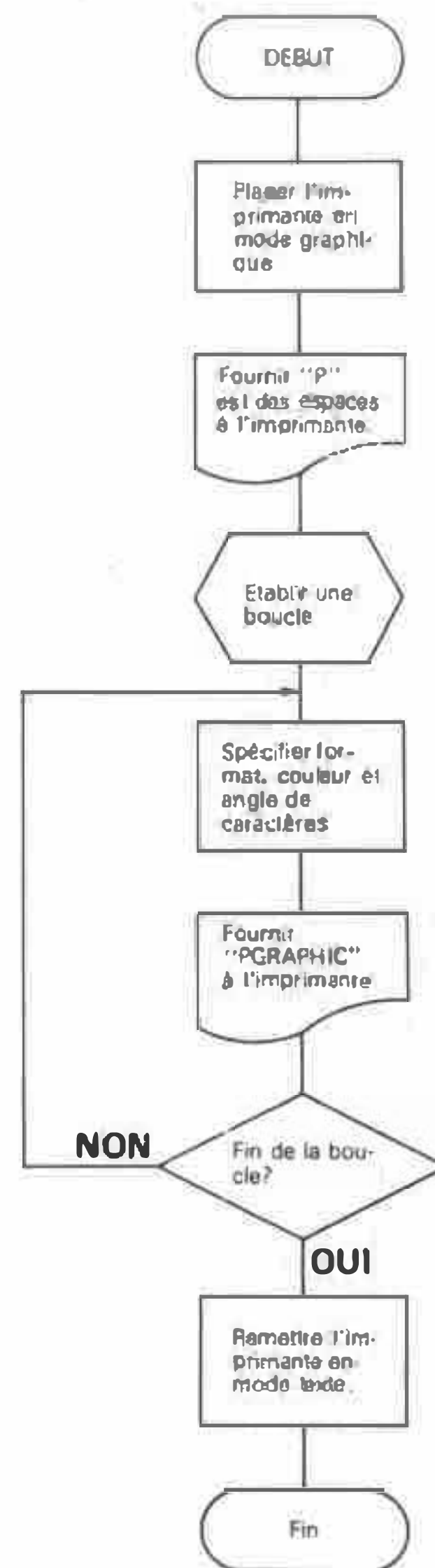
"GRAPHIC" est imprimé ici.

90 LPRINT "S1";CHR\$(17)

CHR\$(17) correspond à DCI. Lorsque celui-ci est envoyé à l'imprimante, elle est remplacée en mode texte. Cependant, l'état de S (format de caractère) est maintenu. Ainsi, S est fixé à S1 (40 caractères/ligne) avant que l'imprimante ne soit remise en mode texte.

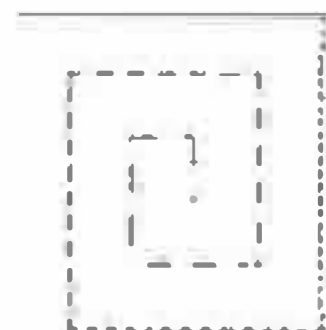
Lorsque ce programme est exécuté, on obtient l'affichage suivant.

GRAPHIC
GRAPHIC
GRAPHIC
GRAPHIC



C. Tracé d'un motif

Le programme suivant trace le motif ci-après:



```
10 REM PGRAPHICS
20 LPRINT CHR$(18)
30 LPRINT "L0"
40 LPRINT "M200,0"
50 LPRINT "I"
60 FOR I=1 TO 9
70 READ X,Y
80 GOSUB 200
90 NEXT I
100 LPRINT "A"
110 END
120 DATA 100,0,100,-100,20,-100,20,-20,8
0,-20
130 DATA 80,-80,40,-80,40,-40,60,-40,60,
-60
200 REM PSUB
210 LPRINT "L";L
220 LPRINT "D"+STR$(X)+", "+STR$(Y)
230 L=L+1
240 RETURN
```

Les lignes 200 à 240 constituent un sous-programme qui trace un trait entre la position actuelle de la plume et les coordonnées (X, Y).

Ce programme comprend plusieurs instructions pour tracer un trait. Examinons-les une à une.

Premièrement, à la ligne 20, l'imprimante est placée en mode graphique.

30 LPRINT "L0"

Lorsqu'on trace un trait droit, L permet de spécifier s'il s'agit un trait plein ou pointillé.

L0 spécifie un trait plein, et L1 à L15 un trait pointillé.

Plus le chiffre après L est grand, plus les espaces entre les points sont grands.

40 LPRINT "M200, 0"

M soulève le stylo et le déplace aux coordonnées spécifiées.

Ici le stylo est emmené aux coordonnées (200, 0).

50 LPRINT "I"

I fixe la position actuelle comme point de départ du stylo.

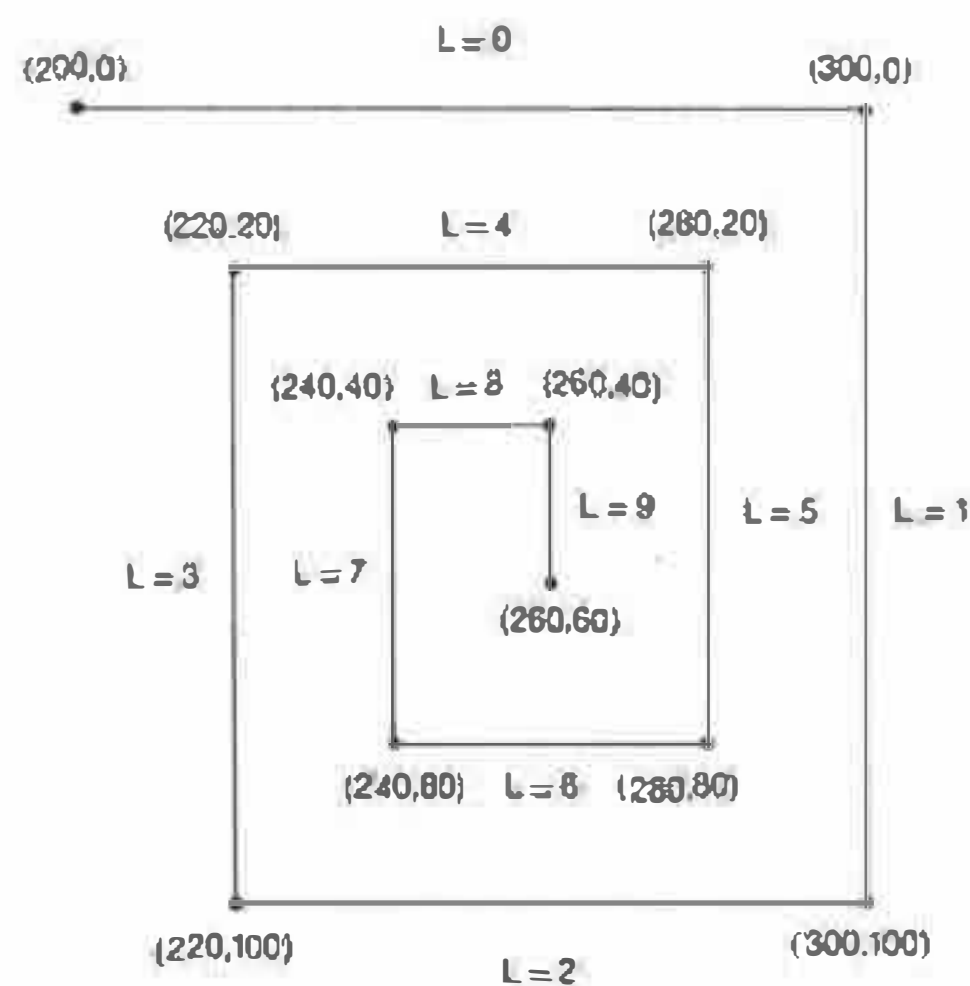
220 LPRINT "D";X;",";Y

D trace un trait entre la position actuelle du stylo et les coordonnées (X, Y).

100 LPRINT "A"

A soulève le stylo, le ramène au coin gauche et remet l'imprimante en mode texte.

Lorsque ce programme est exécuté, les coordonnées sont changées comme indiqué ci-dessous et des traits sont tracés.



*La position d'origine du stylo est fixé comme point de départ. Le trait devient de plus en plus rude au fur et à mesure de son avance.

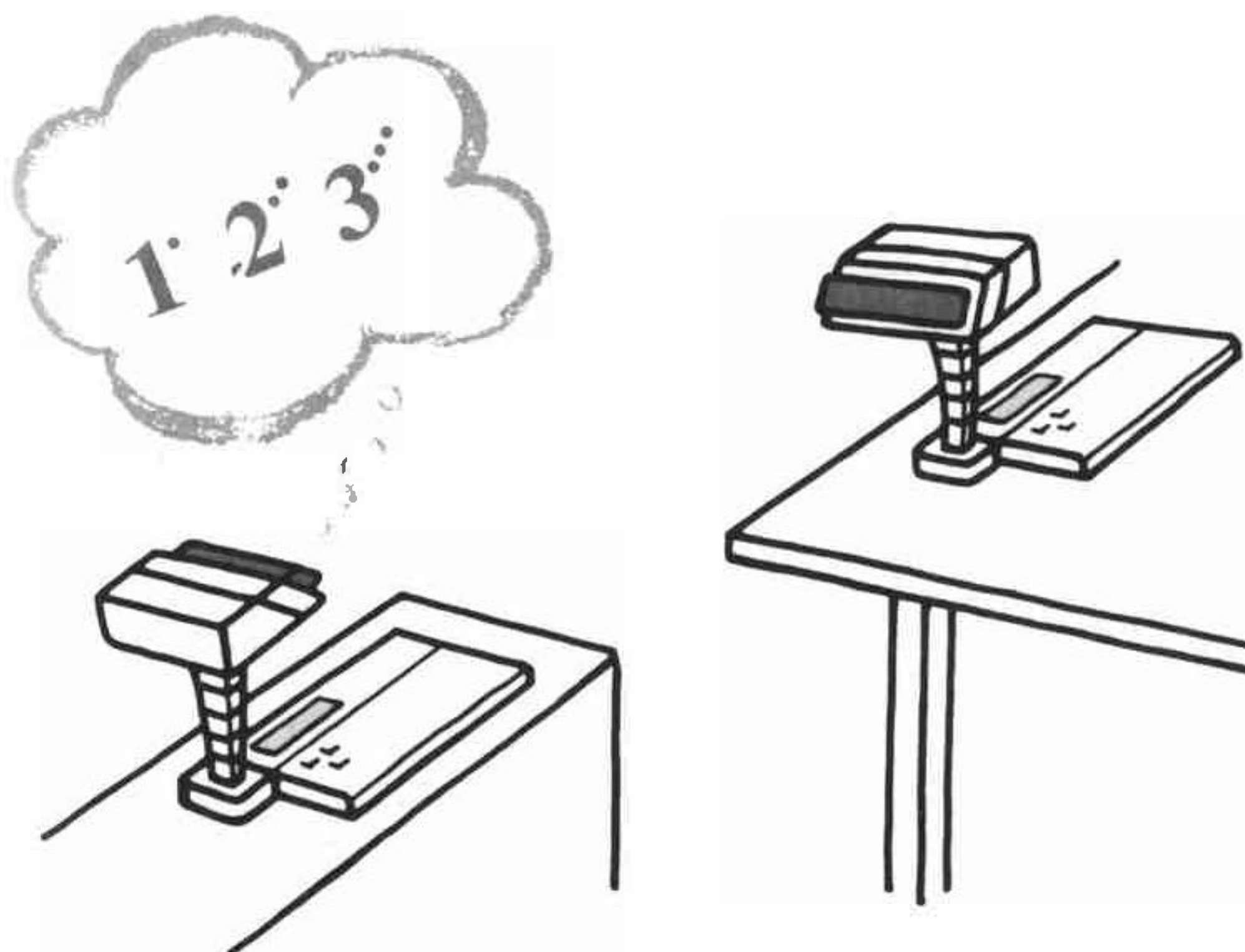


4.5 Coupleur optique

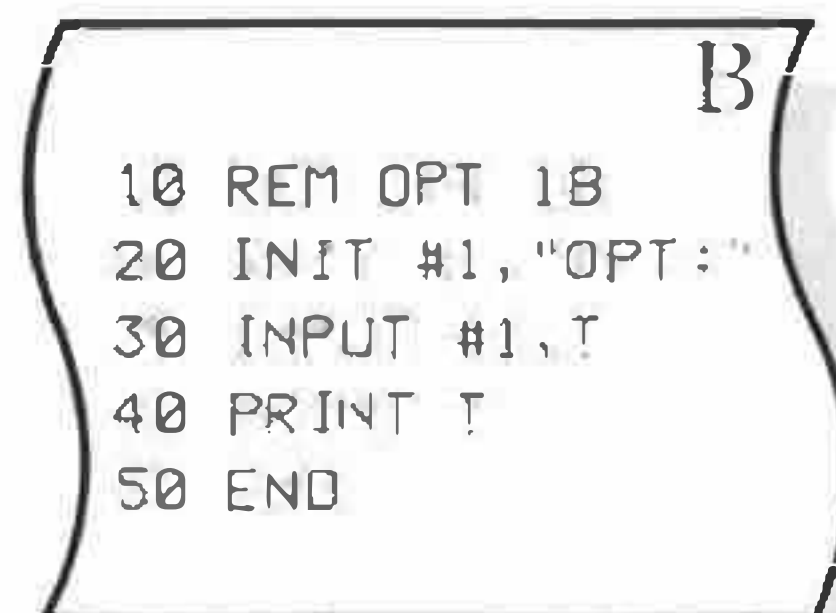
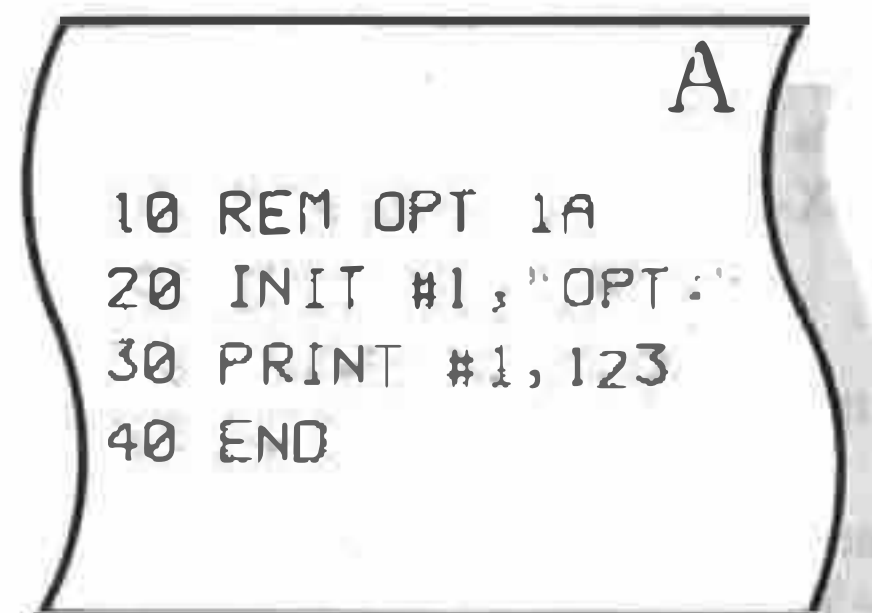
Un coupleur optique X-721 peut être relié à un X-07, et un autre X-721 peut être relié à un second X-07 pour établir une liaison sans fil. Nous introduisons dans cette section les programmes pour la transmission/réception de données entre deux X-07 en utilisant deux coupleurs optiques.

A. Transmission de valeurs numériques

Rédigeons un programme pour la transmission de valeurs numériques d'un X-07 (A) à un autre (B) à l'aide de coupleurs optiques.



Pour transmettre la valeur numérique 123 de B à A, le programme ci-dessous doit être introduit dans chaque X-07.



Examinons le programme A

20 INIT #1,"OPT:"

Cette ligne met en service le coupleur optique.

30 PRINT #1, 123

La ligne 30 transmet la valeur numérique 123 à travers le coupleur optique.

Dans le programme B, l'autre coupleur optique est également mis en service à la ligne 20.

30 INPUT #1, T

A la ligne 30, la donnée reçue à travers le coupleur optique est stockée dans la variable T. A la ligne 40, le contenu de T est affiché.

Exécutons maintenant ce programme.

- Premièrement, lorsque le programme B est exécuté, il attend à l'instruction de la ligne 30 la réception de la donnée.
- Lorsque le programme A est exécuté, la donnée 123 est transmise à travers le coupleur optique.
- B reçoit cette donnée, sort de son mode d'attente et exécute la ligne 40 pour afficher 123 sur son écran.

Veuillez remarquer que si le programme A est exécuté avant le programme B, B n'attendra pas la réception, et le 123 envoyé par A ne sera pas reçu.

B. Transmission de caractères

Rédigeons à présent un programme qui affiche sur B les caractères frappés en A.

A

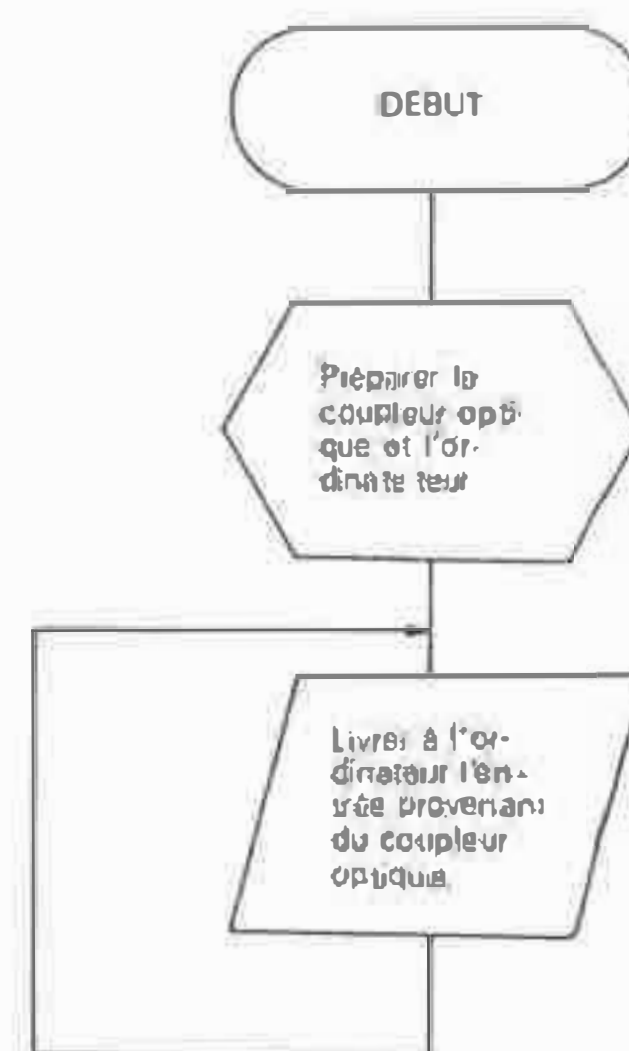
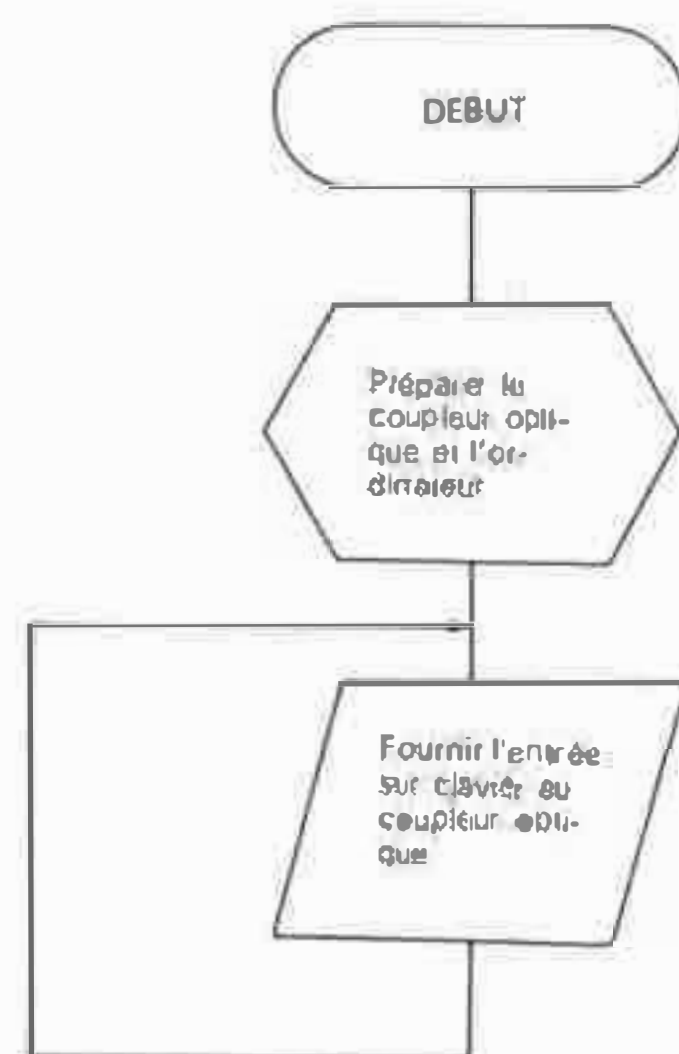
```

10 REM OPT 2A
20 INIT #1, "OPT:"
30 INIT #2, "CON:"
40 OUT #1, INP(#2)
50 GOTO 40
    
```

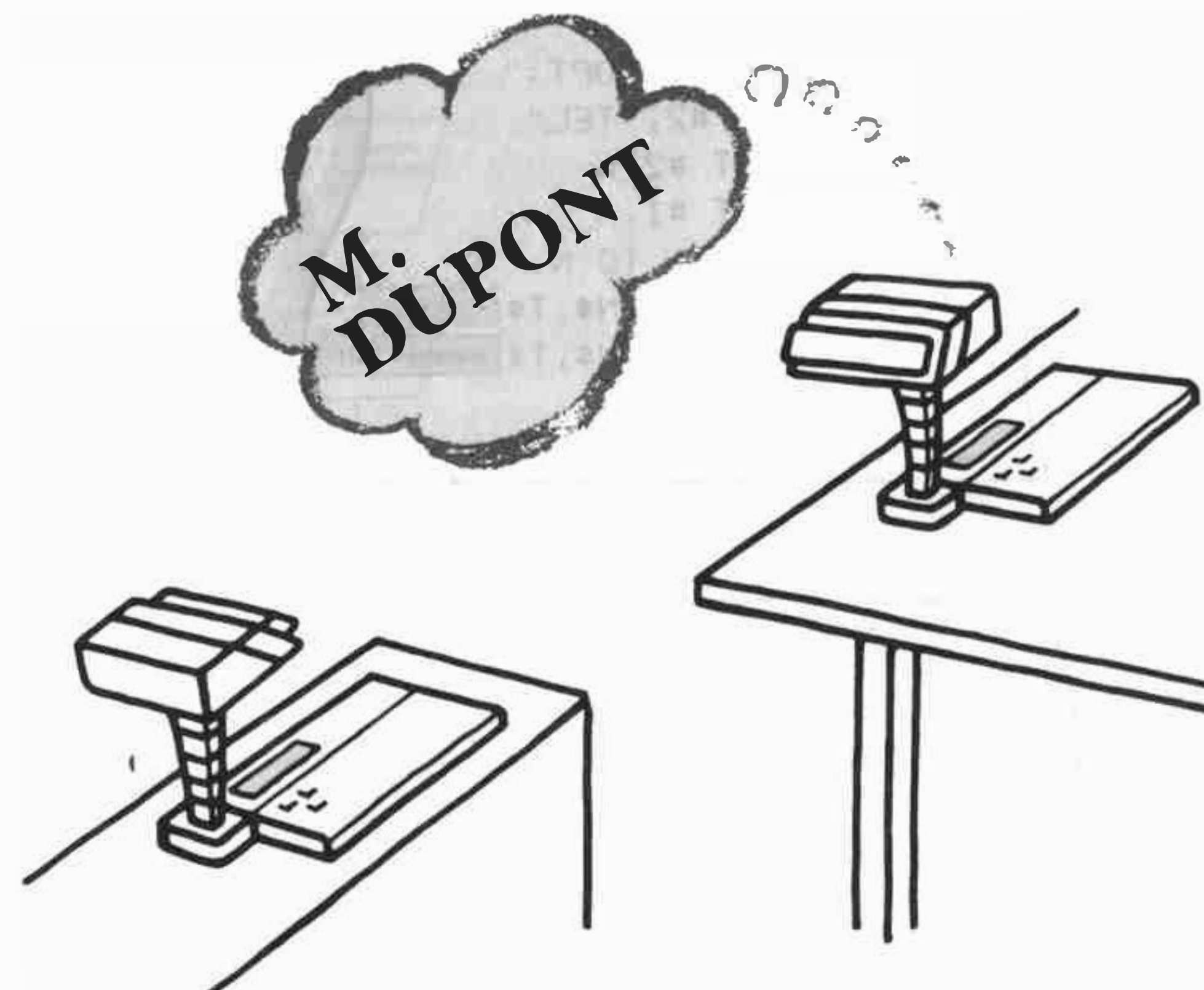
B

```

10 REM OPT 2B
20 INIT #1, "OPT:"
30 INIT #2, "CON:"
40 OUT #2, INP(#1)
50 GOTO 40
    
```



Comme auparavant, il faut exécuter le programme B avant le programme A. Lorsque le programme A est exécuté, les caractères frappés sur A ne sont pas affichés sur son propre écran, mais sur celui de B.



Nouvelles instructions

OUT # Délivre vers # <numéro de fichier>.
INP # Lit la valeur de # <numéro de fichier>.

C. Transmission d'un fichier

Les programmes suivants copient le fichier TEL se trouvant dans la mémoire RAM de A dans celle de B.

A

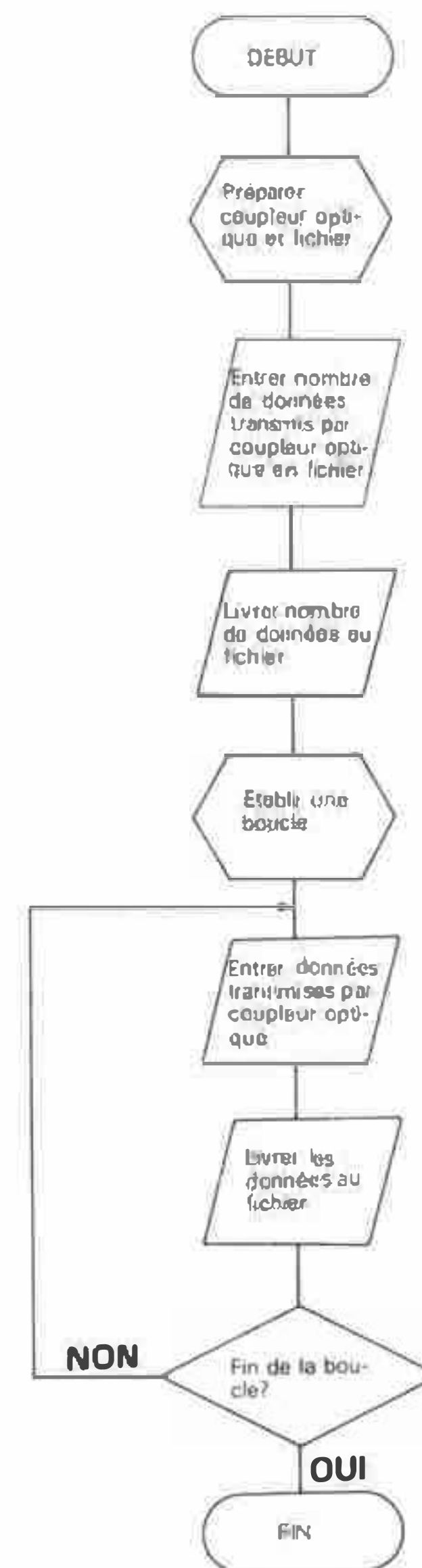
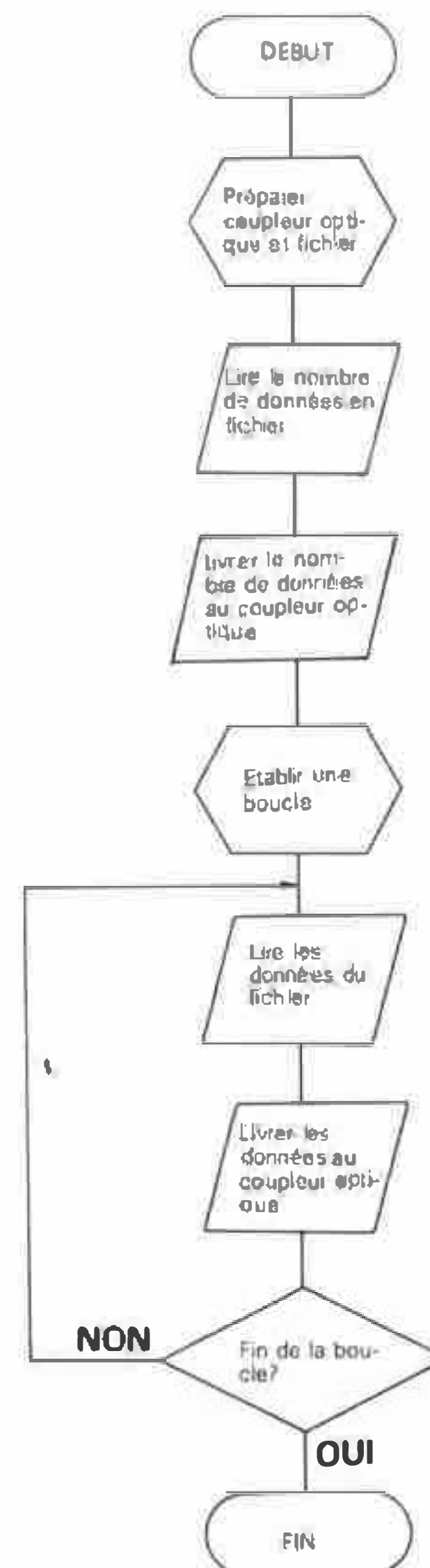
```

10 REM OPT 3A
20 INIT #1,"OPT:"
30 INIT #2,"TEL"
40 INPUT #2,N
50 PRINT #1,N
60 FOR I=1 TO N
70 INPUT #2,N$,T$
80 PRINT #1,N$,T$
90 NEXT I
100 END
    
```

B

```

10 REM OPT 3B
20 INIT #1,"OPT:"
30 INIT #2,"TEL",100,"D"
40 INPUT #1,N
50 PRINT #2,N
60 FOR I=1 TO N
70 INPUT #1,N$,T$
80 PRINT N$,T$
90 PRINT #2,N$,T$
100 NEXT I
110 END
    
```



TEL est le fichier d'annuaire téléphonique que nous avons créé en section 3.5. Ce fichier TEL est lu en A et est transmis à travers le coupleur optique OPT. En B, le fichier TEL est lu à travers son OPT et écrit sur le fichier TEL.

Lorsque ces programmes sont exécutés, il se crée en mémoire RAM un fichier TEL exactement pareil à celui en A.

Fichier

D'aucun pourraient éprouver certaines difficultés de lecture de manuels d'informatique en raison du vocabulaire technique employé. Parmi celui-ci, le terme "fichier" peut être expliqué comme un "assemblage d'informations, etc."

Cependant, cette explication pourrait ne pas vous satisfaire entièrement.

En général, le terme de "fichier" évoque une liasse de feuilles rangées dans un classeur. Cette image convient parfaitement, mais l'ordinateur ne possède pas de doigts habiles pour feuilleter les pages pour les lire. De ce fait, ces liasses de papier sont mises sous une forme différente. Tout assemblage de données peut être appelé un fichier.

Par exemple dans le X-07, les programmes et données sont écrits dans une mémoire interne appelée mémoire RAM pour fichiers. De plus, tous les programmes et données assemblés quelque part sont collectivement appelés "fichiers".

D. Dialogue

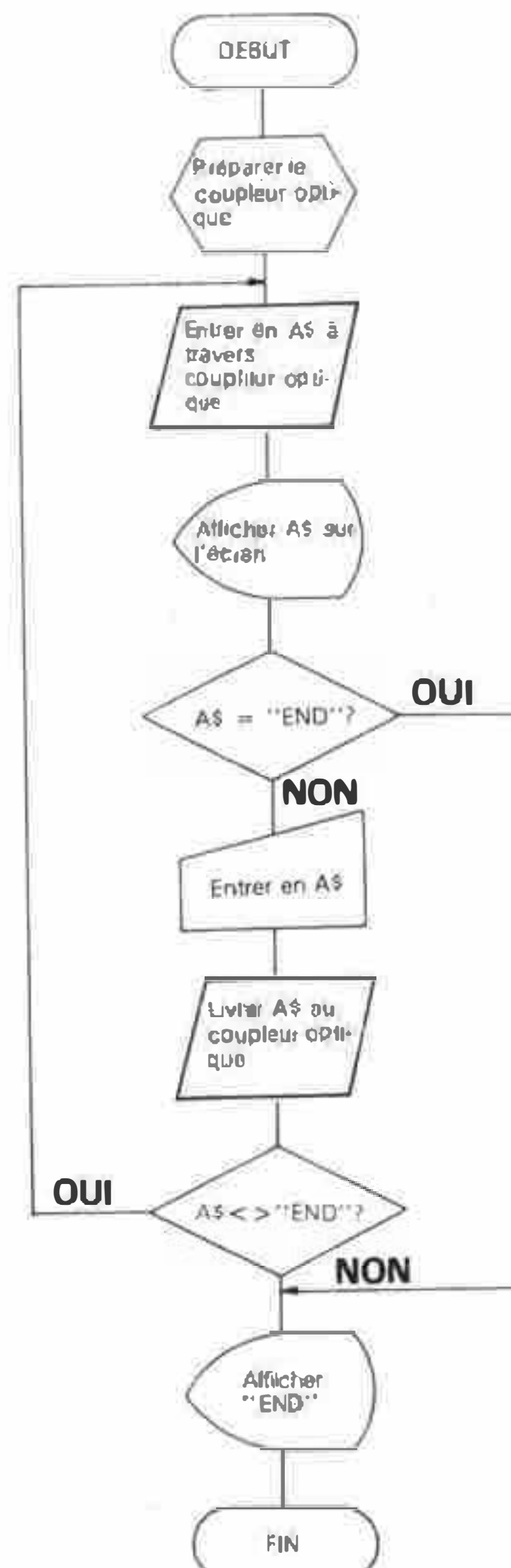
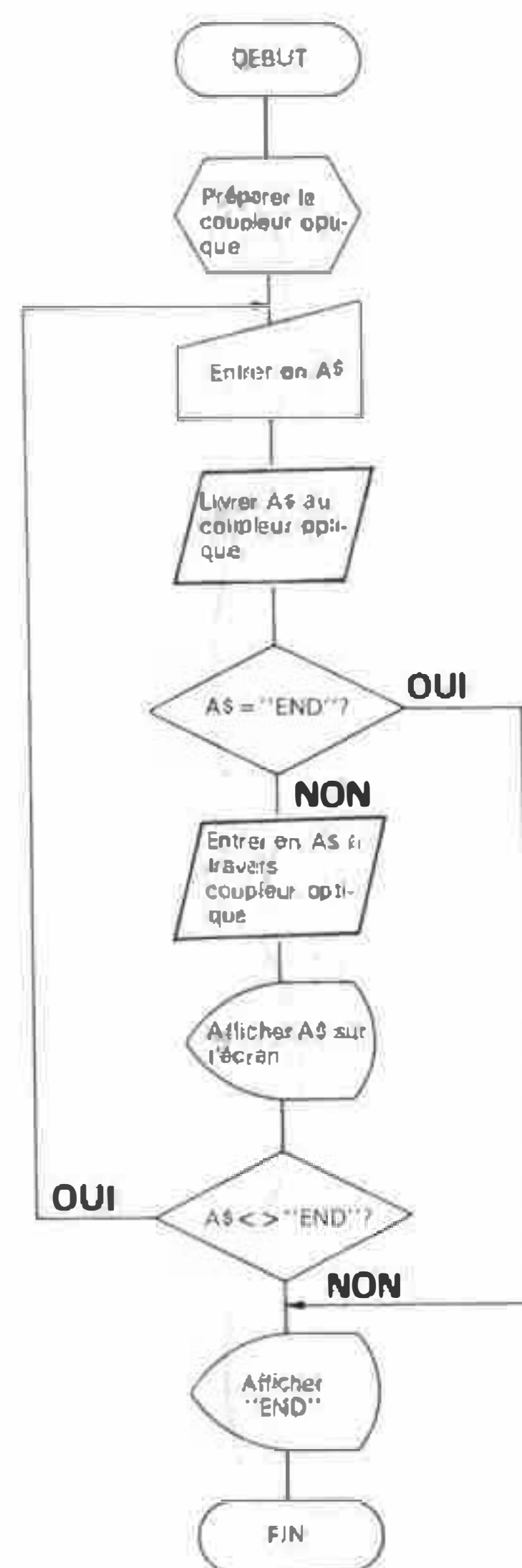
A présente, rédigeons un programme de dialogue entre A et B.

A

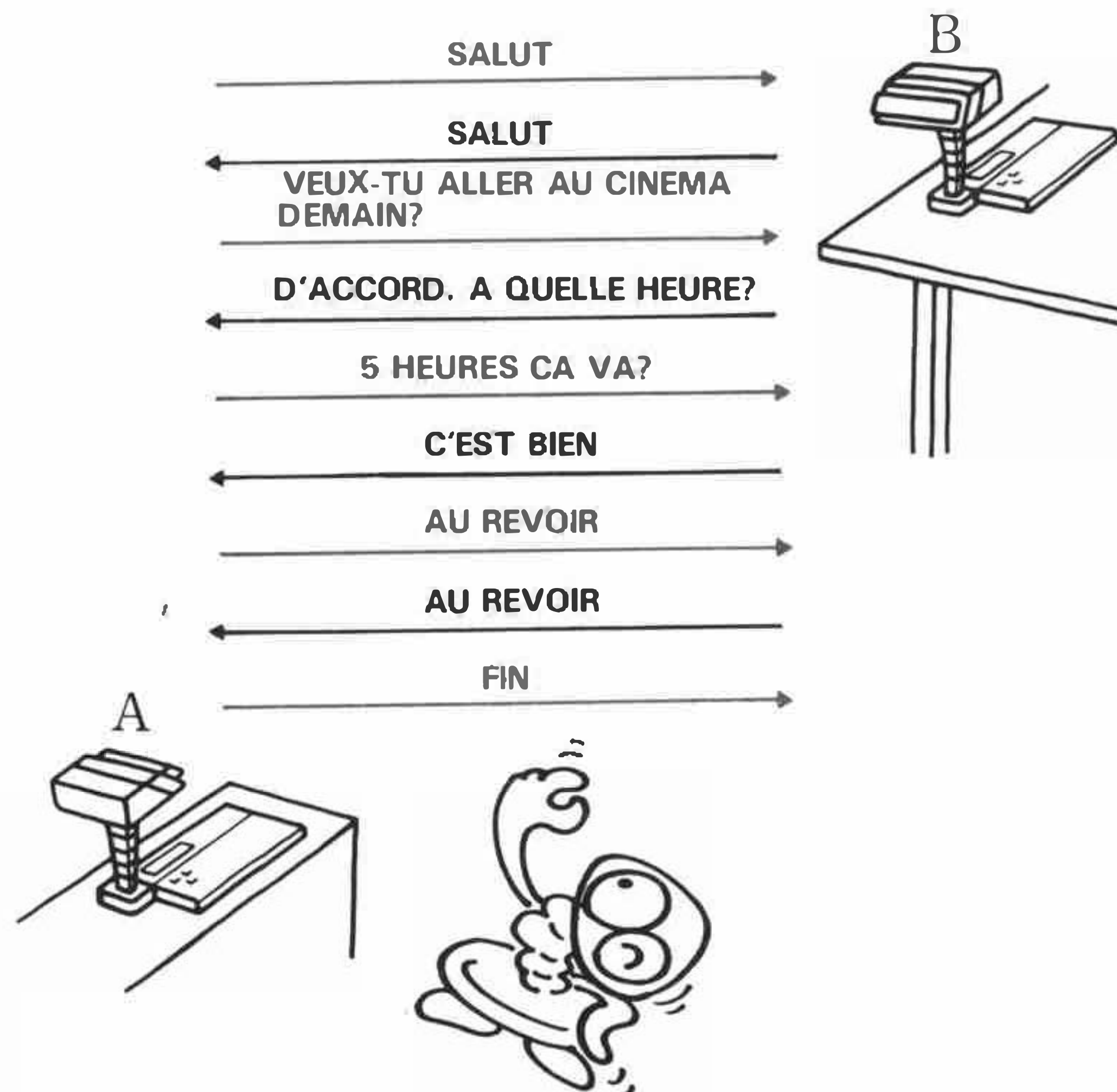
```
10 REM OPT 4A
20 INIT#1,"OPT:"
30 INPUT A$
40 PRINT#1,A$
50 IF A$="END" THEN 90
60 INPUT#1,A$
70 PRINT A$
80 IF A$<>"END" THEN 30
90 PRINT "END"
100 END
```

B

```
10 REM OPT 4B
20 INIT#1,"OPT:"
30 INPUT#1,A$
40 PRINT A$
50 IF A$="END" THEN 90
60 INPUT A$
70 PRINT#1,A$
80 IF A$<>"END" THEN 30
90 PRINT "END"
100 END
```



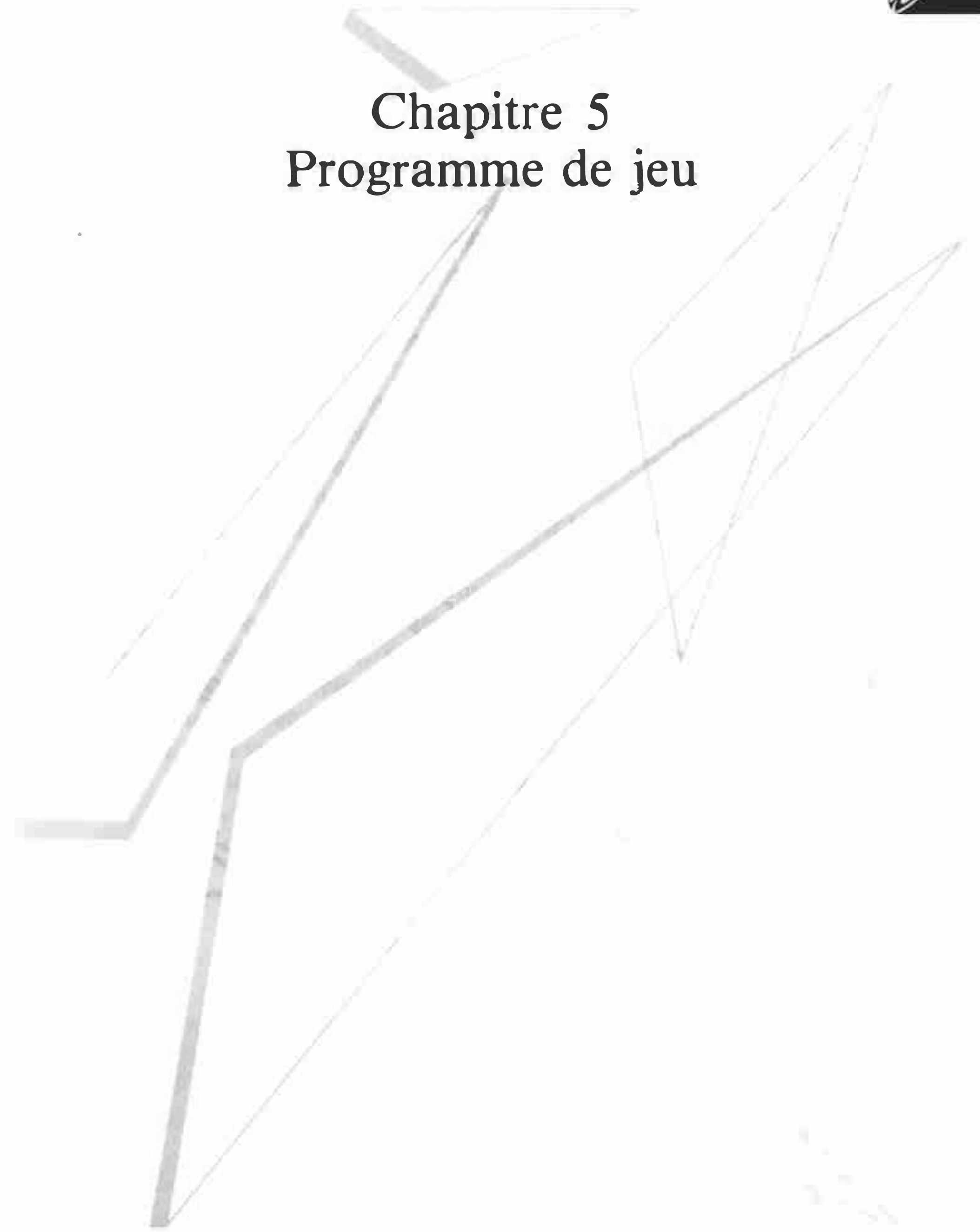
Premièrement, un message est enfoncée sur A. Lorsque la touche RETURN est frappée, le message est transmis et affiché en B. Ensuite, B examine ce message et transmet un message à A. Ce cette manière, un dialogue peut être établi entre A et B. Pour terminer le programme, transmettre "END" de A ou B.



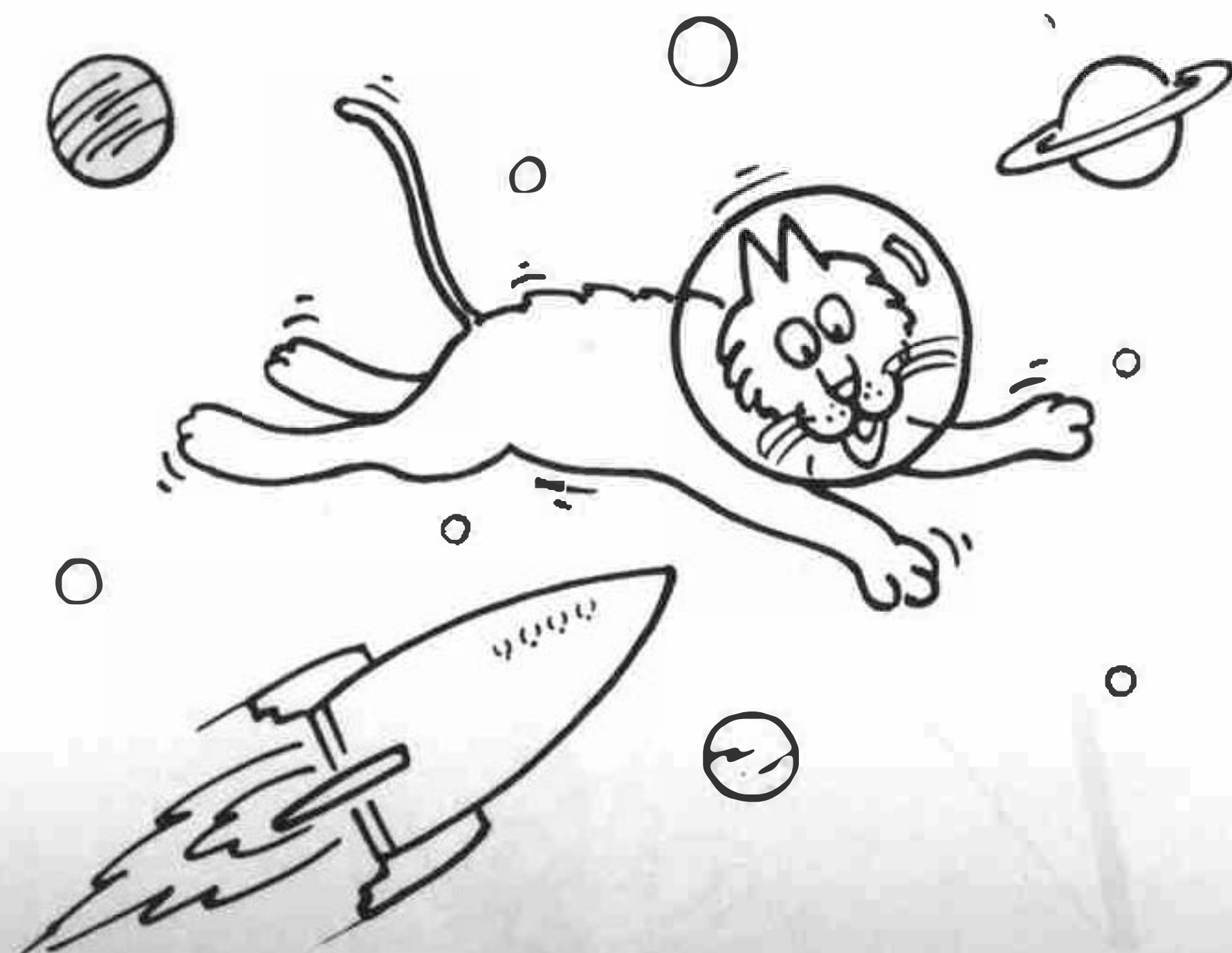


Chapitre 5

Programme de jeu



Le dernier programme est le jeu de marchand de l'espace.
Le résumé, l'explication et l'exécution de ce programme sont expliqués dans les sections respectives. La liste du programme est donnée à la fin de 5.2 Explication du programme .



5.1 Résumé du jeu

Vous quittez la terre en quête d'une fortune immense. Vous allez d'une planète à l'autre en négociant diverses ressources naturelles. A la fin, vous revenez sur terre. A ce point, vous gagnez si vous avez fait du bénéfice, et vous perdez si ce n'est pas le cas.

Les ressources naturelles négociées sur chaque planète sont les suivantes:

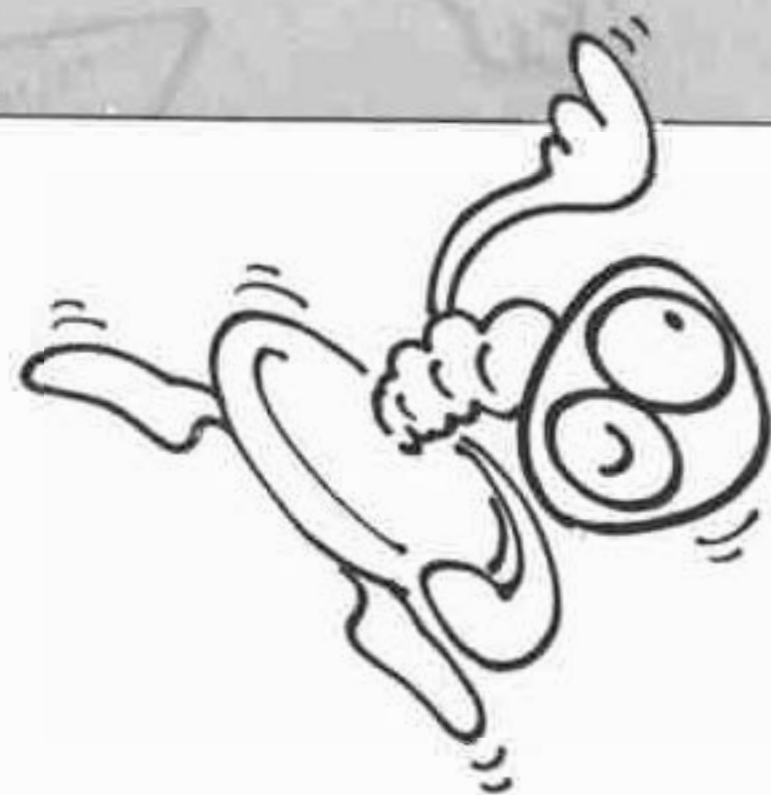
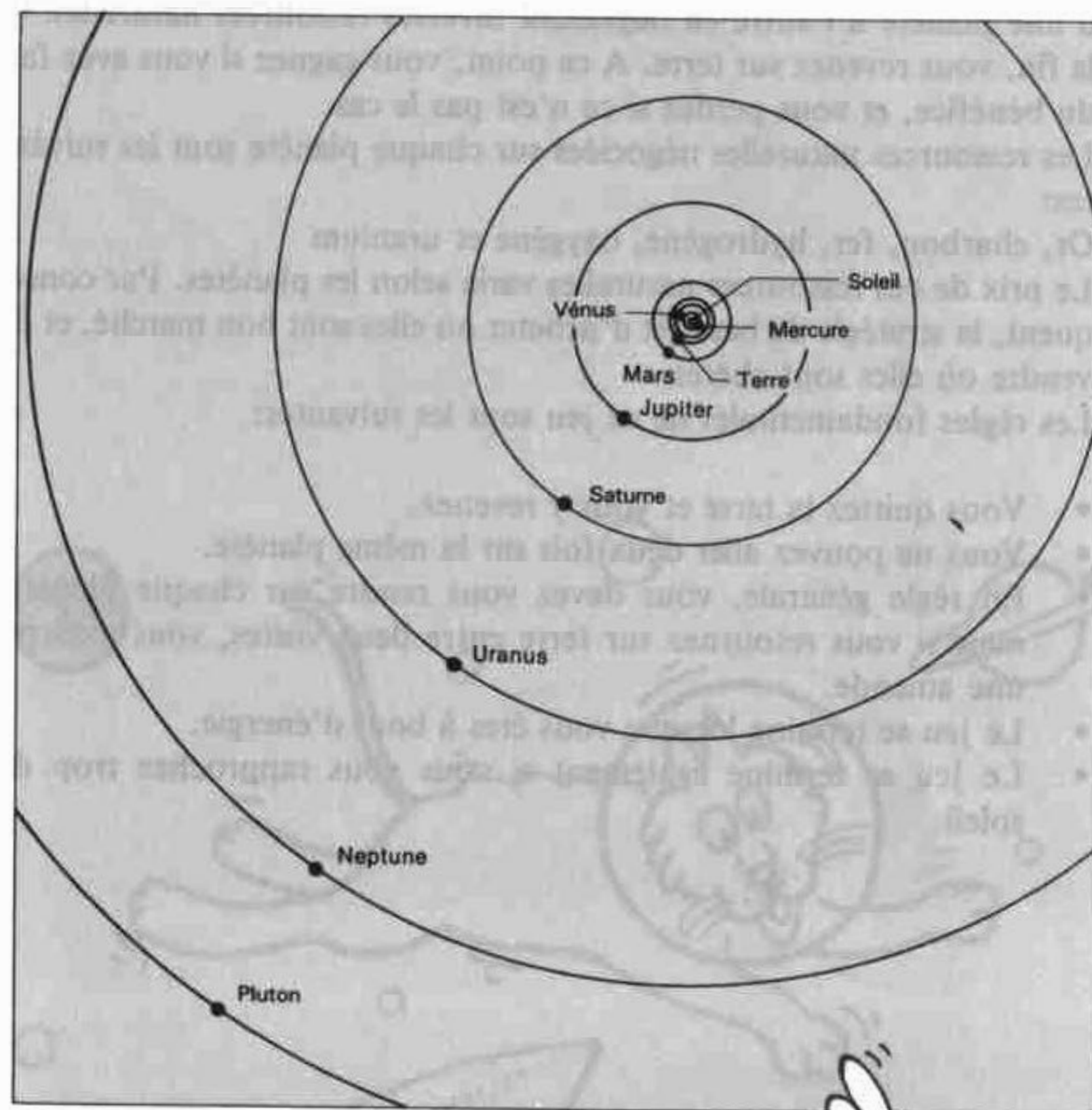
Or, charbon, fer, hydrogène, oxygène et uranium

Le prix de ces ressources naturelles varie selon les planètes. Par conséquent, la stratégie de base est d'acheter où elles sont bon marché, et de vendre où elles sont chères.

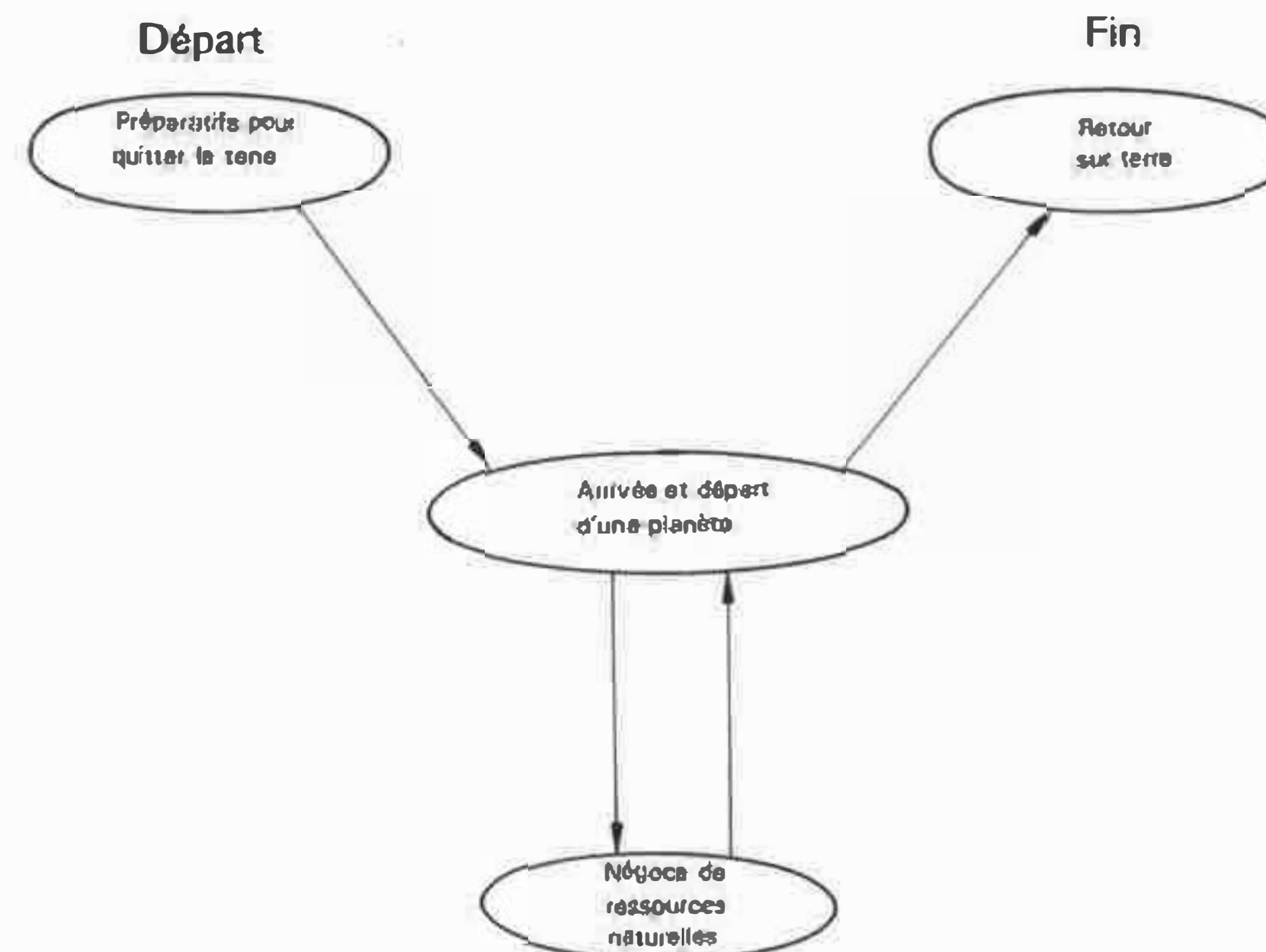
Les règles fondamentales de ce jeu sont les suivantes:

- Vous quittez la terre et vous y revenez.
- Vous ne pouvez aller deux fois sur la même planète.
- En règle générale, vous devez vous rendre sur chaque planète, mais si vous retournez sur terre entre deux visites, vous recevrez une amende.
- Le jeu se termine lorsque vous êtes à bout d'énergie.
- Le jeu se termine également si vous vous rapprochez trop du soleil.

Vous pouvez visiter les planètes suivantes dans l'ordre de rapprochement du soleil.



Ce jeu se déroule dans la séquence suivante.



■ Préparatifs pour quitter la terre

Quand vous quittez la terre, vous emportez de l'argent, de l'énergie et des ressources naturelles en quantités suivantes.

Argent: 4.700 Energie: 50.000

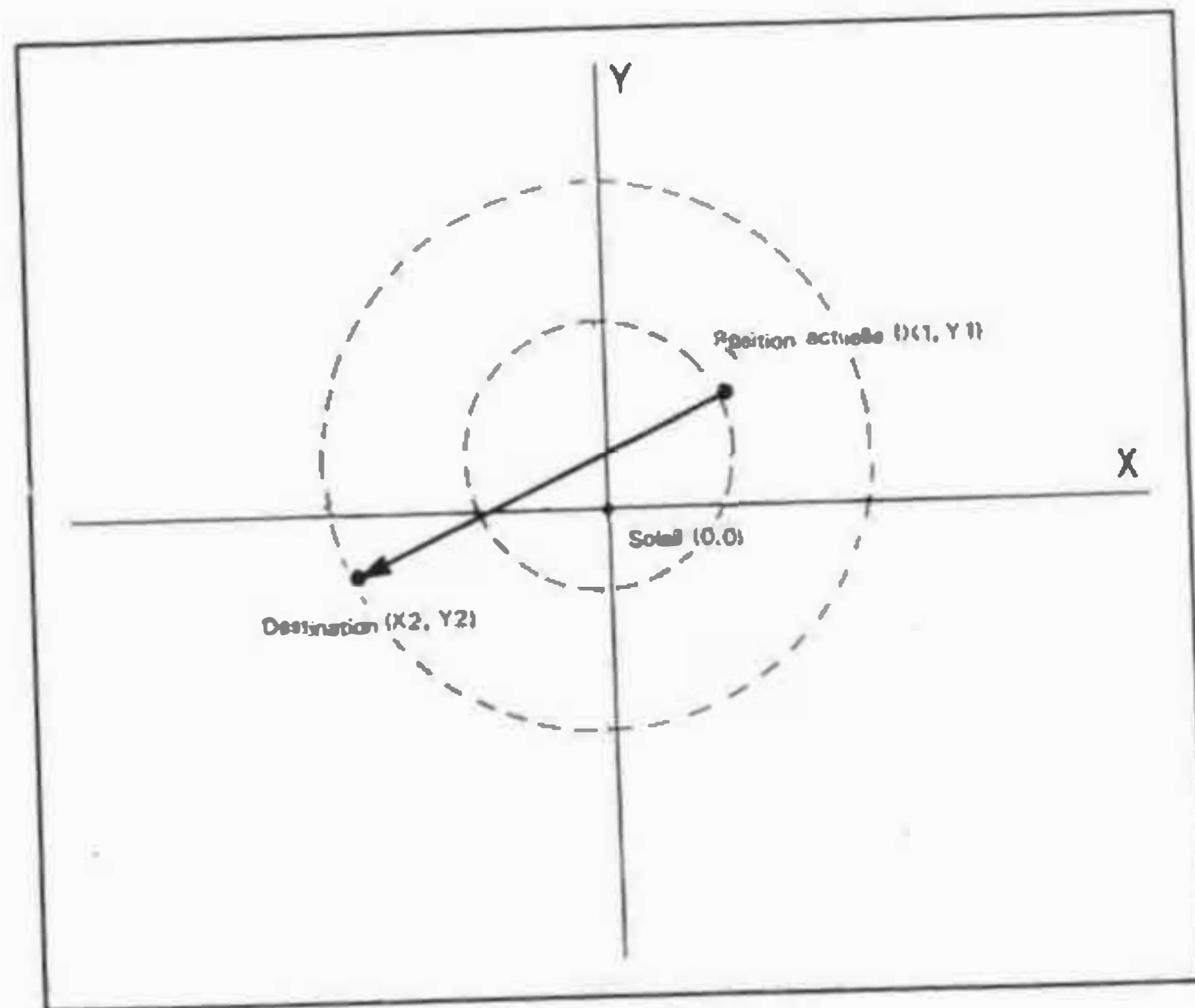
Or: 10 Charbon: 30 Fer: 20

Hydrogène: 40 Oxygène: 40 Uranium: 20

Vous pouvez également emporter de l'argent uniquement et acheter toutes les ressources naturelles. Dans un tel cas, vous recevez 10.000.

■ Arrivée et départ d'une planète

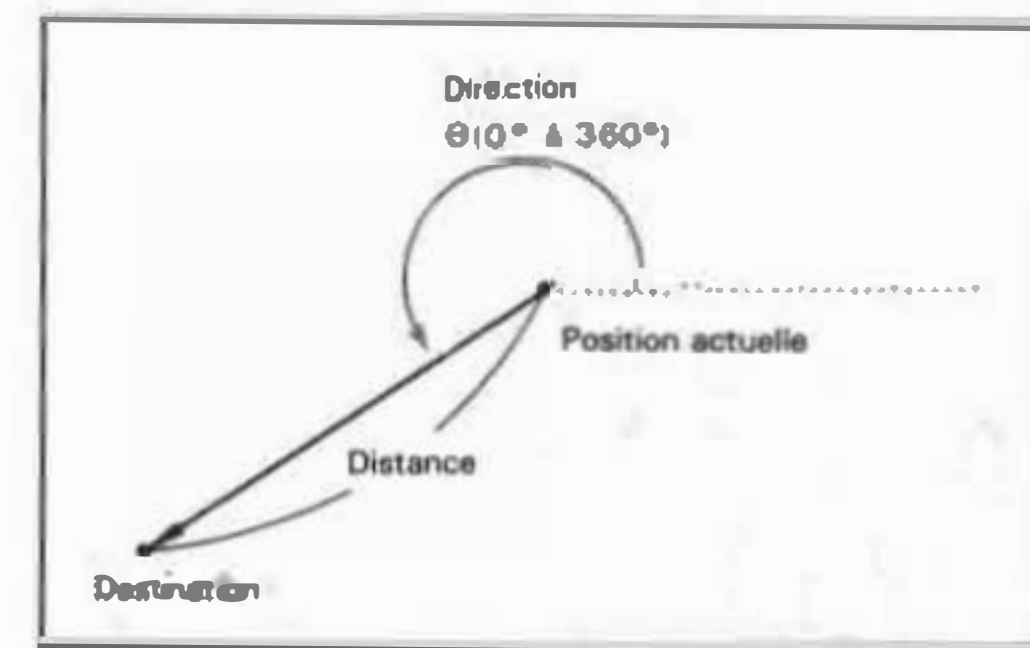
Premièrement, vous spécifiez la planète que vous désirez visiter. Lorsque c'est fait, ses coordonnées sont affichées, dans deux dimensions, avec le soleil en son centre. Les coordonnées sont indiquées en unités de 10.000 km, et toutes les entrées/sorties de ce jeu se font dans ces unités.



Lors du départ, la direction et la distance doivent être indiquées. Celles-ci sont calculées à partir de la position actuelle à la destination. Lorsque la distance est inférieure à 100 millions de km, la direction et la distance peuvent être calculées au moyen de l'ordinateur. Lorsqu'elle est supérieure, les calculer de tête et déplacer le curseur à l'intérieur de la gamme de 100 millions de km de la destination, puis

faire prendre la relève à l'ordinateur pour calculer la direction et la distance restante. La consommation d'énergie est proportionnelle au carré de la distance parcourue.

Par conséquent, il est dangereux de parcourir plus de 100 millions de km, à moins de sauter d'un point à un autre en plusieurs étapes. Lorsque le vaisseau spatial approche d'une planète, il s'agit de se préparer à se poser. A ce point, le vaisseau et la planète sont affichés sur l'écran. Au moyen des touches de curseur, déplacer le vaisseau en direction de la planète. L'atterrissage est réussi lorsque le vaisseau passe sur la planète.



■ Négoce de ressources naturelles

A l'arrivée sur la planète, il faut d'abord vendre ses ressources naturelles, puis acheter celles disponibles sur la planète. Le prix des ressources est variable selon la planète, et évolue quotidiennement. Il existe toutefois une certaine norme sur le marché.

Lors de la vente de ressources, si vous fixez votre prix trop haut, nous ne pourrez pas en vendre une grande quantité. Si vous le fixez trop bas, vous ne ferez pas beaucoup de bénéfice. Vous pouvez ainsi faire preuve de votre habileté à vendre vos ressources naturelles aux prix qui vous procureront le bénéfice maximum.

A l'achat des ressources naturelles, les prix sont plus bas par grandes quantités. Si vous en achetez trop peu, les prix seront élevés.

Lorsque le négoce est terminé, vous vous rendez sur une autre planète. Pour terminer le programme, retournez sur terre.

■ Retour sur terre

Au retour sur terre, les comptes sont bouclés de la manière suivante.

- Toutes les ressources naturelles et l'énergie restantes sont converties en argent à 90% de leur valeur du marché actuelle.
- Vous touchez un bonus si vous rentrez dans les 180 jours.
- Vous touchez une amende si vous avez manqué une ou plusieurs planètes.

A la fin, le jeu se termine par l'affichage du montant total d'argent, du temps de voyage et du bénéfice.



5.2 Explication du programme

Comme le montre l'organigramme, le programme se compose d'un programme principal et de plusieurs sous-programmes.

A. Programme principal

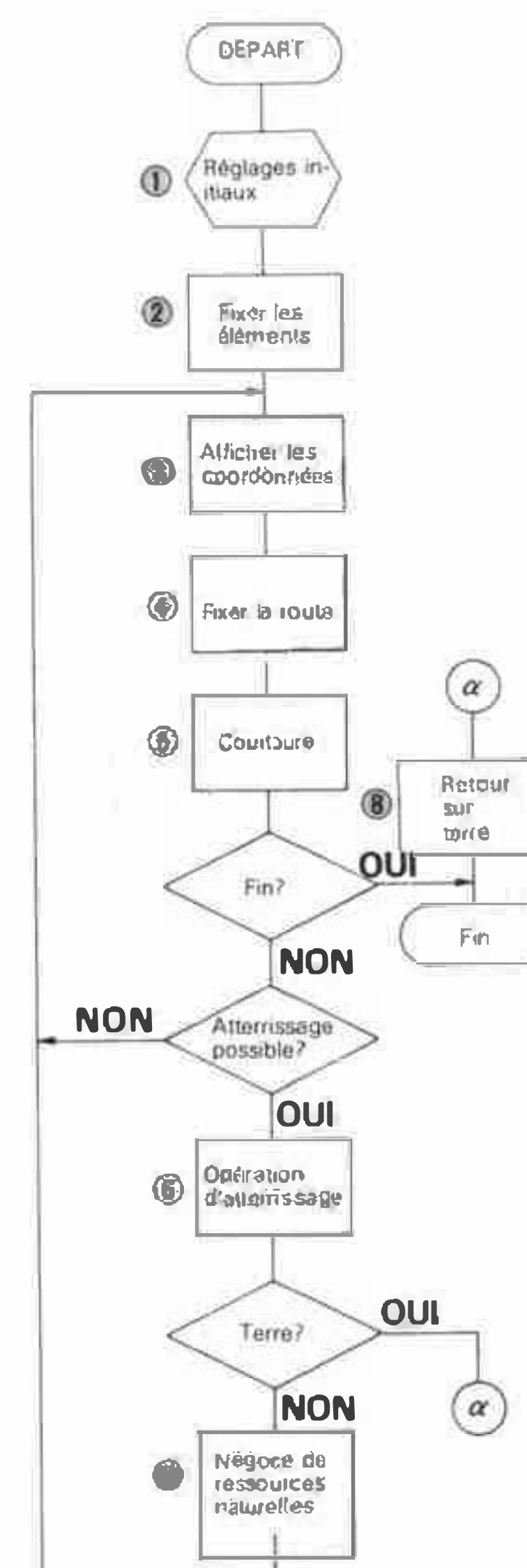
Le programme se déroule de la manière suivante.

① Premièrement, les prix initiaux du marché des ressources naturelles sont fixés. ② L'argent, l'énergie et les ressources naturelles à emporter sont distribués.

③ Lorsque les préparatifs de départ sont terminés, la planète de destination est entrée sur le clavier, et ses coordonnées sont affichées. ④ Ensuite, la route (direction et distance) du vaisseau spatial sont fixés.

⑤ Lorsque la route est fixée, le vaisseau est déplacé en conséquence. Si l'on arrive à bout d'énergie durant le voyage, le programme est terminé.

Si le vaisseau spatial se trouve à distance appropriée de la planète, ⑥ l'opération d'atterrissage est possible.



Si le vaisseau spatial ne se trouve pas à distance appropriée de la planète, l'exécution retourne à (3).

Si l'atterrissage est réussi, (7) le négoce des ressources naturelles peut avoir lieu.

Lorsque le négoce est terminé, l'exécution retourne à (3) pour le voyage vers une autre planète.

Lorsque le vaisseau spatial retourne sur terre, (8) les comptes sont bouclés, les résultats sont affichés et le jeu est terminé.

● Réglages initiaux

Plusieurs réglages initiaux sont opérés à ce stade. Tout d'abord, les quatre fonctions suivantes sont définies:

FNA: Recherche l'angle, en degrés, de la ligne reliant les coordonnées (A,B) et (C,D)

FNK: Recherche la distance entre les coordonnées (A,B) et (C,D)

FNR: Convertit l'angle en radians

FNS: Produit le prix minimum d'achat de la quantité B de ressources naturelles au prix du marché A.



Aux lignes 80 à 200, les données suivantes sont fixées et introduites dans le tableau P:

Données fixées en P(I,J)

	0	1	2	3	4	5	6	7	8	9	10	
0	0	0	0	0	0	0	0	0	0	0	0	Vous
1	57	.24	1	0	0	90	40	50	30	40	40	Mercure
2	108	.62	2	0	0	100	20	40	10	10	50	Vénus
3	150	1	2	0	0	100	30	40	20	10	70	Terre
4	228	1.88	1	0	0	100	30	30	20	20	60	Mars
5	778	11.86	28	0	0	110	10	50	10	20	60	Jupiter
6	1426	29.46	24	0	0	90	10	50	10	20	70	Saturne
7	2871	84.08	10	0	0	90	40	40	10	30	40	Uranus
8	4497	164.82	10	0	0	80	30	40	10	30	80	Neptune
9	5914	248.54	8	0	0	90	40	30	20	40	60	Pluton

Distance du soleil
(millions de km)

Temps de révolution

Rayon
(250 millions de km)

Potentiel
inertiel
(rad/s)

Drapeau de visite

Or

Charbon

Fer

Hydrogène

Oxygène

Uranium

Prix du marché

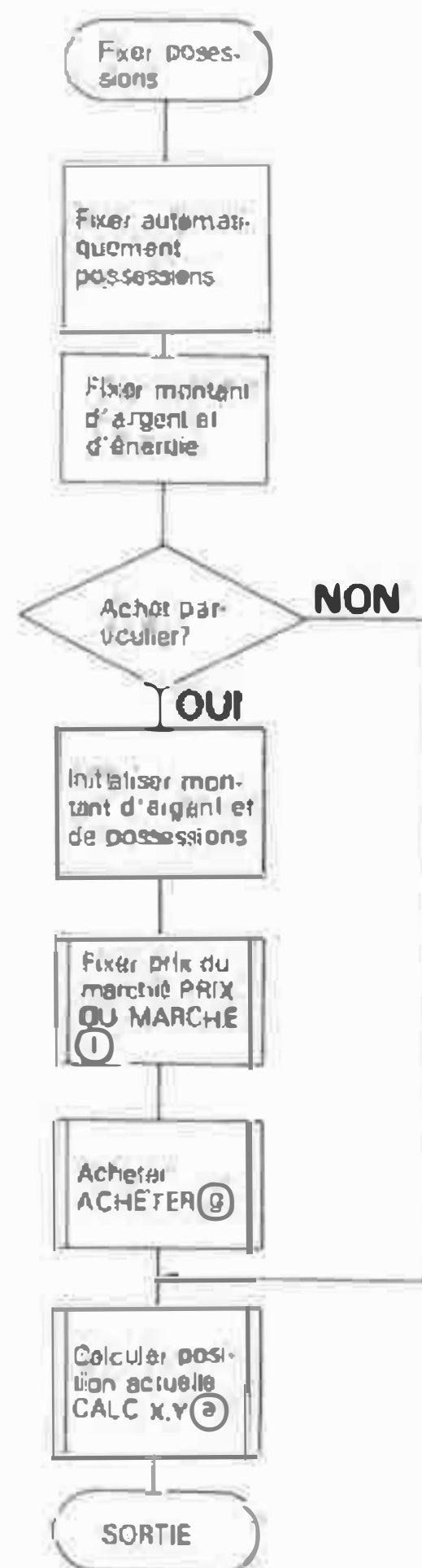
Les lignes 210 à 250 fixent les chaînes de caractères et les placent dans la variable A\$.

Les lignes 260 à 290 placent la position actuelle d'une planète dans le tableau P(I,3). La position actuelle est indiquée en radians, et est produite à l'aide de nombres aléatoires. La ligne 290 fixe votre position actuelle comme celle de la terre.

② Fixation de vos possessions

Aux lignes 300 à 420 sont fixées vos possessions initiales et la date du départ. Aux lignes 300 à 330, le montant de ressources naturelles, d'argent et d'énergie sont fixés.

Ensuite, on vous demandera si vous désirez faire des achats particuliers. Si oui, vous pouvez acheter pour $M=10.000$ de ressources naturelles et $P(0,1)$ est remis à zéro. Finalement, I est ajouté à D , le nombre de jours écoulés, pour obtenir les coordonnées de la position actuelle (terre).



Argent $M = 4700$
Energie $E = 50000$

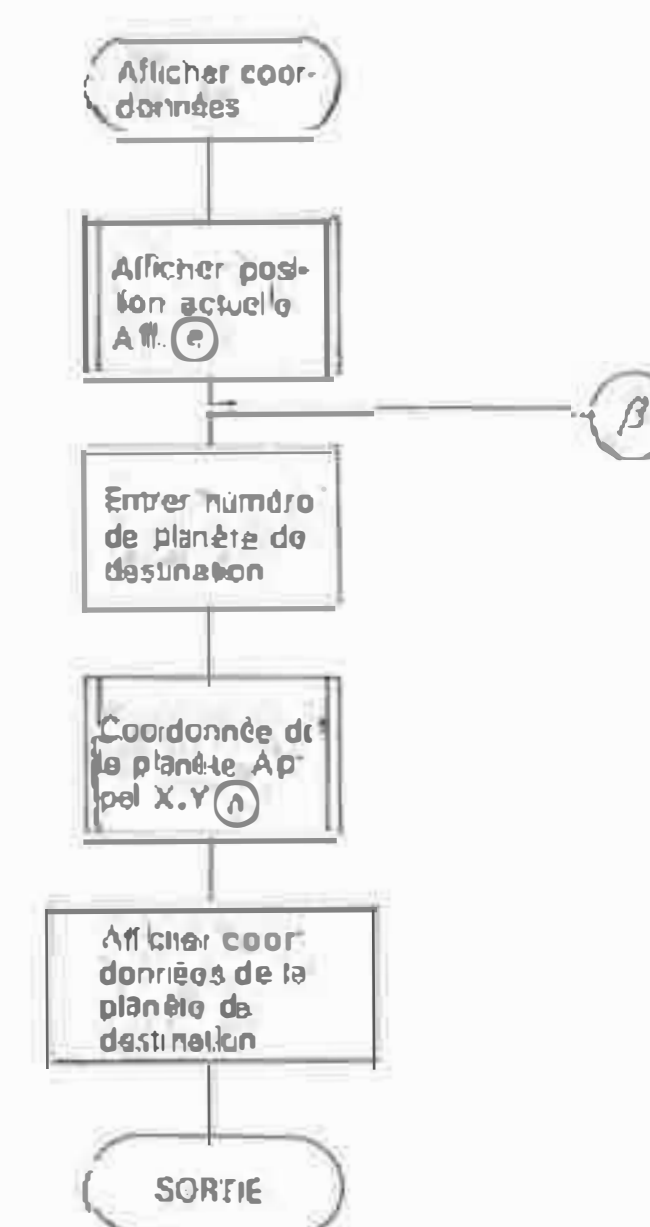
0	1	2	3	4	5	6	7	8	9	10	$P(I, J)$ Ressources naturelles
					10	30	20	40	40	20	
					Or	Charbon	Fer	Hydrogène	Oxygène	Uranium	

③ Affichage des coordonnées

Les lignes 430 à 490 calculent et affichent les coordonnées de la position actuelle et celles de la planète de destination. Cette dernière peut être spécifiée à l'aide des chiffres suivants:

1 = Mercure 2 = Vénus
3 = Terre 4 = Mars
5 = Jupiter 6 = Saturne
7 = Uranus 8 = Neptune
9 = Pluton

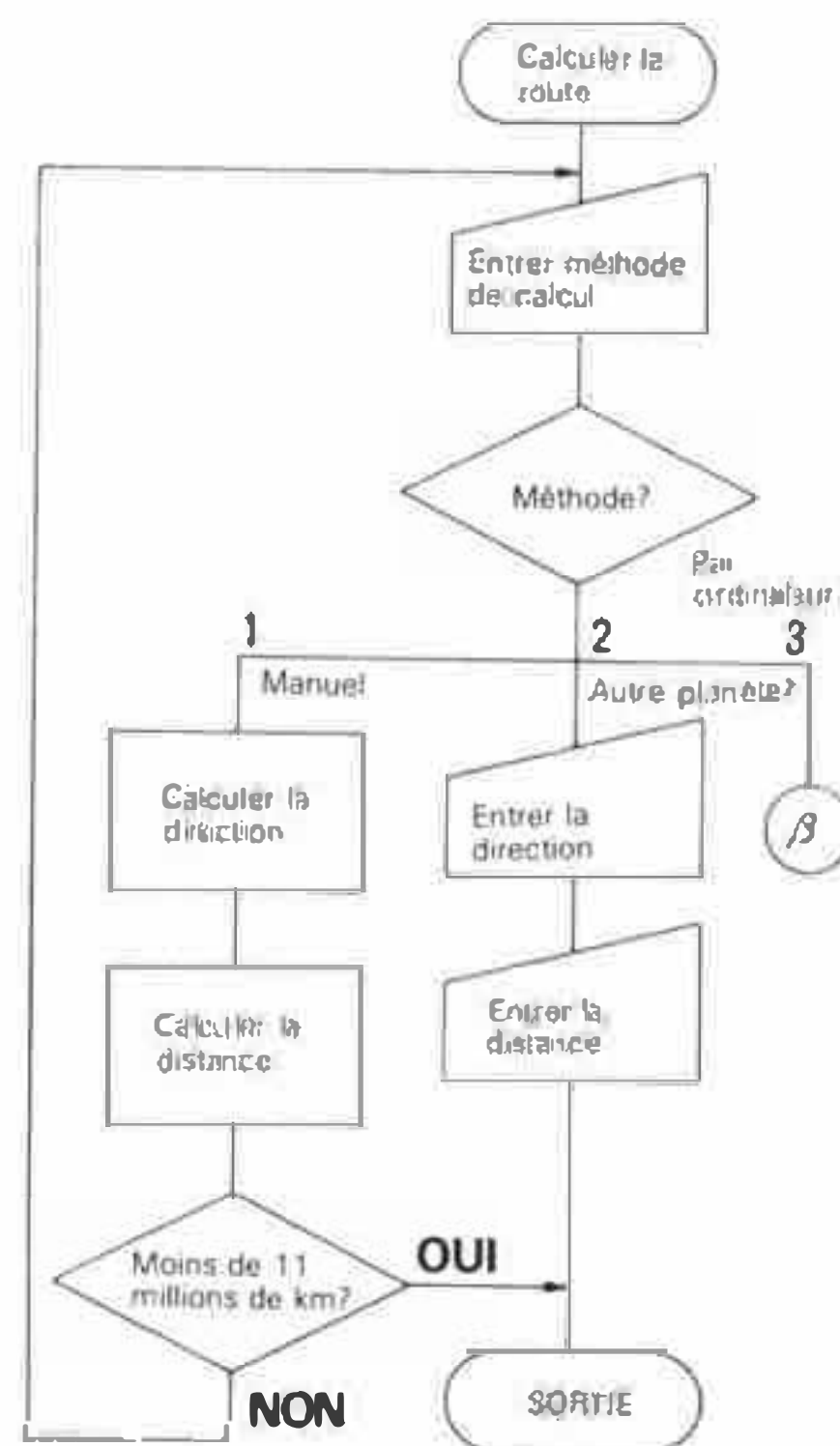
Un autre jour s'écoule durant cette étape.



④ Calcul de la route

Les lignes 500 à 690 calculent la route du vaisseau spatial. La méthode de calcul, qui diffère selon le chiffre choisi, est entrée à la ligne 510.

1. L'ordinateur calcule automatiquement la direction et la distance.
2. La direction et la distance sont frappées sur le clavier.
3. Vérification des coordonnées d'une autre planète.



La sélection est décidée aux lignes 510 et 520. Lorsque $N=1$, le contrôle est transféré à la ligne 540, Lorsque $N=2$ à la ligne 660, Lorsque $N=3$ à la ligne 450 et retour à la ligne 510 pour tout autre chiffre.

L'ordinateur n'effectue le calcul que lorsque la distance est inférieure à 100 millions de km. Lorsque la distance est supérieure, le programme affiche que la distance doit être calculée de tête, et retourne à la ligne d'entrée de la méthode de calcul.

Lorsque la méthode de calcul 3 est entrée, on obtient à nouveau l'affichage des coordonnées.

⑤ Courbure

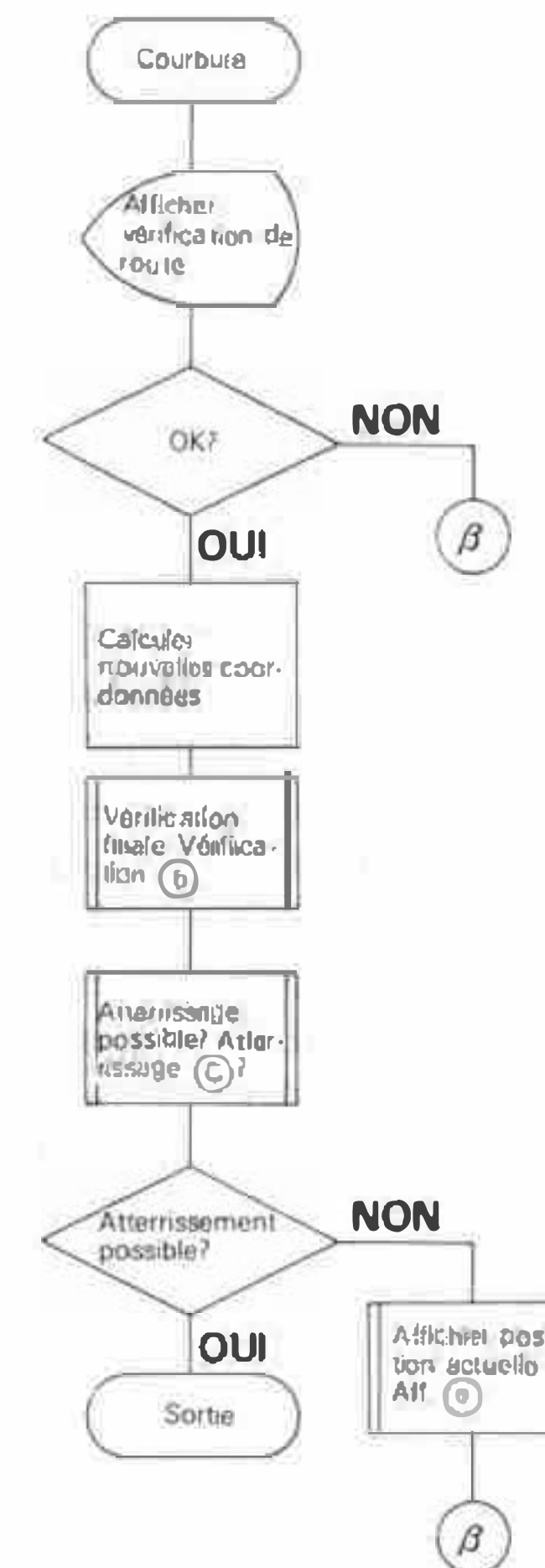
Aux lignes 700 à 810, la position actuelle du vaisseau spatial est déplacée en fonction de la route calculée (direction et distance).

En premier lieu, l'affichage de vérification de route apparaît. S'il est faux, frapper toute autre touche que "Y" pour revenir à l'affichage des coordonnées.

Si c'est correct, l'énergie consommée (ligne 770) et la nouvelle position actuelle sont fixées (lignes 780 et 790).

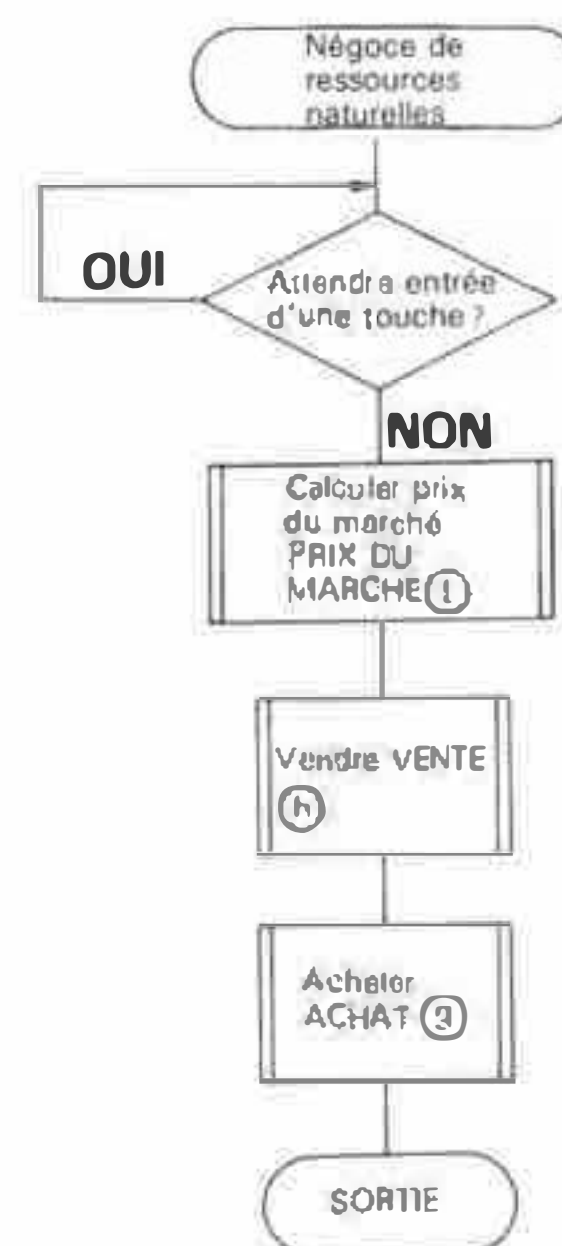
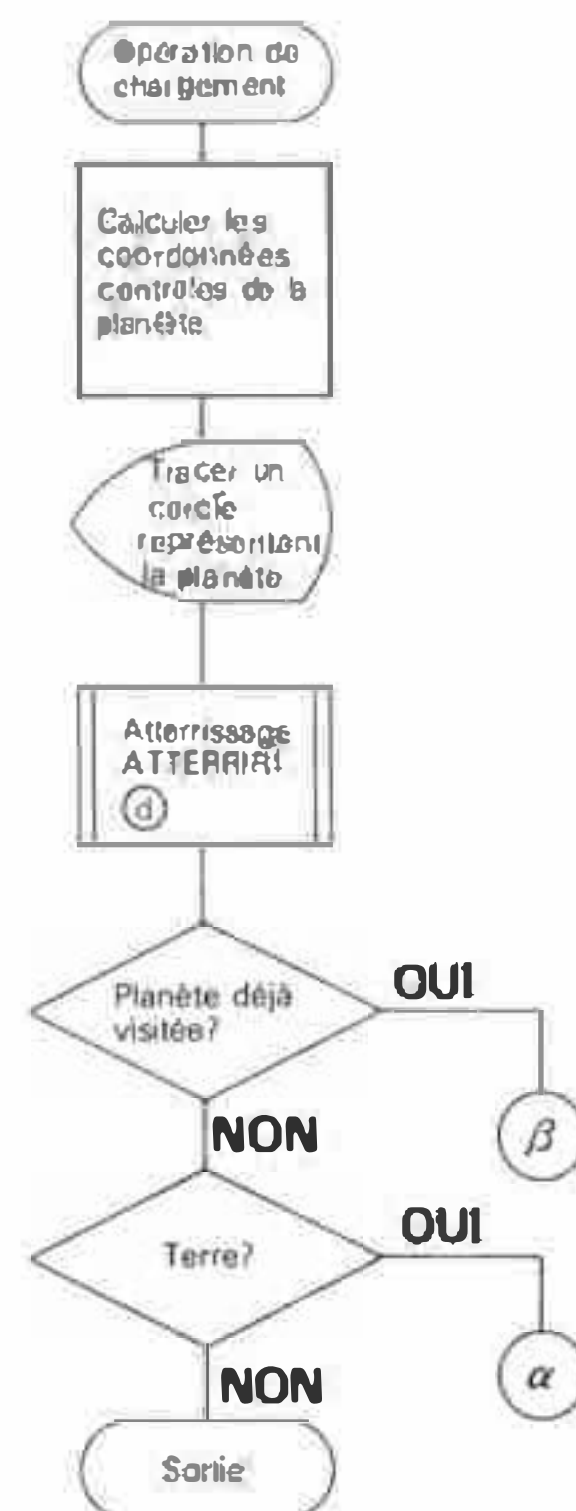
Cette étape vérifie s'il reste de l'énergie et si vous ne vous êtes pas écrasé sur le soleil (GOSUB 1060).

Finalement, il est vérifié si la position actuelle se trouve à l'intérieur de la gamme d'atterrissage possible. Si ce n'est pas le cas, la position actuelle avec ses coordonnées est affichée.



⑥ Atterrissage

L'opération d'atterrissage est effectuée aux lignes 820 à 890. L'écran est d'abord effacé (ligne 820). Ensuite, les coordonnées centrales de la planète sont calculées (ligne 840), et un cercle représentant celle-ci est tracé (ligne 850). Le vaisseau spatial est représenté par un point, qui peut être déplacé au moyen des touches de commande du curseur. L'atterrissage est terminé lorsque le vaisseau spatial passe sur la planète (GOSUB 1210). Si cette planète a déjà été visitée, un bip d'avertissement est produit, et les coordonnées sont à nouveau présentées (ligne 890). Ou bien, si la planète sur laquelle le joueur est arrivée est la terre, l'exécution passe à l'étape "retour sur terre" (ligne 890).



⑦ Négoce des ressources naturelles

Les ressources naturelles sont vendues et achetées aux lignes 900 à 940.

Lorsqu'une touche est frappée (n'importe laquelle), l'exécution est transférée dans l'ordre suivant:

Fixation des prix du marché (GOSUB 1510)

Vente de ressources naturelles (GOSUB 1810)

Achat de ressources naturelles (GOSUB 1610)

Lorsque les affaires sont terminées, vous partez pour la prochaine planète (ligne 940).

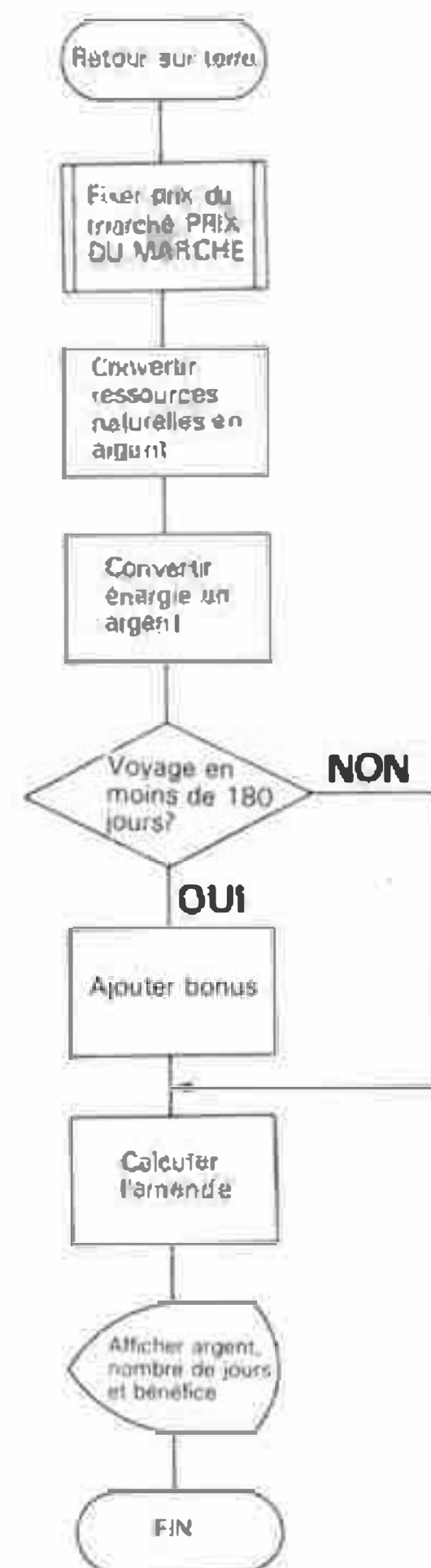
⑧ Retour sur terre

Les lignes 2000 à 2210 exécutent le processus final lors du retour sur terre.

Premièrement, les prix du marché sont fixés (ligne 2030).

Ensuite, les ressources naturelles que vous avez rapportées sur terre sont converties en argent selon les prix du marché (lignes 2050 à 2070). L'énergie restante est également convertie en argent (ligne 2100). Un bonus est ajouté si vous êtes rentré en moins de 180 jours (ligne 2110).

Une amende est soustraite si vous avez évité une ou plusieurs planètes (lignes 2120 à 2160). Finalement, le montant total d'argent, les jours passés en voyage et le bénéfice obtenu sont affichés sur l'écran, et le jeu est terminé.



B. Sous-programmes

Ce programme contient les 8 sous-programmes suivants:

- Ⓐ Calcul des coordonnées d'une planète
- Ⓑ Vérification
- Ⓒ Vérification de la possibilité d'atterrissage
- Ⓓ Opération d'atterrissage
- Ⓔ Affichage de la position actuelle, etc.
- Ⓕ Fixation des prix du marché
- Ⓖ Achat de ressources naturelles
- Ⓗ Vente de ressources naturelles

Ⓐ Calcul des coordonnées d'une planète (lignes 1000 à 1040)

Entrée: P:Numéro de planète (0 à 9)

D:Date actuelle

Sortie: X:Coordonnée X de la position actuelle

Y:Coordonnée Y de la position actuelle

Ce sous-programme calcule et produit la position actuelle (X,Y) de la planète entrée.

$$R \text{ (angle)} = \frac{6,28 \text{ (radian)}}{\text{(ligne 1010) cycle de rotation (ans)} * 365} * \text{jours} + \text{angle initial}$$

$$X \text{ (10.000km)} = \cos(R) * \text{distance du soleil} * 100 \text{ (ligne 1020)}$$

$$Y \text{ (10.000km)} = \sin(R) * \text{distance du soleil} * 100 \text{ (ligne 1030)}$$

Ⓑ Vérification (lignes 1050 à 1080)

Entrée: E(énergie)

XI Coordonnée de la position actuelle
YI

Sortie: Conditions de fin

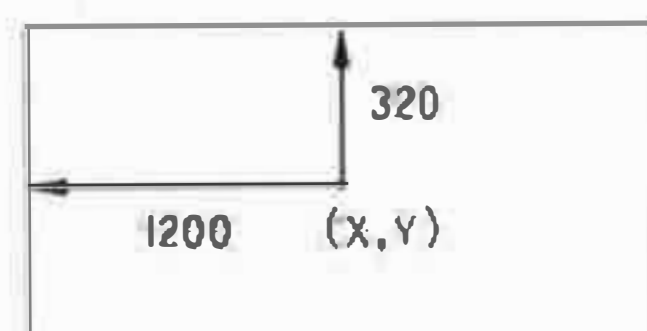
Lorsque le montant d'énergie est négatif (ligne 1060), ou que la position actuelle est trop rapprochée du soleil (ligne 1070), un message est affiché et le programme est terminé.

Bien qu'il soit ordinairement préférable de ne pas quitter un sous-programme par une expression GOTO, il n'y a pas de problème ici car le programme se termine immédiatement à la ligne 2210.

© Vérification de la possibilité d'atterrissage (lignes 1100 à 1160)

Entrée: X, Y: Position de la planète
XI, X2: Position actuelle
Sortie: P: Numéro de planète
≤ 9: Atterrissage possible
> 9: Atterrissage impossible

L'atterrissage est possible lorsque la position actuelle (XI, YI) se trouve dans la gamme indiquée.



A la ligne 1120, la position de chaque planète est vérifiée, et leur éloignement de la position actuelle est calculé. Si l'atterrissage est possible, la boucle se termine en fixant I à 20. Si la planète est encore trop éloignée, la boucle est également terminée, I étant fixé à 10. Cette décision est faite à la ligne 1150, et P est fixé à 10 lorsque l'atterrissage est possible, puis l'exécution est retournée sur le programme principal. C'est une procédure compliquée, qui est utilisée parce qu'il est préférable d'éviter de quitter une boucle FOR - NEXT par une instruction GOTO.

Opération d'atterrissage (lignes 1200 à 1320)

Entrée: XD, YD: Coordonnées centrales de la planète
P: Numéro de la planète (1 ~ 9)
IS: Entrée sur le clavier

Sortie: Affiche jusqu'à ce que l'atterrissage soit terminé.

Premièrement, la position initiale du vaisseau spatial est créée à l'aide de nombres aléatoires (ligne 1210). Si cette position initiale se trouve à l'intérieur de la planète, elle est régénérée (ligne 1220).

Le vaisseau spatial est déplacé et sa vitesse est réglée au moyen des touches de commande du curseur.

→ CHR\$ (28) → PX = PX + 1

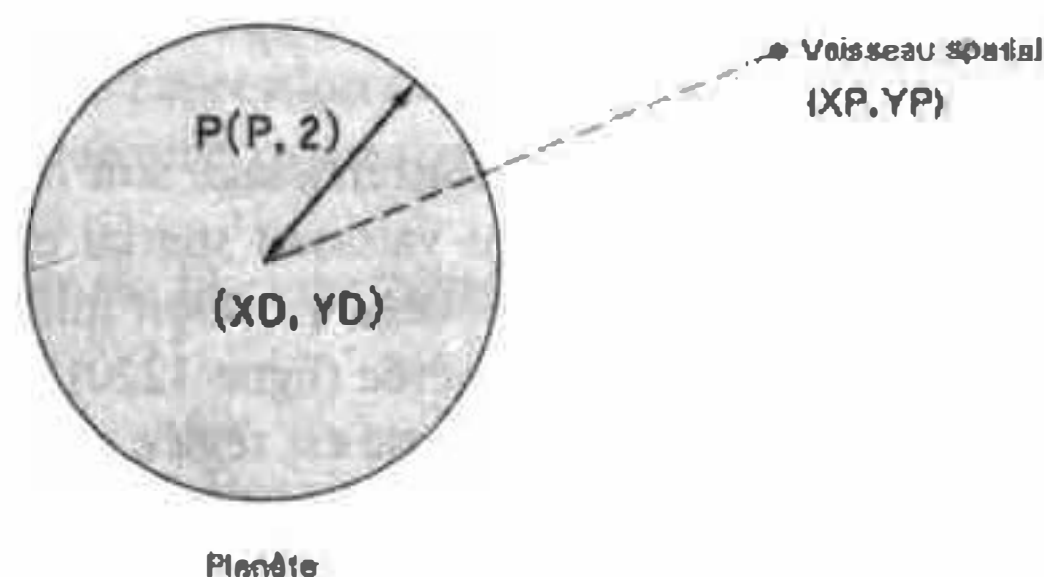
← CHR\$ (29) → PX = PX - 1

↑ CHR\$ (30) → PY = PY - 1

↓ CHR\$ (31) → PY = PY + 1

La position du vaisseau spatial est indiquée par XP et YP. PX et PY indiquent la vitesse du vaisseau spatial, qui varie selon que le résultat de l'expression logique des lignes 1240 et 1250 est vrai (-1) ou faux (0). Elle est utilisée à la ligne 1280 pour déplacer la position actuelle du vaisseau spatial. En d'autres termes, le vaisseau spatial est déplacé à une vitesse fixe après qu'une touche de curseur ait été frappée. Le sous-programme vérifie également que le vaisseau spatial ne quitte pas l'écran (lignes 1290 à 1300).

Le vaisseau spatial a atterri lorsqu'il passe sur la planète.



③ Affiche (lignes 1400 à 1440)

Entrée: M: Argent

E: Energie

X1, Y1: Coordonnées de la position actuelle

Sortie: Affichage sur l'écran

Affiche le montant d'argent et d'énergie disponible ainsi que les coordonnées de la position actuelle.

① Fixation des prix du marché (lignes 1500 à 1530)

Entrée: P: Numéro de la planète

P(P,1): Prix courants du marché

Sortie: P(P,1): Nouveaux prix du marché

Les prix du marché sont fixés comme suit:

Nouveau prix du marché = prix courant du marché $\times$ (RND(1) $\times$ 0,4 + 0,8)

② Achat de ressources naturelles (lignes 1600 à 1730)

Entrée: M: Argent

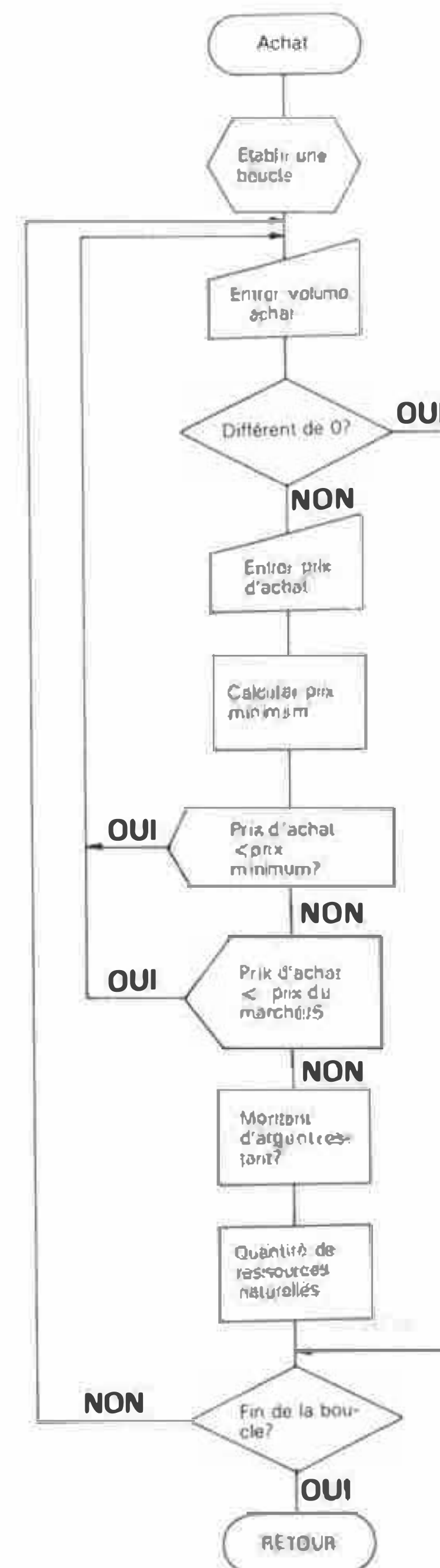
P(0,1): vos ressources naturelles

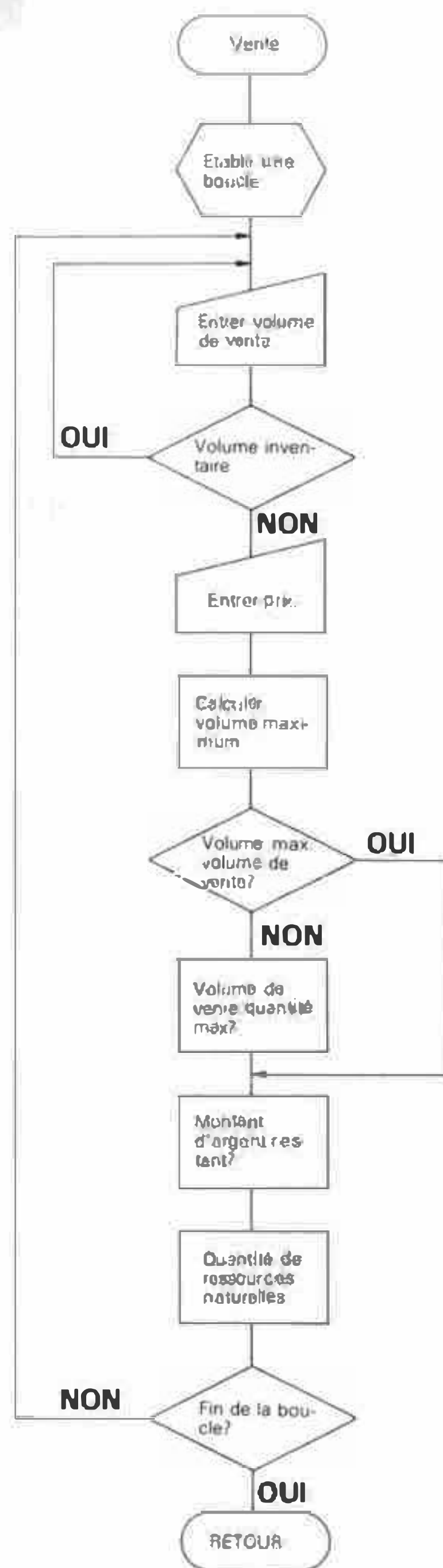
Sortie: M: Argent

P(0,1): vos ressources naturelles

D: Temps écoulé

Les quantités achetées et le prix de chaque ressource sont entrés (lignes 1650 à 1670). Le montant d'argent est diminué et le montant de ressources naturelles est augmenté en conséquence (ligne 1710). Cependant, comme le prix minimum pour chaque volume d'achat est déjà fixé (ligne 1680), le sous-programme demande la réintroduction du prix s'il est inférieur au minimum. La réintroduction est également nécessaire si le prix entré est inférieur à 1/5 du prix du marché (1690). La date est avancée d'un jour à chaque achat de ressources naturelles. Une réintroduction ajoute également un jour.





Vente de ressources naturelles (lines 1800 à 1920)

Entrée: M: Argent
P(0,I): Vos ressources naturelles

Sortie: M: Argent
P(0,I): vos ressources naturelles

Le volume et le prix de vente de chaque ressource sont entrés (lignes 1820 à 1860). Le montant d'argent est augmenté et celui des ressources naturelles est diminué en conséquence. Cependant, le volume maximum de vente de chaque ressource est déterminé par le prix d'entrée et le prix du marché.

Volume maximum de vente = (prix du marché/prix)¹⁰ (ligne 1870)

En conséquence, il n'est pas toujours possible de vendre le volume de ressources désiré (lignes 1880 à 1900).

Un jour est dépensé lors de l'achat de chaque ressource naturelle.

C. Liste du jeu

```

① 10 REM *** JEU DU MARCHAND DE L'ESPACE *
   **
  20 DEFINT I,N:R=RND(0)
  30 DIM P(9,10),A$(16)
  40 DEFFNA(A,B,C,D)=ATN((D-B)/(C-A))*180/
    3.141592653589806#
  50 DEFFNK(A,B,C,D)=SQR((C-A)^2+(D-B)^2)
  60 DEFFNR(A)=A/180*3.141592653589806#
  70 DEFFNS(A,B)=A*(.7-.4*(ATN(B/40-1)))
  80 FOR I=1 TO 9
  90 FOR J=0 TO 10
 100 READ P(I,J)
 110 NEXT J,I
 120 DATA 57,.24,1,0,0,90,40,40,30,40,40
 130 DATA 108,.62,2,0,0,100,20,40,10,10,5
    0,0
 140 DATA 150,1,2,0,0,100,30,40,20,10,70
 150 DATA 228,1.88,1,0,0,100,30,30,20,20,
    60
 160 DATA 778,11.86,28,0,0,110,10,50,10,2
    0,60
 170 DATA 1426,29.46,24,0,0,90,10,50,10,2
    0,70
 180 DATA 2871,84.08,10,0,0,90,40,40,10,3
    0,40
 190 DATA 4497,164.82,10,0,0,80,30,40,10,
    30,80
 200 DATA 5914,248.54,8,0,0,90,40,30,20,4
    0,60
 210 FOR I=0 TO 16
 220 READ A$(I):NEXT
 230 DATA "VOTRE ","MERCURE","VENUS","TER
  
```


RE", "MARS"

240 DATA "JUPITER", "SATURNE", "URANUS", "NEPTUNE", "PLUTON"

250 DATA "ESPACE", "OR", "CARBON", "FER", "HYDROGENE", "OXYGENE", "URANIUM"

260 FOR I=1 TO 9

270 P(I,3)=RND(1)*6.283185307179603#

280 NEXT

290 P(0,3)=P(3,3)

② 300 FOR I=5 TO 10

310 READ P(0,I):NEXT

320 DATA 10,30,20,40,40,20

330 M=4700:E=50000#

340 REM *** DEPART ***

350 CLS:P=3

360 PRINT "ACHAT PARTICULIER?"

370 Y\$=INKEY\$:IF Y\$="" THEN 370

380 IF Y\$<>"Y" THEN 410

390 FOR I=5 TO 10:P(0,I)=0:NEXT

400 M=10000:GOSUB 1510:GOSUB 1010

410 M=M+1:GOSUB 1010

420 X1=X:Y1=Y

③ 430 REM *** ESPACE ***

440 CLS:GOSUB 1410

450 PRINT "QUELLE PLANETE(1-9)?";

460 N=INSTR("0123456789",INKEY\$)-1

470 IF N<0 THEN 460

480 PRINT:P=N:Q=0+1:GOSUB 1010

490 PRINT A\$(N); " DONT LES COORDONNEES SONT",INT(X); " ";INT(Y)

④ 500 PRINT "1=ORDINATEUR:2=MAN:3=AUTRE?";

510 ON VAL(INKEY\$) GOTO 540,660,450

520 GOTO 510

530 REM *** ORDINATEUR ***

540 PRINT " CALCUL DE LA ROUTE POUR ";A\$(P)

550 FOR I=1 TO 24

560 BEEP I,1:NEXT

570 IF X1>X THEN 610

580 IF Y1>Y THEN 600

590 C1=FNA(X1,Y1,X,Y):GOTO 620

600 C1=FNA(X1,Y1,X,Y)+360:GOTO 620

610 C1=FNA(X1,Y1,X,Y)+180

620 K=FNK(X1,Y1,X,Y):R=RND(1)^6

630 IF K<10000 THEN 690

640 BEEP 4,10:PRINT "TROP LOIN; CALCULER MANUELLEMENT!"

650 REM *** MANUEL ***

660 PRINT " NAVIGATION MANUELLE"

670 INPUT "DIRECTION(0-360)";C1

680 INPUT "DISTANCE=";K:R=0

690 C=FNR(C1)

⑤ 700 REM *** COURBURE ***

710 PRINT "DIRECTION=";C1

720 PRINT "DISTANCE=";K

730 INPUT "OK";Y\$

740 IF Y\$<>"Y" THEN 450

750 PRINT "COURBURE?"

760 BEEP 10,20

770 E=E-100-K^2/10000000#

780 X1=X1+COS(C)*K*(RND(1)*R+1)

790 Y1=Y1+SIN(C)*K*(RND(1)*R+1)

800 GOSUB 1060:GOSUB 1110

810 IF P=10 THEN GOSUB 1410:GOTO 450

```

⑤ 820 PX=0:PY=0:CLS
830 GOSUB 1010
840 XD=60+(X-X1)/20:YD=16-(Y-Y1)/20
850 CIRCLE(XD,YD),P(P,2)
860 GOSUB 1210:BEEP 15,10:CLS
870 PRINT "ATTERRI SUR ";A$(P)
880 IF P(P,4)=1 THEN PRINT "DEJA VISITEE
?" :BEEP 2,10:GOTO 440
890 P(P,4)=1:IF P=3 THEN 2010
⑦ 900 IF INKEY$="" THEN 900
910 PRINT "VENDRE":GOSUB 1510:GOSUB 1810
920 PRINT "ACHERER":GOSUB 1610
930 PRINT "DEPART?":BEEP 5,5
940 GOTO 440
⑧ 1000 REM *** CALC X,Y ***
1010 R=6.283185307179605#/P(P,1)/365.242
*D+P(P,3)
1020 X=COS(R)*P(P,0)*100
1030 Y=SIN(R)*P(P,0)*100
1040 RETURN
⑨ 1050 REM *** VERIFICATION ***
1060 IF E<0 THEN PRINT "PLUS D'ENERGIE":
GOTO 2210
1070 IF X1^2+Y1^2<20000 THEN PRINT "COLL
ISION AVEC LE SOLEIL":GOTO 2210
1080 RETURN
⑩ 1100 REM *** ATTERRISSAGE? ***
1110 FOR I=1 TO 9
1120 P=I:GOSUB 1010
1130 IF ABS(X-X1)<1200 AND ABS(Y-Y1)<320
THEN I=20

```

```

1140 NEXT
1150 IF I<11 THEN P=10
1160 RETURN
⑪ 1200 REM *** ATTERRI? ***
1210 XP=RND(1)*120:YP=RND(1)*32
1220 IF FNK(XD,YD,XP,YP)<P(P,2) THEN 121
0
1230 PSET(XP,YP):I$=INKEY$
1240 PX=PX-(I$=CHR$(28)):PX=PX+(I$=CHR$(
29))
1250 PY=PY+(I$=CHR$(30)):PY=PY-(I$=CHR$(
31))
1260 E=E+20*(I$>CHR$(27) AND I$<CHR$(32)
)
1270 PRESET(XP,YP)
1280 XP=XP+PX:YP=YP+PY
1290 IF XP<0 THEN XP=0 ELSE IF XP>119 TH
EN XP=119
1300 IF YP<0 THEN YP=0 ELSE IF YP>31 THE
N YP=31
1310 IF FNK(XD,YD,XP,YP)<P(P,2) THEN RET
URN
1320 BEEP 5,1:GOTO 1230
⑫ 1400 REM *** AFFICHAGE ***
1410 PRINT "M=";MID$(STR$(M),2);
1420 PRINT " E=";MID$(STR$(INT(E)),2);
1430 PRINT " ICI ";INT(X1);INT(Y1)
1440 PRINT A$(P);".":RETURN
⑬ 1500 REM *** PRIX DU MARCHE ***
1510 FOR I=5 TO 10
1520 P(P,1)=INT(P(P,1)*(RND(1)*.4+.8)):N
EXT

```

L 1530 RETURN

Ⓢ 1600 REM *** ACHETER ***

1610 PRINT A\$(0); "ARGENT="; M

1620 FOR I=5 TO 10

1630 PRINT A\$(0)+A\$(I+6); "="; P(0, I)

1640 PRINT A\$(I+6); " DONT LE PRIX DU MARCHE EST "; P(P, I)

1650 INPUT "QUANTITE ACHETEE"; N

1660 IF N<=0 THEN 1720 ELSE D=D+1

1670 INPUT "PRIX UNITAIRE"; W

1680 Z=FNS(P(P, I), N)

1690 IF Z>W OR W*5<P(P, I) THEN PRINT "VENTE IMPOSSIBLE A CE PRIX":GOTO 1630

1700 IF N*W>M THEN PRINT "ARGENT="; M; "RE
STANT!":GOTO 1640

1710 M=M-N*W:P(0, I)=P(0, I)+N

1720 PRINT "ARGENT RESTANT="; M

L 1730 NEXT:RETURN

Ⓢ 1800 REM *** SALE ***

1810 FOR I=5 TO 10

1820 PRINT A\$(I+6); " DONT LE PRIX DU MARCHE EST "; P(P, I)

1830 PRINT "EN MAINS="; P(0, I):INPUT "QUANTITE VENDUE"; N

1840 IF N>P(0, I) THEN PRINT A\$(I+6); P(0, I) "RESTANT!":GOTO 1830

1850 IF N<=0 THEN 1910 ELSE D=D+1

1860 INPUT "PRIX UNITAIRE"; W

1870 Z=(P(P, I)/W)^10

1880 IF Z<N THEN N=Z

1890 P(0, I)=P(0, I)-N

1900 M=M+N*W:PRINT "A "; W; " VENDU "; N

1910 NEXT

L 1920 RETURN

Ⓢ 2000 REM *** FIN DU JEU ****

2010 PRINT "RENTRE SUR TERRE"

2020 PRINT "BOUCLEMENT DES COMPTES"

2030 FOR I=5 TO 10:P(P, I)=P(P, I)*.6:NEXT
:GOSUB 1510

2040 FOR I=5 TO 10

2050 PRINT "SOLDE "; A\$(I+6); ":"; P(0, I)

2060 PRINT "PRIX DU MARCHE "; P(P, I); " * E
N MAIN "; P(0, I); "="

2070 PRINT P(P, I)*P(0, I):M=M+P(P, I)*P(0, I)

2080 IF INKEY\$="" THEN 2080

2090 NEXT:PRINT "ENERGIE RESTANTE:"; INT(E); " EGAL "

2100 PRINT " CONVERTI EN "; INT(E/100):M=M+INT(E/100)

2110 IF D<180 THEN PRINT "RETOUR DANS LES DELAIS: BONUS DE "; 180-D:M=M+180-D

2120 FOR I=1 TO 9

2130 IF P(I, 4)=1 THEN 2160

2140 PRINT "PAS VISITE "; A\$(I)

2150 PRINT "AMENDE "; I^3*10:M=M-I^3*10:BEEP 2*1, 5

2160 NEXT

2170 PRINT "TOUT EST TERMINE":BEEP 1, 10

2180 PRINT "ARGENT="; M

2190 PRINT "JOURS="; D;"BENEFICE="; M-1000
0

2200 REM *** JEU TERMINE ***

L 2210 PRINT "JEU TERMINE" : END

Comment exprimer clairement les ambiguïtés

Comme on l'a vu en "Fais ce que je fais" au chapitre 1, il n'existe pas de mot tel qu' "insinuation" et "approprié" en langage BASIC. Dans un jeu cependant, il est quelquefois désirable de dire "presque". En programmation, ce "presque" doit pouvoir être exprimé clairement. Tentons de le faire à l'aide de la fonction RND.

Français: "X est compris entre 5 et 6, mais il est plus près de 6"

BASIC: $X = 5 + \text{SGN}(\text{INT}(\text{RND}(1) * 10))$

Ainsi $X = 5$ une fois sur 10. Cette probabilité peut être modifiée en changeant la position de "*" 10". Lorsqu'une variable est utilisée, la probabilité change en conséquence.

Traçons un motif.

Français: "Traçons des points sur tout l'écran, en les concentrant surtout en son centre.

```
BASIC: 10 FOR I=0 TO 6.28 STEP .01
        20 N=RND(1)*RND(1)
        30 PSET (SIN(I)*N*60+60,COS(I)*N*16+16)
        40 NEXT
```

Les points sont tracés de telle manière qu'il semble apparaître un système galactique vu sous un certain angle. Les lignes 10, 30 et 40 sont une manière de tracer des cercles et ellipses. Étudiez la manière dont N de la ligne 20 est créé, et cherchez d'autres moyens de réaliser des choses intéressantes.

Par exemple, la formule d'un anneau de nébuleuse:

```
20 N=(RND(1)+RND(1)+RND(1))/3
```

Il existe de nombreuses autres manières de créer N, et certaines sont utilisées dans le jeu de ce chapitre.

"Presque" peut également être exprimé sans utiliser la fonction RND (par exemple, FNS dans ce jeu).

Découvrez d'autres moyens d'exprimer "presque". Cela peut être très amusant.

5.3 Exécution

Le programme demande d'abord si vous désirez faire un achat particulier. Si vous frappez Y, le jeu commence par l'achat de ressources naturelles. Lorsque l'achat est terminé, ou si vous n'avez pas effectué d'achat particulier, le montant d'argent et d'énergie disponible et la position actuelle sont affichés sur l'écran. Le programme demande ensuite sur quelle planète vous désirez vous rendre. Lorsque vous l'avez fait, ses coordonnées sont affichées.

ACHAT PARTICULIER?

M=4700 E=50000 ICI — Argent: énergie

14982 732 — Position actuelle

TERRE. QUELLE PLANETE(1-9)? — Frappez la touche 4.

MARS DONT LES COORDONNEES SONT

-16932 -15271 — Position de mars

Ensuite, le programme vous demande la méthode de calcul de la route. Comme la distance semble supérieure à 100 millions de km, le calcul doit s'effectuer d'abord manuellement.

Si vous frappez (3), vous n'effectuez pas de courbure, et vous cherchez une autre planète.

1=ORDINATEUR:2=MAN:3=AUTRE? — Frappez la touche 3

QUELLE PLANETE(1-9)? — Frappez la touche 2

VENUS DONT LES COORDONNEES SONT

9030 -5925

1=ORDINATEUR:2=MAN:3=AUTRE? Frappiez la touche 2

NAVIGATION MANUELLE

DIRECTION(0-360)? 250 Entrez 250

DISTANCE=? 13000 Entrez 13000

DIRECTION= 250

DISTANCE= 13000

OK?Y Direction h. Distance h') Entrez Y

COURBURE?

Lorsque **Y** est frappé durant la vérification, le vaisseau spatial effectue un changement de direction. Lorsque cela est terminé, les nouvelles coordonnées du vaisseau spatial sont affichées sur l'écran.

M=4700 E=49883 ICI

10535 -11484 Près de VENUS

ESPACE. QUELLE PLANETE(1-9)? Frappiez la touche 2

M=4700 E=49778 ICI

9423 -4443

ESPACE. QUELLE PLANETE(1-9)?

VENUS DONT LES COORDONNEES SONT

9762 -4619

1=ORDINATEUR:2=MAN:3=AUTRE? Frappiez la touche 1

CALCUL DE LA ROUTE POUR VENUS

DIRECTION= 332.50701066549

DISTANCE= 382.18037033549

OK?Y

COURBURE?

Ici à nouveau, le jeu demande sur quelle planète vous désirez vous rendre, et la méthode de calcul de la route. S'il semble que le vaisseau spatial se trouve à moins de 100 millions de km du but, la route est calculée par l'ordinateur.

VENUS DONT LES COORDONNEES SONT

9491 -5153

1=ORDINATEUR:2=MAN:3=AUTRE?

CALCUL DE LA ROUTE POUR VENUS

DIRECTION= 99.367447235642

DISTANCE= 6416.2197709715

OK?Y

COURBURE?

Il arrive que l'ordinateur produise une divergence de calcul qui rendra le jeu encore plus intéressant. Il n'est donc pas toujours nécessaire d'effectuer une courbure jusqu'à la planète de destination en un seul calcul.

Lorsque le vaisseau spatial se trouve à l'intérieur de la gamme d'atterrissage, l'écran est effacé, puis il affiche la planète et le vaisseau spatial. Amenez ce dernier sur la planète à l'aide des touches du curseur. Après avoir confirmé la planète sur laquelle vous vous êtes posé, frappez une touche.

ATTERRI SUR VENUS

Après l'atterrissage, il s'agit de vendre et acheter des ressources naturelles.

VENIRE

OR DONT LE PRIX DU MARCHE EST 109

EN MAIN=10 QUANTITE VENDUE?

2
PRIX UNITAIRE? 106
A 106 VENDU 2

URANIUM DONT LE PRIX DU MARCHE EST 51
EN MAIN= 20 QUANTITE VENDUE?

0
ACHETER
VOTRE ARGENT= 4806

VOTRE OR= 9
OR DONT LE PRIX DU MARCHE EST 109

QUANTITE ACHETEE? 0
ARGENT RESTANT= 4806

VOTRE CARBON= 30
CARBON DONT LE PRIX DU MARCHE EST 20

QUANTITE ACHETEE? 10
PRIX UNITAIRE? 19
VENTE IMPOSSIBLE A CE PRIX

VOTRE CARBON= 30
CARBON DONT LE PRIX DU MARCHE EST 20

QUANTITE ACHETEE? 20
PRIX UNITAIRE? 19
ARGENT RESTANT= 4426

ARGENT RESTANT= 3246
DEPART!

Lorsque vos affaires sont terminées, spécifiez la prochaine planète et partez.

M=1110 E=22143 ICI
-4402 21074
ESPACE. QUELLE PLANETE(1-9)?
TERRE DONT LES COORDONNEES
-8181 12573
1=ORDINATEUR:2=MAN:3=AUTRE?
CALCUL DE LA ROUTE POUR
TERRE
DIRECTION= 246.03430746893
DISTANCE= 9302.949468452
OK? Y
COURBURE?

RENTRE SUR TERRE
BOUCLEMENT DES COMPTES
SOLDE ARGENT: 8
PRIX DU MARCHE 63 * EN MAIN 8=
504 Frappez une touche ici
SOLDE CARBON: 30
PRIX DU MARCHE 18 * EN MAIN 30=
540 Frappez une touche ici

ENERGIE RESTANTE: 21974 EGAL
CONVERTI EN 217
RETOUR DANS LES DELAIS: BONUS DE 56

PAS VISITE SATURNE
AMENDE 2160
TOUT EST TERMINE
ARGENT= 15106
JOURS= 124 BENEFICE= 5106
JEU TERMINE

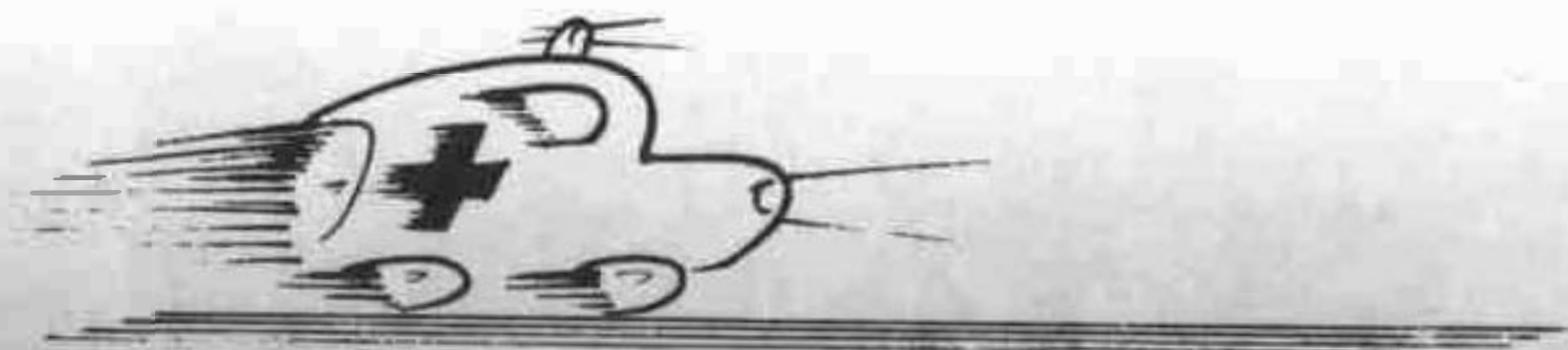
Nouvelles instructions

ON n GOTO	Commande le branchement sur une ligne fonction de la valeur de n.
ATN	Arctangente
SIN	Sinus
COS	Cosinus
ABS	Valeur absolue

Chapitre 6 Localisation des défauts



Lors de la création de programmes complexes, vous buterez souvent sur des problèmes. Si le déroulement du programme est arrêté en raison d'une erreur, reportez-vous à la section concernant les messages d'erreur dans le guide de l'utilisateur. Ce chapitre donne des exemples de programme qui, bien qu'aucune erreur n'ait été commise, présentent des difficultés d'exécution, ou qui contiennent des erreurs difficiles à déceler même en analysant les messages d'erreur. Nous espérons que ce chapitre vous aidera à mettre au point vos programmes.



6.1 Boucle sans fin

Problème: Les mêmes lignes se répètent sans fin, et le programme ne peut passer à la ligne suivante.

• Aucune autre touche que ON/BREAK est acceptée.

Explication: Les instructions du genre 10 GOTO 10 sont hors de question. Les boucles sans fin se produisent normalement lorsqu'un branchement inconditionnel a été rédigé incorrectement.

■ Variable du type incorrect.

```
10 DEFINT A-Z
20 PRINT I
30 I = 1 + 0.1
40 IF I = 1 THEN END
50 GOTO 20
```

Le programmeur ayant aparamment essayé d'incrémenter la valeur de I de 0,1 à chaque fois à la ligne 30, il a déclaré par erreur la variable I comme de type entier. En conséquence, I reste toujours égal à 0. La même chose se produit avec une boucle FOR - NEXT.

```
10 DEFINT A-Z
20 FOR I = 0 TO 1 STEP .1
30 PRINT I
40 NEXT
```

Ces programmes ont été rédigés de façon à ce que les erreurs sautent aux yeux. Cependant, lors de la rédaction d'un programme, on a souvent tendance à n'utiliser que des entiers et à rédiger des expressions telles que celle de la ligne 10, puis celle de la ligne 20 lorsqu'on a besoin de nombres réels. En conséquence, lors de l'utilisation de variables, toujours être attentif à leur type.

■ Variable de contrôle changée de manière incorrecte

```
10 FOR*=0 TO 10
20 I=I B
30 PRINT I
40 NEXT
```

Voici un exemple d'un programme simple, où des erreurs telles que celle de la ligne 20 peuvent néanmoins se produire lorsqu'on inclut de nombreuses instructions à l'intérieur de la boucle FOR - NEXT. Si la variable B est 0, I reste toujours 0, et la boucle ne se termine jamais.

■ Erreur lors d'un sous-programme de traitement d'erreur

```
10 ON ERROR GOTO 50
20 INPUT A
30 PRINT SQR (A)
40 END
50 A = -A
60 PRINT "IMAGINAIRE NON."
70 RESUME
```

Ce sous-programme est destiné à empêcher la génération d'une erreur lorsque la valeur de A est négative pour un calcul de racine carrée. Cependant, lorsque ce programme est exécuté, une boucle sans fin se produit. Ceci est dû au fait que le retour du sous-programme de traitement d'erreur est ordonné par RESUME, de sorte que le programme entreprend sans fin le même processus. Bien que ce programme traite les valeurs négatives grâce à $A = -A$ pour calculer leur racine carrée, ce qui est juste, l'instruction PRINT de la ligne 30 est nettement

un genre de PRINT. Puisque cette erreur de syntaxe est différente de l'erreur prévue (appel de fonction inadmissible en raison de SQR(-)), une boucle sans fin est produite à la ligne 30 sans aucune solution possible. Lors de l'application d'un sous-programme de traitement d'erreur dans un programme, évitez d'utiliser l'instruction ON ERROR à moins que vous soyez certain qu'aucune erreur inattendue est possible. Il est judicieux de spécifier le type d'erreur (voir variable réservée ERR ou ERL) que le sous-programme de traitement d'erreur est appelé à traiter.

De même, la déclaration de l'instruction ON ERROR GOTO est valable même si l'exécution du programme est arrêtée et placée en mode direct. Lorsqu'une erreur est produite en mode direct, le sous-programme de traitement d'erreur est mis en service (à moins que l'exécution ne soit arrêtée par pression de la touche ON/BREAK ou que "Abort" ne soit affiché sur l'écran). Dans un tel cas, exécutez soit ON ERROR GOTO 0, ou un ordre NEW.

6.2 Problèmes d'affichage

Problème: Les valeurs numériques et les chaînes ne sont pas affichées correctement.

Explication: La sortie de caractères et de nombres est une fonction essentielle d'un programme.

■ Problèmes associés au roulement de l'écran

Pour être à même d'afficher le plus grand nombre d'informations sur l'écran, on place un point-virgule après chaque élément à afficher. Cependant, tous les éléments sont affichés en continu (pas de retour chariot ni interligne). Envisageons un programme affichant les informations ligne par ligne, comme pour l'ordre LIST.

Les caractères devant être affichés sont lus dans les variables A\$(0) à A\$(10). Lorsque la touche RETURN est frappée, l'affichage avance d'une ligne.

```
100 FOR I=0 TO 10
110 PRINT A$(I);
120 LINE INPUT I$
130 PRINT
140 NEXT
```

Ou alors, la ligne 120 peut être fixée à IF INKEY\$="" THEN 120, ou après l'affichage de 4 lignes, le roulement peut être arrêté en ajoutant IF MOD 4=0 THEN LINE INPUT I\$. En outre, vous devez également considérer le traitement des cas spéciaux, notamment lorsque A\$(I) est supérieur à 20 caractères (IF LEN(A\$(I)) > 19).

■ Alignement des valeurs numériques et des chaînes

Supposons que des noms de 10 caractères soient stockés dans les variables N\$(0) à N\$(10), et que des numéros de téléphone le soient dans les variables T\$(0) à T\$(10).

```
100 FOR I=0 TO 10
110 PRINT N$(I); TAB (10); T$(I)
120 NEXT
```

Les noms seront affichés à partir du coin gauche, et les numéros de téléphone à partir de la 11^e position sur l'écran. Attention: Même si les données se composent uniquement de chiffres, comme c'est le cas des numéros de téléphone, toutes celles commençant par un "0" doivent être traitées comme des chaînes. Si elles étaient traitées comme des valeurs numériques, les premiers zéros seraient supprimés. Il se peut également que vous deviez traiter des chiffres comme des valeurs numériques, puis

comme des caractères.

L'expression de conversion d'une valeur numérique en caractère est la suivante:

```
A$ = STR$(12)...Contenu de A$ "12"
A$ = STR$(-12)...Contenu de A$ "-12"
```

Comme vous pouvez le voir, le signe moins précède la valeur numérique si elle est négative, et un espace la précède si elle est positive. Si vous savez qu'une valeur numérique sera toujours positive, ou si vous n'avez pas besoin du signe, l'expression

```
A$ = MID$(STR$(X),2)
```

produit des chaînes numériques sans espace, comme dans "12".

De même, comme en PRINT 5:5, lorsque des valeurs numériques sont présentées en série, vous devez penser à créer un espace après chacune d'elles, et deux espaces entre deux nombres.

Si vous désirez afficher un nombre justifié sur la marge droite, utilisez l'instruction PRINT USING.

Exemple

```
PRINT USING "####,."; 156.21256
156
PRINT USING "E###.##*"; 23.567
E23.6*
```

Pour plus de détails, reportez-vous à PRINT USING dans le manuel de référence. Etudiez également la manière d'utiliser les instructions LOCATE et CONSOLE.

6.3 Valeurs numériques

Problèmes: Résultats de calcul inattendus

Production d'un paramètre d'instruction inattendu, résultant en une erreur FC.

Explication: En général, le problème réside dans le type de variable adopté.

■ Erreur de type de variable

```
10  DEFINT K-Z
20  DEFFNX (A,B)=(A + B) * 10000
30  ...
40  ...
```

Dans une fonction définie par l'utilisateur telle que celle-ci, comme `X` est une variable de type entier, `FNX` doit aussi être une fonction de type entier. En conséquence, si la valeur de `A` ou de `B` est trop grande, une erreur OV pourrait se produire.

Voici un exemple de conversion de type.

```
10 A = 10
20 A = A/3
30 A = A*3
40 PRINT G$(A)
```

Le programmeur désire obtenir la chaîne de caractères G\$(10). Cependant, après l'exécution des lignes 20 et 30, la valeur de A est 9,999999999999999 selon la structure du basic. A la ligne 40, A est utilisé comme indice (toujours entier) d'une liste, et est tronqué à 9. Comme vous pouvez le voir, il est nécessaire de se rappeler, lors d'utilisation de variables ou paramètres de type entier, que les décimales sont tronquées.

■ Répétitions et autres erreurs de noms de variables

```
10  PROG = 10
20  PRICE = 20
30  SUM = PRICE * PROG
```

Dans ce programme, la variable PROG et la variable PRICE sont traitées comme la même variable. Les variables étant toujours identifiées par leur deux premiers caractères, le programmeur doit être prudent lorsqu'il utilise des longs noms.

■ Erreur dans l'ordre de calcul

```
10 I=9-5 MOD 3
20 PRINT I
```

Ici, l'opération MOD qui est dotée de la priorité de calcul est calculée avant la soustraction. De ce fait, $5 \text{ MOD } 3 = 2$, $9 - 2 = 7$, ce qui produit la réponse $I = 7$. Il est préférable d'utiliser des parenthèses telles que $(9 - 5) \text{ MOD } 3$ ou $9 - (5 \text{ MOD } 3)$ afin d'éviter ces erreurs.

6.4 Autres erreurs difficiles à déceler

Problème: Il semble n'y avoir aucune erreur sur la ligne dans laquelle l'erreur SN s'est produite.

Explication: Type incorrect de donnée lue par l'instruction READ.
Dans l'exemple ci-dessus, une erreur SN est produite à la ligne 30.

```
10 READ A
20 PRINT A
30 DATA B
```

■ Autre exemple d'erreur lors d'utilisation d'une fonction définie par l'utilisateur.

Dans le prochain programme, l'erreur SN se produit en ligne 20.

```
10 DEFFNA (B,C)= 5*(B,C)
20 A=FNA (2,4)
30 PRINT A
```

L'erreur se produit réellement sur la ligne 10, dans laquelle la fonction est définie. Les erreurs difficiles à déceler lors de la rédaction d'un programme comprennent notamment la confusion de l et I, "O" et "0" et ";" et ":", ainsi que les erreurs de placement de virgules. Deux lignes reliées ensemble parce que la touche RETURN n'a pas été frappée à la première est une erreur courante encore plus difficile à trouver. Un examen attentif à l'aide de l'ordre LIST devrait permettre de les découvrir.

```
LIST 20
20 INPUT R:A1 = COS (R)
30 BEEP 4, 10
```

Ici, la ligne 20 et la suivante sont assemblées

A1 = COS(R) 30 BEEP 4,10

Problème: Le programme ne se déroule pas correctement, est interrompu par les erreurs RG, NF ou FC.

Explication: Un programme rédigé de façon maladroite produit souvent des erreurs difficiles à déceler.
En voici quelques exemples:

■ Instruction ON - GOTO

```
10 INPUT N
20 IF N>3 THEN 10
30 ON N GOTO 100, 200, 300
40 REM BEEP SUB
50 BEEP H, S:RETURN
100 H=10:S=10:GOSUB 40:GOTO 10
200 H=20:S=20:GOSUB 40:GOTO 10
300 H=30:S=30:GOSUB 40:GOTO 10
```

Ce programme peut sembler étrange, mais des passages tels que celui-ci peuvent se produire dans les longs programmes. Le problème réside dans le fait que l'exécution passe de la ligne 30 à la ligne 40 lorsque N=0. Il en résulte qu'une erreur RG est produite lors du passage à RETURN de la ligne 50. Si le sous-programme commençant à la ligne 40 est trop long, il est quelquefois difficile de trouver son point de départ. Pour l'éviter, examinez la possibilité de passage à la ligne suivante après l'instruction ON - GOTO, et assurez-vous toujours que le sous-programme est précédé de END, STOP, RETURN ou GOTO.

■ Erreur pour laquelle SN Error n'est pas affiché.

```
10 A$="ABC":FOR I=0 TO 5
20 BEEP I,2:NEXT
30 PRINT A$
```

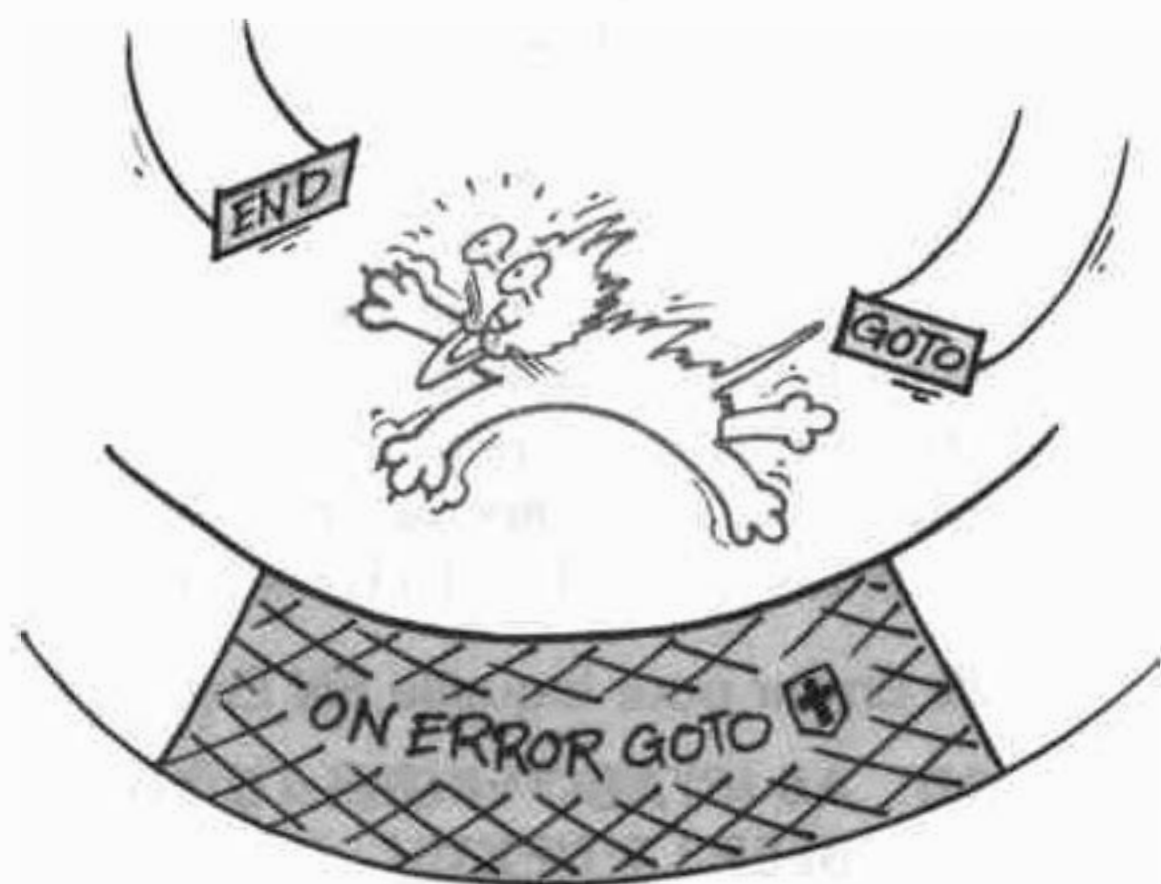

ici, "ABC:FOR I=0 TO 5" est affecté à A\$ à la ligne 10, puis l'exécution passe à la ligne 20, produisant SN Error. Lorsqu'il existe plusieurs instructions entre les lignes 10 et 20, une erreur telle que celle ci-dessus nous échappe facilement et est difficile à découvrir. Il est préférable d'éviter les lignes à multi-instructions inutiles, et d'adopter des instructions simples telles que FOR-NEXT, GOSUB - RETURN, etc. Des erreurs semblables peuvent être commises avec l'apostrophe ('). Par exemple:

10 PRINT A:UN EXEMPLE

ne constitue pas un SN Error, mais est traité comme

10 PRINT A:REM UN EXEMPLE

Si cette expression était suivie de l'instruction RETURN ou NEXT, le déroulement du programme serait désespérément embrouillé.



Canon CANON INC.

7-1, 2-chome, Nishi-shinjuku, Shinjuku-ku, Tokyo 160, Japan
P.O. Box 5050, Shinjuku Dai-ichi Saimai Building, Tokyo 160, Japan

CANON U.S.A., INC.

HEAD OFFICE One Canon Plaza, Lake Success, N.Y. 11042, U.S.A.

CHICAGO 140 Industrial Drive, Elmhurst, Illinois 60126, U.S.A.

LOS ANGELES 123 Paulerino Avenue East, Costa Mesa, California 92625, U.S.A.

ATLANTA 6380 Peachtree Industrial Blvd., Norcross, Georgia 30071, U.S.A.

DALLAS 2035 Royal Lane, Suite 200, Dallas, Texas 75229, U.S.A.

CANON CANADA INC.

HEAD OFFICE 6390 Dixie Road Mississauga, Ontario, L6T 1P7, Canada

CALGARY 2828, 18th Street, N.E. Calgary, Alberta, T2E 7K7, Canada

MONTREAL 10652 Côte de Liesse, Lachine, Quebec, H6T 1A5 Canada

CANON EUROPA N.V.

P.O. Box 7907, 1008 AC Amsterdam, The Netherlands

CANON FRANCE S.A.

DIVISION CALCUL 93154 La Blanc Mesnil, Cedex, France

CANON RECHNER DEUTSCHLAND GmbH.

Fraunhoferstrasse 14, Postfach 8033, München-Marlried, West Germany

CANON UK LTD.

Belgrave House, No 1 Manor Road, Wallington, SM8 0BW, England

CANON LATIN AMERICA, INC.

SALES DEPARTMENT P.O. Box 7022, Panama 5, Rep. of Panama

REPAIR SERVICE CENTER P.O. Box 2019, Colon Free Zone, Rep. of Panama

CANON HONG KONG TRADING CO., LTD.

Golden Beer Industrial Centre, 7th Floor, 66-82 Chei Wan Kok Street, Tsuen Wan, New Territories, Hong Kong

CANON AUSTRALIA PTY. LTD.

1 Hall Street, Hawthorn East, Victoria 3123, Australia